PUREMETAL FORGE

The Anvil Guide

Build it, install it on a Raspberry Pi 4, write payloads for it, and use all four cores.

Generated 2026-10-25 · tools/build_help_pdf.py

Contents

What Anvil is

- The one-sentence version
- What "resident" means, and why it is the whole design
- The three things Anvil is, at once
- The shape of the source
- Which boards, and how far each one has got
- What runs where
- What a payload is
- What a module is
- The memory map is measured, not declared
- The first boot
- What this book covers, and in what order
- One last thing, about how this project talks

Building Anvil

- What you need
- Telling the build where the compiler is
- The targets
- What the build actually runs
- What you will see
- The build number, and why it moves every single time
- What lands in build/
- Building the EL3 stub, which is not part of all
- Verifying the tree before you hand it to anyone

Installing Anvil on a Raspberry Pi 4

- What this chapter assumes
- The short way: the skeleton in the repository
- What goes on the boot volume
- config.txt
- Wiring the serial console

First boot: what you will see

Two things about that banner

The network console

How the board gets an address

Two commands to run first

The EL3 stub, and the reason to be careful

Keep a way back

The console

The prompt

Messages never land on the command line

Every command is a whole word

help

map

version

Looking at memory

memory is for a person, readback is for a machine

sha256sum, and the hex-digit rules

Files on the boot medium

What `usb tree` says about the link under the stick

`usb link` - the same link in raw registers

`measure` - the board times its own commands

`usb devices` - every device, as it describes itself

Where the USB stack spends its time: `usb timing`

What `usb storage` says about the stick

save

Getting an image in over the network

run, go, and boot

Programs on the medium

The deadman

What happened to the last payload

The boot trail

Settings that survive a power cycle

What SETTINGS.TXT looks like on the medium

Network and radio

Who the network console belongs to

reset

The board itself

info, and how much memory this board has

There is no loop command, on purpose

Writing a payload

What a payload is

The three addresses

The two switches

What you may assume at your first instruction

The service table

The version gate, and it is not optional

The groups

The lifecycle slots you will actually use

SvcRequireCap returns the sentence; it does not print it

Errors are sentences, and you compose them

The deadman

What a payload must not do

Returning a value

Where your payload may go

Staging assets at addresses

A complete payload

Building it

Running it

A checklist before your first run

Optional Pi4 Bluetooth transmit buffering

Running payloads from the host

The tools, and what each is for

Ask the board where to put things

One complete run

What it leaves behind, while it is still running

A refusal is an answer, and it arrives at once

When nobody is listening on the console port

A long silence, and the host's firewall

A missing return line is asked about, not believed

What a run actually does, in order

Step 3 is the one that matters

What a refusal looks like

Every run comes back with a picture

The two checks that make a picture evidence

One command for a model run

Uploading without running

The negative control

Reading things back

How fast a transfer goes, and the rule behind it

Screenshots on their own

What none of this proves

Using all four cores

What the service does, in one paragraph

The three slots

Ask for the count at the right moment

Partitions are yours, and that is the point

The floating-point control register must agree

Do not flush caches at the join, and this is not a performance note

What your worker procedure may not do

The five-second bound

When a core wedges

The full list of refusals

coretest: proving the cores before you trust them

The lifecycle limit you will hit, and it is still open

One more thing, if your four-core run is long

Updating the monitor in place

The idea

The sequence

Step 2, and why it is step 2

The baud rate is left alone

What you will see

Stop one: the save did not complete

Stop two: the directory disagrees

Prove it harder, when it matters

Recovery, in the order to reach for it

The guard that makes a boot loop impossible

Why a payload cannot write the two counters

What this chapter is not

Driver modules

What a module is meant to be

Read this before you plan anything around it

The container

What is validated, and when

The board's half of the contract

Building one

The manifest

The commands

The lifecycle states

The example driver

The ABI boundary a module may not cross

What is proven

What the modules are for, eventually

Proving things

The shape of the argument

The house rule
The interpreter
Running a gate
The one gate every lane runs
A gate that examined nothing has not passed
The template
Adding a check
The mutants are what make it a gate
The last paragraph of every gate
And then the board
Pi4 optional Bluetooth TX queue evidence
Exact shared SBC analysis movement and matrix reduction

Where the engineering notes are

Why this chapter exists
Start here
Building and identity
Payloads, services and the console
Cores
EL3 and boot firmware
Driver modules
Networking
USB, PCIe and storage
Display, touch and graphics
Other boards
Related project notes
Where this book lives, and when it has to move
The one habit worth copying

Anvil on a Raspberry Pi 3

What this board runs
Direct firmware entry
Startup and storage

Building and checking

Replacing the boot files

TrueType font container

The memory contract

Format references

TrueType font metrics

The four required tables

Optional OS/2 metrics

The reader contract

Gate and independent fonts

TrueType character mapping

Mapping contract

Emitted proof

Format references

TrueType outlines

The outline contract

Emitted and independent proof

Format references

TrueType coverage rasterization

The raster contract

Emitted and independent proof

Format references

TrueType pair positioning and line layout

Open and close order

Pair values and fallback

UTF-8 layout

Emitted proof

Format references

TrueType console commands and defaults

Load and select a font

Size and saved defaults

Limits and proof

References

TrueType fonts in the resident Vulkan console

Try a checked font

The atlas cache

Ownership and safe failure

What has been proved

Source and proof references

Anvil on the ROCK Pi 4C

Boot and recovery

Files on SD and eMMC

Display and graphics

Raspberry Pi 5: source bring-up

Current boundary

Check the source device trees

Vulkan on a Raspberry Pi 4

Rendering and transfers

The supported image readback

Build and proof boundary

Bounded partial optimal-to-optimal edge-tail copy

Transactional Neon atlas generations

Combined Neon draw records

Every command

Every board

Raspberry Pi 4 adds

What Anvil is

A monitor that stays resident while your program runs, owns the machine's physical services, and lends them back through a versioned table.

The one-sentence version

Anvil is a bare-metal mini-kernel, bootloader and recovery monitor. It boots a board with no operating system underneath it, gives you a prompt, and can load and run your program while remaining in memory the whole time.

It is MIT licensed, and the source is the whole of it - no vendor toolchain, no runtime library, no second hardware stack hiding underneath.

What "resident" means, and why it is the whole design

The usual arrangement for a bare-metal program is that it owns everything. It brings up its own clocks, its own serial port, its own storage, its own network. Two such programs on one board have two copies of every driver, and neither can help you when the other one hangs.

Anvil is the other arrangement. It **stays in memory while your program runs.** It owns storage, interrupt control, clocks, console paths, network links, display, DMA and watchdogs, and it exposes them through a versioned table of function pointers instead of making every program bring a second copy of the hardware.

Your program - a **payload** - can therefore be pure computation, or computation plus a few service calls, and it does not have to know how any of the peripherals work.

> **Note.** The practical consequence is the one you will feel first. When

your payload finishes, there is still a monitor there to tell you what happened, take a screenshot of the frame you left behind, and let you type the next command. When it hangs, there is still a watchdog to reset the board and a prompt waiting afterwards. You do not have to power cycle to iterate.

The three things Anvil is, at once

A monitor. It gives you a prompt on a serial line and over the network: read and write memory, hash a region, list and load files, look at the memory map, drive GPIO and I2C, take a picture of the screen. Chapter 4 is the command set.

A bootloader. It can load a program from the boot medium or from memory, verify it against its own digest, place it where it was built to run, and enter it - automatically at power-up if you ask it to, with a loop guard so a payload that keeps hanging cannot take the prompt away from you.

A mini-kernel. It publishes services to a running payload through a table, and it takes the machine back afterwards - restoring the devices the payload did not claim, including rebuilding a network controller without losing its addresses or its unexpired lease.

The shape of the source

One portable core, one hardware composition per target:

```
Anvil/Core/*.pbi        the portable monitor: command loop, parsing,
                        settings, transfer and boot logic, hashes,
                        network orchestration
Anvil/Hal/*.pbi         the hardware contract and the versioned ABIs
Anvil/Kernel/*.pbi      AArch64 architecture, not any chip: the
                        exception vectors and the scheduler's contexts
Anvil/Storage/*.pbi     FAT32, exFAT, the one path layer over them,
                        the file seam any block-device board answers
                        with, and the A/B boot-control record
Anvil/Graphics/Vulkan/  an experimental Vulkan-compatible foundation
RaspberryPi4/Board/     the BCM2711 memory map, boot order, console,
                        storage, display, board policy
RaspberryPi4/Lib/       the BCM2711 drivers
ArduinoQ/, RaspberryPi3/, RockPi4C/    the other targets, the same way
```

The core calls portable names. A board owns the choice of hardware and the implementation behind those names. Shared code uses `.pbi`, which carries no target identity; Pi 4 hardware code uses `.pi4`, and the other boards their own extensions.

A LIBRARY EXISTS ONCE. Boards differ in the interface under it, not in a second copy of it. If a file has no chip in it, it belongs in a shared folder with a `.pbi` name, and every board includes the same one.

Every one of those interfaces is declared in `Anvil/Hal/hal.pbi`, including the two that were real and undeclared until 2026-09-18: the console bytes a board prints and reads through, and the monotonic clock it counts in. A board that wants a prompt implements the first; a board that wants a timestamp implements the second; neither invents a name.

> **Note.** The extension is not a label, it is enforced. Including a `.pi4`

file from a `-t pi3` build is refused by the compiler, by name, at the first cross-chip include: "If it is really for the BCM2837 (Raspberry Pi 3 Model B): include that chip's own libraries instead." That refusal is right - it stops a board collapsing into dozens of undeclared-register errors inside a library nobody meant to include. It also means a file that is perfectly portable is UNREACHABLE from another board while it carries a board's extension. Architecture-neutral code therefore has to be moved and renamed to be shared at all; leaving it where it is guarantees the second board writes its own.

> **Note.** Target-specific names deliberately do not leak into the portable

contract. That is not tidiness - it is what makes a new board a matter of writing one composition rather than forking a monitor.

Which boards, and how far each one has got

```
Raspberry Pi 4 / BCM2711 / Cortex-A72
    the most mature target, and what this book is about. Booting with
    caches enabled, HDMI and native DSI output, DMA-backed console
    rendering, wired and Wi-Fi console access.

Arduino UNO Q / QCM2290 / Cortex-A53
    early. A UEFI application. Console, file access and GPIO exercised
    on hardware; most native drivers, networking and display work
    outstanding.
```

```
Raspberry Pi 3 / BCM2837 / Cortex-A53
    early. Direct firmware cold entry through a Pi 3 EL3 stub into the
    complete kernel8.img monitor. USB host and USB Ethernet not brought up.

ROCK Pi 4C / RK3399
    early. MiniDP proven on hardware at 1024x768; the preferred 1080p
    mode is not working. A UART recovery console is proven. No network
    command service.
```

Trap. Read that table as a statement about **today**, and check the

README before you plan around any row of it. This project's own habit is to write down the boundary rather than the achievement, and a status line in a book goes stale faster than one in a repository.

What runs where

AArch64 processors have privilege levels numbered EL0 to EL3, with EL3 the most privileged. What matters to you is a short list.

The firmware runs first, before anything in this repository. It reads `config.txt`, sets up clocks and power domains, and loads `kernel8.img` to the address it was told to.

Without the firmware stub, the firmware can drop the image to **EL2**. Anvil refuses that entry level; this is not a supported boot arrangement.

With the Pi 4 firmware stub installed, its AArch64 `armstub8.bin` runs first at EL3, sets up the Pi 4 machine state and enters that monitor at EL3. The Pi 3 uses a separate stub: it starts at EL3, configures the BCM2837 timer and SMP state, parks the other cores, and branches to its complete `kernel8.img` while remaining at EL3. It passes the firmware DTB in x0. Both supported Pi boot paths keep Anvil at EL3.

A payload is entered at the same level Anvil is running at. Anvil does not change the level to enter you and you must not change it either.

Trap. Never hard-code the exception level in a payload. Anvil requires

EL3 on every supported monitor. A payload should still use the service table for hardware services instead of taking ownership of board registers.

Note. The Pi 4 stub remains a separate boot artifact; the Pi 3 `pi3`

target builds its own stub together with the direct monitor. Boot configuration and replacement procedures are board-specific; see chapters 3 and 12 before changing either one.

What a payload is

A compiled image with two properties: it was built to **return**, and - if it wants services - it declared that it wants the service table.

Anvil places it at the address it was linked for, verifies its SHA-256 **at that address**, zeroes its BSS, switches to its stack, masks interrupts, and branches into it with the service table's address in `x0`. When it returns, Anvil prints all sixteen hex digits of `x0`.

```
Starting the payload at 00500000. Control leaves the monitor now, and
anything printed below this line came from the payload, not from Anvil.
```

```
The payload returned to the monitor. Its x0 register held 000000000000000A,
which is whatever it chose to return. A stock image never returns at
all, so seeing this line means the payload was built to come back.
```

The contract that goes with it is short. The payload may use its declared image, BSS and stack, the processor state it was handed, and devices it explicitly claims. It must preserve Anvil's image and BSS and must not reconfigure unrelated devices. Chapter 5 is that contract in full.

> **Note.** The service table is **call-and-return only**. Anvil never calls

back into a payload: no callbacks, no handlers, no "register a function and I will call it". Where a service looks like it wants a callback it provides a **queue** instead - which is why touch and vehicle input are polls. The reason is that Anvil's drivers keep their state in globals with one writer and no lock, and a callback would re-enter them on top of a call already in progress.

What a module is

A payload borrows the machine. A **module** extends the monitor.

A module is a driver in a file - a `.MOD` container on the boot medium, verified against its own digest, placed in a reserved area, relocated, matched to a device the board declares, and asked to publish the service it implements. The point is that a board which does not have a particular piece of hardware should not be carrying its driver.

> **Trap.** The published state of that work is **placement infrastructure

only**: the container format and the relocator exist and have executable tests, and neither board activates the engine. Chapter 9 says exactly what exists and what does not, because this is the one part of Anvil where it would be easy to read a feature list and plan around something that is not there yet.

The memory map is measured, not declared

Anvil reads its own image-extent symbols at run time. Its code reservation is the live image rounded up to a megabyte, and the first payload window begins at the next boundary above that.

So **the addresses move when the monitor does.** The low payload window started at `00400000` on one build and `00500000` on the next, for no reason except that the image grew.

```
pmf> map
THE MEMORY MAP, AS THIS IMAGE MEASURES IT
  this image      00200000 to 004B144B   2823244 bytes, measured from the image itself
  ...
```

```
  stage a file at 00500000              which is what the upload tools ask for

Where a payload may go:
  the low window   00500000 to 07EFFFFF   (about 122 MB)
  the high window  40000000 to FBFFFFFF   (about 3 GB)
  Nothing in this map is a constant this program was told.
```

Trap. Every host tool asks the board for that address instead of

carrying one. Four tools used to carry `0x400000` as a constant of their own - four copies of one number on the far side of a cable, none of which could find out it had changed. **Do not copy an address out of a note, an example, or this book.** Type `map`.

The monitor also refuses writes into the regions it owns - its code, its variables, the autoboot record, the framebuffer, the secondary-core stacks, the module arena, the boot phase record, the screenshot area - so `write`, `fill` and `copy` cannot reach them by accident.

The first boot

What you will see. These lines are from a recorded board session of 2026-09-05, which was an EL2 boot:

```
PureMetal Forge - Anvil, the Raspberry Pi 4 monitor. Version 2.0,
64-bit AArch64, running with no operating system underneath it.
The serial port is the PL011 at FE201000, set up from cold to 115200 baud,
eight data bits, no parity, one stop bit.
The processor is a Cortex-A72 in AArch64 at exception level 2.
This monitor occupies 00200000 onwards, with its stack growing down
from 00100000 and its variables at 0D000000.
A payload may be loaded into either of these:
  the low window   00400000 to 07EFFFFF   (about 123 MB)
  the high window  40000000 to FBFFFFFF   (about 3 GB)
The tick counter every timeout here is measured on runs at 54000000 Hz.
Type info to see what board this is, or help for the list of commands.
```

Note. The exception level in that line is read from the processor, not

printed as prose. This transcript is a historical EL2 session. Hardware records later confirm EL3 startup through the required firmware stub; verify the live board level with `info`.

What this book covers, and in what order

```
2   building it
3   putting it on a Raspberry Pi 4
4   the console
5   writing a payload
6   running payloads from a host machine
7   using all four cores
8   replacing the monitor in place
```

```
9   driver modules
10  how things are proven here
11  a map of the engineering notes
```

If you have a board in front of you and want a prompt today, read chapters 2 and 3 and stop. If you have a prompt already and want to run your own code, read 5 and 6.

One last thing, about how this project talks

You will notice that the monitor's messages are long. A refusal is a complete sentence, it keeps the numeric code, it says what the state now is, and it names the first thing to check.

That is deliberate and it is the project's whole philosophy in one line: **a loud error beats a silent wrong answer, always.** A program that refuses is fixable. One that quietly does the wrong thing costs somebody a week.

You will also notice that the documentation says what has **not** been proven, in the same breath as what has. Chapter 11 is a map of seventy notes that mostly exist to mark a boundary. Trust that habit; it is the most useful thing in the repository.

Building Anvil

One program compiles it, one script drives that program, and every build moves a number that the board can show you.

What you need

Three things, and only three.

A checkout of this repository. Python 3, which the build wrapper and every check script are written in; they use nothing but the standard library. And the PureMetal application, which is the compiler.

The compiler is not in this repository and never will be. It is a separate, freely downloadable program, and this repository deliberately does not carry a copy so that nobody has to wonder which copy is the real one. The README points at where to get it.

> **Note.** The editor and the compiler are one program. `PureMetalForge.exe`

opens a window when you double-click it and compiles with no window at all when you pass `--compile`. There is no separate command-line compiler to install, and there is no second toolchain hiding underneath it - no assembler you must fetch, no linker, no runtime library. The one program reads the board source and writes the raw image the board runs.

Telling the build where the compiler is

You have three ways, and they are tried in this order: the `--compiler` option, then the `PMF_COMPILER` environment variable, then a search of your `PATH` for `PureMetalForge.exe`, `PureMetalForge.linux` and `PureMetalForge`.

There is no built-in default path. If none of the three finds anything, the build refuses and tells you all three ways in one sentence. That is what you will see:

```
$ python tools/build.py pi4
PureMetal compiler not found. Put PureMetalForge on PATH, set
PMF_COMPILER, or pass --compiler /path/to/PureMetalForge.exe. The same
application is the IDE and the command-line compiler; --compile selects
the build.
```

(That refusal really is one long line; it is wrapped here to fit the page. Every other transcript in this book is unwrapped.)

Pick whichever of the three suits you. On PowerShell:

```
$env:PMF_COMPILER = "C:\PureMetal\PureMetalForge.exe"
python tools/build.py all
```

On a shell where `python3` is the spelling:

```
python3 tools/build.py all --compiler /path/to/PureMetalForge.exe
```

> **Trap.** A retired console compiler called `pmfc` used to exist. It is

gone, and the build refuses it **by name** - if you point `--compiler` or `PMF_COMPILER` at anything whose filename stem is `pmfc`, the build stops rather than using it. This is deliberate. An old `pmfc` on a machine still runs and still produces an image, and that image is built by a compiler nobody is maintaining. Do not go looking for one.

The targets

Run `python tools/build.py --help` and the list is the first thing you see. This is the whole of it:

```
$ python tools/build.py --help
usage: build.py [-h] [--compiler COMPILER]
                [{all,pi3,pi4,unoq,pi3-stub,pi3-monitor,pi3-loader,pi3-updater,armstub,rockpi4c}]

Build Anvil targets with the PureMetal application in command-line mode. The
editor and the compiler are one program: `PureMetalForge.exe --compile` builds
without opening a window. Point at it with --compiler or PMF_COMPILER.

positional arguments:
  {all,pi3,pi4,unoq,pi3-stub,pi3-monitor,pi3-loader,pi3-updater,armstub,rockpi4c}

options:
  -h, --help            show this help message and exit
  --compiler COMPILER   path or command name of PureMetalForge (or set
                        PMF_COMPILER); it is run with --compile
```

There are exactly two options and one optional target name. The target defaults to `all`.

What each target builds, and where it lands:

```
pi4             RaspberryPi4/Board/board.pi4    -> build/pi4/anvil.img
unoq            ArduinoQ/Board/board.unoq       -> build/unoq/anvil.img
pi3-stub        RaspberryPi3/Board/armstub8.asm -> build/pi3/armstub8.bin
pi3-monitor     RaspberryPi3/Board/board.pi3    -> build/pi3/kernel8.img
pi3-loader      recovery-only legacy A/B loader -> build/pi3/recovery/kernel8.img
pi3-updater     recovery-only legacy A/B updater -> build/pi3/recovery/anvil.img
                                                +  build/pi3/recovery/anvil-slot.img
armstub         RaspberryPi4/Board/armstub8.asm -> build/pi4/armstub8.bin
rockpi4c        RockPi4C/Board/board.rockpi4c   -> build/rockpi4c/anvil-flat.img
                                                +  build/rockpi4c/Image
pi3             an alias for pi3-stub and pi3-monitor
all             pi4 and unoq, and nothing else
```

> **Note.** `pi3` builds the direct firmware set: a Pi 3-specific `armstub8.bin`

and the complete `kernel8.img` monitor. The separately named legacy loader and updater targets are recovery artifacts; neither participates in normal firmware cold boot. The monitor's load, BSS and stack placement is declared in `board.pi3` and checked against the emitted PMF metadata.

> **Note.** `all` means the two shipping monitors, not everything in the list.

Pi 3 and Pi 4 firmware stubs are never built by `all` and are built only by their explicit board targets. Building boot firmware as a side effect of typing `all` would be exactly the kind of quiet surprise this project refuses to ship.

If you only care about the Raspberry Pi 4 - which is what most of this book is about - one target is all you need:

```
python tools/build.py pi4
```

What the build actually runs

No magic and nothing hidden. For the Pi 4 the wrapper runs this, from the repository root:

```
PureMetalForge.exe --compile RaspberryPi4/Board/board.pi4 -t pi4 -o build/pi4/anvil.img
```

Two details about that invocation are worth knowing, because they are what stop a build from being quietly wrong.

First, the wrapper sets `PMF_ROOT` to this checkout. Every include in this repository is written relative to the repository root, so pinning the root means a missing file **fails** instead of being silently borrowed from some other PureMetal installation on the same machine. A build that quietly picked up last month's driver from a different folder would be the worst kind of bug: it compiles, it runs, and it is not your source.

Second, the wrapper copies the compiler into a temporary directory next to a copy of this repository's `Boards/` profiles before running it. The compiler resolves board profiles beside its own executable, outside `PMF_ROOT`, so staging it is the only way to guarantee that the target profile used is this repository's.

You will also notice there are no address flags on the Pi 4 or UNO Q command lines. Those boards declare their own load, BSS and stack addresses in their source. The Pi 3 diagnostic and the ROCK Pi 4C pass them on the command line instead. Chapter 5 explains what those three addresses mean, because a payload you write will have to state all three.

What you will see

Three lines for a clean Pi 4 build, in this order: the announcement, the size, and the build number.

```
Building pi4: RaspberryPi4/Board/board.pi4 -> build/pi4/anvil.img
Built build/pi4/anvil.img (2930784 bytes).
  #ANVIL_BUILD raised from 141 to 142 in board.pi4 after a successful build; build date set to 20260915 and
build time to 221311.
```

Those are the wrapper's own three messages with real values filled in; they are not a screenshot of one session, and the byte count and the numbers are from this repository's build directory and its ledger rather than from a run made while writing this chapter. The wording is exact.

A failure is a full sentence and it tells you the state you are left in:

```
The pi4 build failed with exit code 1. Its build identity and prior artifacts were preserved.
```

> **Note.** "Prior artifacts were preserved" is a promise, not a hope. Each

build compiles to a private staged filename and only moves the result into place at the end, keeping a backup while it does. A compiler crash, a full disk or a failed hash leaves your previous working image exactly where it was. You never end up with half an image at the name you were about to flash.

The build number, and why it moves every single time

Type `version` on a board and it prints a number. That number exists so that a person can tell which image is running without hashing anything - and that only works if the number moves with **every** build, including the ones nobody meant to keep.

It lives in the board source itself, as an ordinary trailing comment on a constant:

```
#ANVIL_BUILD = 142        ; pmf:build
#ANVIL_BUILD_DATE = 20260915   ; pmf:builddate
#ANVIL_BUILD_TIME = 220310     ; pmf:buildtime
```

`tools/build_count.py` is the one mechanism that moves it. Every tool in `tools/` that compiles a real board file calls it after a compile it has proven succeeded. It works out which real board file the compiled path was a build of - a temporary copy included - raises the number by exactly one, rewrites the date and time beside it, and appends one line to `build/BUILDS.log` carrying the image hash and the name of the tool that asked.

You can read the current state at any time. This is a real desk run:

```
$ python tools/build_count.py
pi3: 7  (RaspberryPi3/Board/board.pi3)
pi3-loader: 9  (RaspberryPi3/Board/loader.pi3)
pi3-updater: 12  (RaspberryPi3/Board/updater.pi3)
pi4: 142  (RaspberryPi4/Board/board.pi4)
unoq: 53  (ArduinoQ/Board/board.unoq)
rockpi4c: 80  (RockPi4C/Board/board.rockpi4c)
ledger: build/BUILDS.log - 221 recorded builds
```

A ledger line looks like this:

```
2026-09-16T03:13:44Z target=pi4 build=142 stamp=20260915-221311 board=RaspberryPi4/Board/board.pi4
source=RaspberryPi4/Board/board.pi4 image=_work/flash-candidate-build142-20260915/anvil.img
sha256=ec9abb8418344b38b3638b6d6980e0e7c0d81ba52eb967bbbb6b47c0a25ce439
compiler=06efee6763efb53ae5a2ca325b367ce9cbb8cb80680d76d491d82de44120b740 by=tools/build.py
compile=4ef69b3f86436462
```

The `stamp` is the local date and time the board will print. The `sha256` is of the image. The `compiler` field is the hash of the compiler executable that produced it, so an image can be traced to the exact compiler build. `by` names the tool. `compile` is a short fingerprint of the compiled source, which is how the same compile is never counted twice.

> **Trap.** There is **no way to build without counting**, and that is on

purpose. Gate scripts compile the monitor too - often ten or more times in a single run, into temporary directories - and for a while those were invisible. Ten images went out over one evening all claiming to be the same build, which makes `version` a lie and makes a board impossible to identify. The compiler's own bump flag was the cause: it bumped whatever file it was

handed, which for a gate is a throwaway copy. Nothing passes that flag any more, and the opt-out that used to exist is gone. A verification build is still a build.

There is exactly one way to hold the number still, and it is for a release, where the number is supposed to stand still:

```
#ANVIL_BUILD = 142        ; pmf:build off
```

The build still happens and is still written to the ledger; the line just carries `frozen=yes` and the number does not move.

> **Note.** `build/BUILDS.log` is not committed. The durable count is the

marker line in the board source, which is; the ledger is local evidence about this machine. A mechanism nobody checks is a mechanism that rots, so there is a gate for it: `python tools/build_count_check.py` runs twelve named checks and eight deliberate sabotages, each of which must be rejected by the check that owns that property. Chapter 10 is about how that style of gate works.

What lands in build/

`build/` is ignored by Git. Nothing in it is source and everything in it can be reproduced by building again.

For each target you get the image plus a set of sidecar files the compiler writes beside it:

```
anvil.img            the raw binary the board executes
anvil.img.pmf        the same image inside a PMFBOOT container
anvil.img.sym        flat symbol table, one name=address per line
anvil.img.sym.meta   per-symbol size, kind, scope and origin
anvil.img.dbg        address -> source file and line
```

There are no `.elf` files and no linker map, because there is no link step. The compiler writes the flat image and these sidecars directly.

The `.pmf` file is the one to understand, because chapters 5 and 6 both lean on it. It is the image with a fixed-size header in front carrying the load address, the entry address, the BSS address and length, the stack address, flags, and a SHA-256 of the image bytes. A loader can therefore place and enter the image correctly without being told anything out of band - the container says where it goes.

> **Trap.** The build treats a missing `.pmf` as an incomplete build and

refuses to publish anything:

```
The pi4 compiler wrote an image without its PMF container. Nothing was published and the build number was not
consumed.
```

An image with no container is a file that somebody will later have to guess the load address for, and guessing a load address on this machine is how you overwrite a running monitor. Better to fail now.

Building the EL3 stub, which is not part of all

The optional Pi 4 firmware stub is 512 bytes of AArch64 that runs before Anvil does. It is built on its own and it is **not installed by any build**:

```
python3 tools/build.py armstub --compiler /path/to/PureMetalForge.exe
python3 tools/a64/a64_el3_check.py --compiler /path/to/PureMetalForge.exe
python3 tools/a64/el3_runtime_emitted_check.py --compiler /path/to/PureMetalForge.exe
```

That produces `build/pi4/armstub8.bin`. It is the only target whose output has no `.pmf` container, because it is not a container - the firmware loads it raw at a fixed address.

Chapter 3 says what the stub does, why you might want it, and the specific reason not to install it unless you can physically recover the board.

Verifying the tree before you hand it to anyone

Two commands, and the difference between them matters:

```
python3 tools/verify_export.py --working-tree
python3 tools/verify_export.py
```

The first checks your complete non-ignored working tree. The second checks the exact set of files Git is tracking - which is the set another person would actually receive. The verifier checks repository boundaries, required licence notices, source include closures, and whether every dependency of both board entry files is present in the selected public tree.

That last check is the one that earns its keep. A build works on your machine because the file it needs happens to be sitting there untracked. It fails on the first clone somebody else makes, and it fails with a missing-include error that tells them nothing about your working directory. Run the tracked-set form before you publish anything.

Installing Anvil on a Raspberry Pi 4

Six required files on a boot volume: three from Raspberry Pi and three from the Anvil package. Then a serial cable, and a prompt.

What this chapter assumes

A Raspberry Pi 4 Model B. A boot volume the Pi's firmware can read - the bench this was written against boots from a **USB stick, not an SD card** - a USB-to-serial adapter, and the image you built in chapter 2.

> **Note.** On the card question. Everything in this chapter is about the FAT

boot volume, and the layout is identical on an SD card. But be honest with yourself about what is proven: no procedure in this project has ever been executed against a Pi 4 SD card. The bench uses a stick. The host tool's `--to card` option is only the tool's word for "the boot medium"; it writes the stick. If you are on a card and something behaves differently, you are ahead of this book.

The short way: the skeleton in the repository

If you would rather copy files than read forty paragraphs first, `Boards/RaspberryPi4/sdcard/` holds the Anvil files for the boot volume; add the three firmware files published by Raspberry Pi:

```
Boards/RaspberryPi4/sdcard/
  kernel8.img            build 212 monitor image
  armstub8.bin           required EL3 entry stub
  SHA256SUMS             image and stub hashes
  config.txt             selects the required EL3 stub
  SETTINGS.TXT.example   optional; rename and edit
  README.md              the card layout, and the first commands
```

The package contains **build 212**, 4,200,128 bytes, SHA-256 `d6d40968e176aba4d69501119d5a84c0f6d221763d7fbc3c4a285721c253294f`, and the 512-byte EL3 stub with SHA-256 `9f96324d137ebb4c1452ab00477ffe8c5bf77c53a3bb80ae7fb34234d693f2a2`. Verify both with `sha256sum -c SHA256SUMS` before copying. Build 212 is the current compiled package identity; the historical hardware acceptance record is Build 99, so these hashes do not claim that Build 212 itself has completed a new physical acceptance run.

The rest of this chapter is the same thing with the reasons attached, and the reasons are what stop you losing an evening to `disable-bt`.

What goes on the boot volume

Six required files. Three come from this repository.

```
start4.elf          Raspberry Pi firmware. Not in this repository.
fixup4.dat          Its matching file. Not in this repository.
bcm2711-rpi-4-b.dtb The device tree. Not in this repository.
CONFIG.TXT          You write this. Contents below.
```

```
KERNEL8.IMG         Boards/RaspberryPi4/sdcard/kernel8.img
ARMSTUB8.BIN        Required EL3 firmware entry stub.
```

Trap. There is **no** `bootcode.bin` **on a Pi 4**. The Pi 4 boots its

second stage from an onboard EEPROM. `bootcode.bin` is a Pi 3 file, and the only place it appears in this repository is the Pi 3's pinned firmware manifest. Copying a Pi 3 boot set onto a Pi 4 volume will not work and will waste your afternoon.

Note. This repository deliberately does not bundle vendor boot firmware,

a complete boot partition, or board-specific display configuration. Get the firmware files from the official Raspberry Pi firmware distribution. The vendor firmware files are not redistributed here. The Anvil image and stub are pinned by the package's `SHA256SUMS` file.

Rename the image when you copy it:

```
copy build\pi4\anvil.img E:\KERNEL8.IMG
```

Trap. Names on the volume are **8.3**. The monitor's own filesystem code

writes short directory entries only and generates no long-name chains, so a file it writes is `KERNEL8.IMG`, `SETTINGS.TXT`, `CONFIG.BAK` - and anything you put there with a long name will not be visible to it the way you expect. Keep every name you intend the board to touch inside eight characters plus three.

If you want Wi-Fi, two more files from this repository go on the volume as well:

```
Firmware/CYW43455/brcmfmac43455-sdio.bin       -> BRCMFW.BIN
Firmware/CYW43455/brcmfmac43455-sdio.clm_blob  -> BRCMCLM.BLB
```

A third, the NVRAM file, is **deliberately not bundled** because it may carry calibration and identity data. You supply a board-appropriate one as `BRCMNV.TXT`. See `docs/LICENSING.md` for the redistribution terms on all three - the radio firmware is separately licensed and is not MIT.

config.txt

This is the live bench file, quoted whole. The comments in it are the reasons, and they were all paid for:

```
# Raspberry Pi 4 / BCM2711 - bare-metal boot via U-Boot
# PureMetal Forge, 2026-08-24

arm_64bit=1

kernel=kernel8.img
kernel_address=0x200000
armstub=armstub8.bin

dtoverlay=disable-bt
```

```
enable_uart=1
init_uart_clock=48000000

disable_overscan=1
dtoverlay=vc4-kms-dsi-waveshare-panel-v2,10_1_inch_a
uart_2ndstage=1
dtdebug=on
```

The first two comments in the historical config referred to U-Boot. The current package boots Anvil directly from firmware; it does not load U-Boot. The active directives above select the required custom stub so Anvil enters at EL3.

Line by line, the ones that matter:

`kernel_address=0x200000` is **not optional**. Anvil is built to run at `0x200000`, and its BSS - which includes a large USB host arena - runs well past the end of the image. Loaded at the 64-bit default of `0x80000`, that BSS would reach down through `0x100000`, which is where Anvil puts its own stack. Loading it where it was linked keeps the memory map that has actually been tested.

`dtoverlay=disable-bt` is **not optional either**, and this one cost an evening. With Bluetooth enabled, the Pi 4 gives header pins 8 and 10 to the mini-UART and routes the PL011 to the Bluetooth module. A program writing to the PL011 is then talking to a device that works perfectly and is not attached to any wire. Reads came back healthy, writes vanished, and nothing faulted. `disable-bt` puts the PL011 back on the header.

`init_uart_clock=48000000` matters once the PL011 is back on the header. The mini-UART derives its baud rate from the video core clock, which the firmware scales; a PL011 with a pinned reference does not move.

`disable_overscan=1` stops the firmware trimming 48 pixels from every edge of a TV-style HDMI mode. Without it, a 1920x1080 panel comes up as 1824x984 - and it goes unnoticed, because the display code asks the firmware what the physical size is and then faithfully requests the already-shrunken answer.

The `vc4-kms-dsi-waveshare-panel-v2` line is bench-specific: it is there for one particular DSI panel. If you have no DSI panel, leave it out.

Wiring the serial console

Three wires, all on the even-numbered row of the header, consecutive:

```
adapter GND  ->  pin 6    GND
adapter RX   ->  pin 8    GPIO14 / TXD0
adapter TX   ->  pin 10   GPIO15 / RXD0
```

Pin 2 is the corner nearest the USB-C end; pins 6, 8 and 10 are the third, fourth and fifth along that row. RX and TX cross - the adapter's receive takes the Pi's transmit.

115200 baud, 8 data bits, no parity, one stop bit, no flow control.

> **Trap. Leave the adapter's VCC disconnected.** The Pi powers itself.

Back-feeding 5 V onto the header is how boards die. The serial lines are 3.3 V TTL; never attach an RS-232 voltage-level port directly.

Trap. Ground can bite you in the other direction too. On this bench the

serial adapter stopped enumerating on the host with "Device Descriptor Request Failed", and it came back the instant its **ground** wire was lifted off the Pi. A DSI panel's return current was flowing through the adapter's ground into the laptop. If you power a panel, give it its own 5 V supply rather than the Pi's header pins.

First boot: what you will see

The lines below are from a recorded board session of 2026-09-05. The network name and the board's hardware address are replaced with placeholders; nothing else is changed.

```
PureMetal Forge - Anvil, the Raspberry Pi 4 monitor. Version 2.0,
64-bit AArch64, running with no operating system underneath it.
The serial port is the PL011 at FE201000, set up from cold to 115200 baud,
eight data bits, no parity, one stop bit.
The processor is a Cortex-A72 in AArch64 at exception level 2.
This monitor occupies 00200000 onwards, with its stack growing down
from 00100000 and its variables at 0D000000.
A payload may be loaded into either of these:
  the low window   00400000 to 07EFFFFF   (about 123 MB)
  the high window  40000000 to FBFFFFFF   (about 3 GB)
The tick counter every timeout here is measured on runs at 54000000 Hz.
Type info to see what board this is, or help for the list of commands.
```

Then the board brings itself up, one line per thing it starts:

```
Bringing up the board - each line names what it is starting, so a
hang or a failure shows exactly where.
  PCI Express .......... ok
  USB host (xHCI) ...... ok
  USB devices .......... mass storage
Mounted the USB stick, which calls itself Samsung  Flash Drive FIT  and
holds 119 GB. Files will be read from and written to this.
  settings ............. read from SETTINGS.TXT
  firmware uploaded to the radio in 315 ms.
Joining a known Wi-Fi network...
  <your network>: using the cached key (no ten-second derivation this boot).
    associated - running the four-way handshake...
KEYED to <your network> - the link is up and encrypted.
Wireless console listening on UDP 192.168.1.25 port 5555.
  A host reaches it with tools/anvil_wifi.py <ip> - send it a line
  and the prompt answers over the air. The serial console still works.
  network  <your network>
  MAC      <this board's MAC>
  IP       192.168.1.25
  netmask  255.255.255.0
  router   192.168.1.2
  console  UDP 192.168.1.25 port 5555 - reach it with tools/anvil_wifi.py
pmf>
```

That `pmf>` is the prompt. You are in.

> **Note.** "each line names what it is starting, so a hang or a failure shows

exactly where" is not decoration. If your board stops, the last line printed is the thing that hung, and that is usually the entire diagnosis. A boot that stops at `USB host (xHCI)` and a boot that stops at `PCI Express` are completely different problems.

Two things about that banner

The exception level is read from the processor, not printed as prose. The line above says "exception level 2" because that session was an EL2 boot, before the firmware stub existed. With the stub installed it reads "exception level 3", because the code decodes the current level register and prints the digit it finds.

> **Note.** This transcript is historical and predates the EL3 stub. Hardware

acceptance for the Pi 4 EL3 path is recorded in `docs/EL3_BOOT_ACCEPTANCE.md`; the current supported config selects the stub. On your board, verify the live level with `info` rather than relying on this old transcript.

The memory windows in the banner are the real ones, measured at runtime rather than baked in. The monitor reads its own image extent symbols at boot and rounds its reservation up to a megabyte boundary. That is why chapter 6 insists you read `map` from the running monitor before staging anything, instead of copying an address out of a note.

The network console

The monitor listens on **UDP port 5555** as soon as it has an address, on whichever interface holds one - the Ethernet cable or the Wi-Fi radio, wired preferred when both are up. Whoever sends it a datagram becomes the peer: from then on everything the monitor prints comes back as UDP, and every line you type is fed into its command input, exactly like the serial console.

```
python tools/anvil_wifi.py <board-ip>
python tools/anvil_wifi.py <board-ip> "info"
python tools/anvil_wifi.py --find
```

The first is interactive. The second sends one command and prints the answer. The third broadcasts a `version` to every directly connected IPv4 subnet and lists who answers, together with the local interface that heard it - which is how you find a board whose address you have lost.

> **Note.** The tool is called `anvil_wifi.py` for historical reasons and the

name is kept so older notes still work. It is the network console, same port and same datagrams over either link.

How the board gets an address

Three rungs, tried in this order, and the boot log says which one it took in one line:

```
wired: lease from 192.168.137.1
wired: saved address 192.168.137.2, no DHCP answer
wired: link-local 169.254.170.250 (no DHCP answer)
wired: no cable
```

A board that took a link-local address on the cable then **becomes the DHCP server on it**, because nothing else was (`Anvil/Core/dhcpd.pbi`). What it prints, quoted from `RaspberryPi4/Board/eth.pi4`:

```
wired: nothing answered a DHCP discover on this cable, so THIS BOARD is
   the DHCP server on it now: 192.168.137.1 for itself, leases from 192.168.137.2 to 192.168.137.20,
   one hour each. A machine at the other end set to DHCP - which is what
   an Ethernet port is by default - gets a real address with nothing
   changed on it. If a real DHCP server ever answers on this cable, this
   board gives the role up at once and takes a lease like anybody else.
```

A DHCP lease first, because a DHCP server is the only thing on a segment that can say for certain an address is free. Then a saved address, if somebody typed one on purpose and ran `settings save` - that outranks anything the board invents for itself. Then a link-local address the board has proved free with its own probes, because an address it invented safely still beats no address, and no address means a board nobody can reach.

The practical consequence is the one worth remembering:

> **Note.** **A Pi 4 on a bare Ethernet cable into a laptop answers its

console after a reset with nothing typed on the board and nothing changed on the laptop.** No DHCP server, no connection sharing, no static address, no firewall rule. The board serves the laptop a lease, so the laptop is on `192.168.137.x` and the board is `192.168.137.1`. The board's link-local address stays on the cable as well (it is seeded from its hardware address, so it is the same one across flashes), and the console answers on both.

> **Trap.** A link-local address is **never** written to `SETTINGS.TXT`. If it

were saved, the next boot would put a `169.254.x` address on the wire without probing whether it was still free. If you find one saved, some tool put it there and it is a bug.

Two commands to run first

`version` tells you which image is running:

```
pmf> version
```

It prints the build number from chapter 2, and the date and time that build was made. That is what the number is for.

> **Trap. The build number is not an identity.** It is a label that a

person reads, and a label can be stale if an image was built somewhere the counter did not reach. This bench once had an image reporting build 20 that was actually build 21. **The length identifies it**, together with its CRC32 or SHA-256. Chapter 4 shows how to get both off the board itself.

`map` tells you what memory is yours:

```
pmf> map
```

Read it before you stage anything anywhere. Every host tool that writes to this board parses a validated `map` response from the running monitor and refuses unknown or malformed output. Do not derive an upload address from an example, from a previous build, or from this book.

The EL3 stub, and the reason to be careful

Anvil's supported runtime is EL3 - the highest privilege level on this processor. `armstub8.bin` is required. The firmware loads it at address zero and enters it at EL3; it does the machine setup the stock stub does, then branches straight into `KERNEL8.IMG` without returning to EL2. The monitor checks its current exception level and refuses a lower-level entry.

It is 512 bytes. It is a source translation of Raspberry Pi's own stub, pinned to a named upstream commit, and it keeps its upstream BSD-3-Clause notice.

> **Trap. A monitor watchdog cannot repair a bad firmware-stage stub.** If

the board fails before Anvil starts, nothing on the board can help you and you must restore files from a host. Keep physical access to the medium and a known good copy of both the image and stub before changing the package. Do not test the new monitor at EL2 first: Anvil refuses that entry level.

```
1. Back up the existing boot files and `CONFIG.TXT` to a host and verify the
   backup.
2. Copy `kernel8.img`, `armstub8.bin`, and `config.txt` from the same verified
   package. Keep the firmware files already supplied for the board.
3. Verify `kernel8.img` and `armstub8.bin` against that package's
   `SHA256SUMS`; ensure `config.txt` contains `armstub=armstub8.bin`.
4. Connect serial, power the board, and confirm `info` reports EL3 before
   using other commands.
```

If the board fails before the prompt, restore the complete known-good image, stub, and configuration from the host. Removing `armstub=` is not a usable Anvil fallback because the monitor deliberately refuses EL2.

Keep a way back

Before you install anything, put a known-good image on the medium under its own name, and know the command that restores it:

```
pmf> load SCALEOK.IMG 800000
pmf> save KERNEL8.IMG 800000 20C940
pmf> reset
```

That is a real recovery from this bench: a named spare image is loaded to a staging address, written over `KERNEL8.IMG`, and the board reset into it.

> **Trap.** Two details in those three lines have each cost a board's worth of

trouble. `save` takes an address and a **LENGTH**, in hex, not an end address - and so does `crc32`. And `load` with no address defaults to `00400000`, which on a large image can land on the tail of the running monitor. Stage at `800000`. Chapter 4 goes

through the argument shapes properly.

> **Trap. Never enter a monitor image as a payload.** `boot mem` places an

image at the address it was built for, which for a monitor is `$00200000` - over the running monitor - and enters it. Do that while anything else is in flight and the board goes down with no console and no serial, and somebody has to pull the medium. This is not theoretical; it was avoided once by a margin of one command.

If the board is silent on the serial line a minute or so after a reset, write down that it hung and what the last step was, and stop. A power cycle is the next move, not another command into the dark.

The console

Every command prints something, on every path, whether it worked or not. If one ever returns to the prompt in silence, that is a fault.

ï»¿ Chapter 4 of THE ANVIL GUIDE.

The prompt

```
pmf>
```

Five ASCII characters, the trailing space included, and no newline after them.

> **Note.** That is a **protocol string**, not decoration. Host tools wait for

exactly those five bytes, and at least one of them matches on the end of what it has read - so the trailing space is load-bearing. On a board with an on-screen console the prompt is drawn in bright green, but no escape sequence is inserted into the stream: the colour is a style byte attached to the screen grid, and the serial and network paths receive the plain characters. A tool cannot tell the difference and does not have to.

Messages never land on the command line

The command line carries three things: the prompt, what you type, and the output of the command you ran. **Nothing else is ever written into it.** Anything a background service has to say - the network console arming on a new address, a DHCP lease renewed or dropped, the radio losing the link or getting it back, a Wi-Fi rekey, the mouse being looked for - is a **message**, and a message goes to the message log instead, whether or not you are halfway through typing a line.

On the serial line and the network console, read them with `messages`. On a board with a screen, the banner's status row also shows the newest one you have not read, after the clock and the temperature:

```
2026-09-16 13:40:12 CDT    112 F    2 new messages: Wi-Fi: group key rekeyed.
```

`messages` prints the log oldest first and marks it read, which clears the banner within a second. Its first line is for a script. This is the exact output of the monitor's own code, executed under the desk gate `tools/console_messages_emitted_check.py`. On a Raspberry Pi 4 running build 164 the same command answered `messages 2 kept 3 lost 0`, message #1 being the USB mouse and keyboard search - the sentence that used to print straight after the first prompt:

```
pmf> messages
messages 3 kept 4 lost 0
Messages from background services, oldest first. These are never
printed on the command line; this is where they are read.
  #1  at 1 s
      The network console is now listening on:
          192.168.137.1 port 5555, over the wired Ethernet
  #2  at 3 s
      Wi-Fi: group key rekeyed.
```

```
  #3  at 4 s
      Wi-Fi: lease renewed, 7200 s
```

`total` counts every message since start, `kept` the lines still held (64 at most), and `lost` the lines pushed out to make room. A line with nothing visible in it is not kept, and a byte that cannot be shown as itself appears as `\xNN`. Everything up to the first prompt - the boot log - still prints on every console as it always did.

> **Note.** A command's own output is never a message: what `dhcp`, `wifi join`

or `net` print about the network while you wait for them is their answer. The rule and how it is built are in `docs/CONSOLE_MESSAGES.md`.

Every command is a whole word

The single letters still work - host tools type them, and lines pasted out of a U-Boot session land here too - but the words are what you should learn:

```
b  receive     l  srecord     g  run/go      m  memory
w  write       i  info        c  clock       a  autoboot
f  load        r  reset       d  deadman     p  screenshot
s  screen      h  help        ?  help
```

Type something the monitor does not know and it says so and does nothing:

```
pmf> frobnicate
? this monitor does not know the command frobnicate, so nothing was done.
  Type help, or just a question mark, for the list of commands.
  Every command is a whole word; the old single letters still work.
```

help

```
pmf> help
```

It is long - several hundred lines - and it is organised. The sections, in order:

```
GETTING CODE IN
RUNNING IT
LOOKING AROUND
FROM U-BOOT, AND HARDWARE-FREE
THE SCREEN
OVER THE NETWORK
SETTINGS THAT SURVIVE A POWER CYCLE
THE BOARD
WHERE A PAYLOAD MAY GO
WHAT THIS MONITOR OWNS
```

The last two are not text. They call the same code `map` calls, so the addresses in `help` and the addresses in `map` cannot disagree.

Its opening paragraph is the promise this chapter's title quotes:

```
Every command is a whole word. The old single letters all still work
and are shown in brackets after each one, because the host tools type
them and because U-Boot's own loads and go land here too.
Every command prints something, on every path, whether it worked or
not. If one ever returns to the prompt in silence, that is a fault.
```

> **Note.** `help` documents `net stats` and `net counters`. On images built

before they were wired into the `net` command, typing either got the `net` refusal listing the subcommands that did exist. On a current image both print what each network controller counted; see Network and radio below.

map

```
pmf> map
```

This is the command you type before anything else that involves an address. What you will see, from a recorded board session on build 101:

```
THE MEMORY MAP, AS THIS IMAGE MEASURES IT
  this image      00200000 to 004B144B   2823244 bytes, measured from the image itself
  reserved        00200000 to 004FFFFF   this monitor's code, measured from the image
  reserved        08200000 to 089FFFFF   its variables, placed by --bss-addr
  reserved        001FF000 to 001FFFFF   the autoboot record
  reserved        08A00000 to 091FFFFF   the DSI framebuffer
  reserved        001FC000 to 001FEFFF   the reserved raw-secondary stacks
  reserved        09200000 to 099FFFFF   the driver-module arena and its staging window
  reserved        001FB000 to 001FBFFF   the boot phase record
  reserved        0B000000 to 0BBFFFFF   the screenshot capture area
  its stack top   00100000               and it grows downwards from there
  stage a file at 00500000               which is what the upload tools ask for

Where a payload may go:
  the low window   00500000 to 07EFFFFF   (about 122 MB)
  the high window  40000000 to FBFFFFFF   (about 3 GB)
  Nothing in this map is a constant this program was told. The
  monitor's extent is the size of the image the firmware loaded, and
  the low window begins above it, so both move when this program
  does. There is no maximum size.
ok
```

> **Note.** This is the recorded build 101 map. Current Pi 4 builds reserve the

Vulkan heap at `0C000000..0CFFFFFF` and the 16 MiB monitor BSS window at `0D000000..0DFFFFFF`; `map` on the running board is authoritative.

> **Note.** Read that last paragraph twice. **The low window's bottom edge

moves with the monitor's size.** It was `00400000` on one build and `00500000` on the next, for no reason other than the image growing. Any address you copy out of a note, an example, or this book is a guess about a build you are not running. Type `map`.

The `stage a file at` line is the single most useful row: it is what `HwStageAddr()` answers, and it is exactly what every host tool asks for.

version

```
pmf> version
```

The first line is machine-readable and fixed-width:

```
build 142 date 20260915 time 220310
```

Then the banner, then this, which is the honest part:

```
  This is build 142 of this board's monitor, made 2026-09-15 at 22:03:10.
  The number is raised by one by every successful build, so two images
  can be told apart from the prompt. It says WHICH build this is, not
  WHAT is in it - a number can be reset, frozen, or carried by a source
  somebody edited without rebuilding. The digest below is what is in it.
```

Then the image's own measured extent, and then:

```
  That size and the build number above are enough to tell two images
  apart. For the digest that says WHAT is in it, type:
    version full        - or crc32 <base8hex> <len8hex>
  It is not done here unasked because it reads every byte of the
  image, which is not instant on a board with its caches off.
```

`version full` does the CRC and prints:

```
  Its crc32 is <hex8>, worked out just now over the running code.
  IEEE 802.3, reflected, polynomial EDB88320 - the same value
  tools/anvil.py computes on the host, so the two agree independently
  or they do not. THIS is what says which image you are talking to.
```

> **Trap.** Do not scrape `EDB88320` out of that paragraph as if it were the

answer. It is the **polynomial**, printed for provenance, and a host tool that matched loosely on "eight hex digits near the end" once read it as the CRC and reported a good file as corrupt. Anchor on the phrase `Its crc32 is`.

Looking at memory

```
memory <address> <count>     also dump, md, m
readback <address> <length>
write  <address> <value>     also poke, w
fill   <address> <value> [count]     also mw
copy   <source> <dest> <count>       also cp
compare <first> <second> <count>     also cmp
crc32  <address> <count>     also checksum
sha1sum   <address> <count>
sha256sum <address> <count>
```

All numbers are **hex**. The memory family takes a width suffix: `memory.b`, `memory.w`, `memory.l` for 1, 2 and 4 bytes.

memory is for a person, readback is for a machine

`memory` prints hex with an ASCII column, at most 4096 bytes a call, and it goes to **every** console at once - the serial port, the screen and the network - so all three agree about what was printed. That makes it exactly as fast as the slowest of them. Every character the monitor prints is pushed through the serial port at 115,200 baud before the network console sees it, and a dump spends 78 characters on 16 bytes. Read back over a gigabit cable it managed **1,729 bytes a second**.

`readback` is the same bytes for a tool. It prints a header, the range as base64 at 48 bytes a line, and a closing line with the byte count and a crc32 - the format, as `help` gives it:

```
readback 57400000 1812000 bytes base64
<64 base64 characters>  ... and 37,750 more ...
readback end 1812000 bytes crc32 A1B2C3D4
```

The base64 goes only to the console that typed the command. Typed over the network console it leaves by the network at that link's speed and never touches the serial port; typed on the serial port it goes out the serial port. The other consoles still see the command and its closing line - the parts a person can read. There is no length limit and no default length, the address is not moved by `base`, and Ctrl-C stops it with `readback stopped after N of M bytes crc32 X`, the count and checksum of what really went.

> **Trap.** A reader must check **both** numbers on the closing line. A lost

datagram takes whole base64 lines with it and leaves a stream that still looks orderly, so the length is the first thing that catches it; a byte changed on the way keeps the length and fails the crc32. The polynomial is the one `crc32` uses and the one Python's `zlib.crc32` uses, so the host's answer is independent of the board's.

Measured on the bench board over the cable, the same host, the ranges compared byte for byte with the image on disk afterwards:

```
                        memory (build 148)   readback (build 151)
98,304 bytes            56.823 s             0.262 s
                        24 commands          1 command
1,812,000 bytes         1,048 s (build 142)  1.280 s, 1,383 KB/s
                        443 commands         2 commands
```

```
per 4,096 bytes           2.37 s              -
retries                   0                   0
```

> **Note.** Most of the 0.262 s is not the transfer. The reader waits for one

receive timeout on the host before it sends, to clear anything still arriving from an earlier command, and on a small read that is the larger part of the time. The megabyte figure is the one that shows the rate. The caches were on for both `readback` runs, as the board was found.

sha256sum, and the hex-digit rules

```
pmf> sha256sum 400000 18CE24
sha256sum over 1625636 bytes, 00400000 .. 0058CE23 ...
The sha256sum is 85370e48df79c7fe06956c2c47a27fef049b3e475d44cbba6395422df6ab463f
  sha256sum of the same bytes on any other machine gives the same digest, so a
  match proves this range is byte-for-byte what that machine has.
```

Sixty-four lowercase hex digits, the same as the `sha256sum` tool on any other machine, so the two compare directly. `sha1sum` gives forty.

Three rules about the numbers, and they matter:

Sixteen hex digits, and no more. A number of seventeen or more hex digits is **refused**, not truncated and not wrapped. The count is of digits typed, leading zeros included - so `0000000000000400` is fine at sixteen and one more zero in front is not.

```
!! that address has more than sixteen hex digits, and sixteen is all
   a 64-bit address can have. Nothing was computed.
```

> **Trap.** That message is printed for the **address** only. Give the

count seventeen digits and you get this instead:

```
!! a byte count is required, in hex.
```

which says a count was missing when in fact one was typed and rejected for its length. That is a real defect - the over-length flag is checked for the first argument and not the second - and it is named here so that if you meet it you know what actually happened.

Exactly two arguments. A third is refused by name, with the reason:

```
sha256sum here takes exactly two arguments, an address and a count.
   U-Boot's has a third that stores the digest in memory; it is not
   implemented, because the byte order it stores is decided in
   lib/hash.c, which is not in this repository. Nothing was computed.
```

> **Note.** That is the house style in one paragraph: the command does not

half-implement somebody else's interface, it does not guess at a byte order it cannot check, and it tells you exactly why rather than saying "bad arguments". A guessed-at compatibility is worse than none.

A stopped hash prints nothing. Press a key during a long digest and you get:

```
   stopped part way, so NO DIGEST IS PRINTED. A hash of part of a
   range looks exactly like a hash of all of it and is not one.
```

Files on the boot medium

```
ls [path]                    also fatls
load <path> [address]        also mount, fatload, f
save <path> <address> [length]
cat <path>
mkdir <path>
rm <path>                    also del
rmdir <path>
rename <old path> <new path> also mv
```

A path names its partition. Put the partition number and a colon in front: `2:/exfat test/random-1MiB.bin` is on partition 2, and `2:hello.txt` is the same as `2:/hello.txt`. A path with no prefix is **always partition 1**, the boot partition - never "whichever partition was used last" - so `save KERNEL8.IMG ...` cannot be redirected onto a data partition by an `ls 2:/` typed earlier. Quote a path that contains spaces. `f` on its own lists every partition in the table, what it is, and its cluster size.

A data location that the medium decides. Recipes and tools do not write `2:/` or a bare partition-1 name for data such as weights and model packs. They write `data:/models/MODEL.PMW`, which means `<data.path>/models/MODEL.PMW`, where `data.path` is an ordinary setting:

```
settings set data.path 2:/
settings save
load data:/models/MODEL.PMW 40000000
```

One partition: leave `data.path` unset, and `data:/` is the root of partition 1. **Two partitions: set it to `2:/` once** and save it. `settings` says where `data:/` points, or that it is unset. `load`, `save`, `ls`, `cat`, `mkdir`, `rm` and `rename` all take the form. A `data.path` that itself starts `data:` is refused, and so is a joined path longer than 1023 bytes.

> **Note.** `data:` is in the source from commit `c5e70f8`, which comes after build

165. A board running 165 or earlier refuses a `data:` path (it reads it as a folder on FAT partition 1: volume error 3). The bench stick's weights still sit on partition 1 until they are moved.

FAT32 and exFAT partitions take the same paths: folders at any depth and long UTF-8 names, matched without regard to case. On FAT32 a file with a long name can also be reached by its short alias (`B84DET~1.IMG`), and a name Anvil creates that does not fit 8.3 gets one, generated the way Windows does. Until build 168 a FAT32 partition was read at its root only, by 8.3 short name.

`mkdir` makes one directory (its parent must exist), `rm` removes one file or one EMPTY directory and never anything recursive, `rmdir` removes one EMPTY directory and refuses a file, and `rename` renames or moves an entry within one partition - including a rename that only changes the case of a name. Each is write-gated the same way `save` is: the block writer is armed for the one change, the volume is marked dirty before anything is written, the stick is asked to commit its cache, the volume is marked clean, and the writer is disarmed again. A file or folder created or written gets the wall-clock time when the clock holds a trusted time (set by hand or by SNTP), and 1980-01-01 00:00 when it does not.

From a recorded board session on build 164, with a USB stick whose partition 1 is the FAT32 boot partition and partition 2 an exFAT data partition:

```
pmf> f
The boot medium is a USB stick holding 250626566 blocks of 512 bytes each.
Its MBR partition table names these partitions:
  1:  FAT32, clusters of 4096 bytes, 1024 MB from block 2048
  2:  exFAT, clusters of 131072 bytes, 121351 MB from block 2099200
pmf> ls "2:/exfat test/models"
Directory 2:/exfat test/models on the boot medium:
  Kokoro-82M weights (FP32).pmw   324621440 bytes
  1 file, 324621440 bytes, 0 directories.
pmf> mkdir "2:/anvil exfat 2026-09-16"
Made the directory 2:/anvil exfat 2026-09-16. The volume was flushed and the block
writer disarmed again.
pmf> rename "2:/anvil exfat 2026-09-16/scratch file.tmp" "2:/anvil exfat 2026-09-16/scratch renamed.tmp"
Renamed 2:/anvil exfat 2026-09-16/scratch file.tmp to 2:/anvil exfat 2026-09-16/scratch renamed.tmp.
The volume was flushed and the block writer disarmed.
pmf> rm "2:/anvil exfat 2026-09-16/scratch renamed.tmp"
Removed 2:/anvil exfat 2026-09-16/scratch renamed.tmp. Its space is free again, the volume was flushed
and the block writer disarmed.
```

On build 165 and earlier the stick read at about 3 MB a second whichever filesystem was asked - the 324 MB weights file loaded from partition 2 in 105.6 s and the same bytes from partition 1 in 103.7 s - and a 16 MiB `save` to exFAT took 172 s. Neither was the filesystem's format. The USB driver moved one 512-byte block per device command, three round trips per block (forum 895); the exFAT writer read every sector before writing it and zeroed every new cluster first (forum 893); the FAT32 writer wrote four FAT sectors per cluster (forum 896); and an exFAT mount read one sector per byte of its up-case table, so every switch between partitions cost about two seconds (forum 897).

> **Note.** The storage speed change of 2026-09-17 moves up to 1 MiB per READ (10)

or WRITE (10), lets the stick send its data in bursts, hands every run of whole sectors to the stick in one call, keeps FAT and allocation-bitmap changes in memory until a sector of them is complete, and no longer reads or zeroes a file cluster before writing it. The order in which a change reaches the stick - allocation, then data, then the directory entry, with the volume marked dirty first and clean last - is unchanged. The speeds a board shows with it have not been measured yet; this page gives them once they are. `usb read <block> <address> <count>` moves 2048 blocks per call, so it times the transport without a filesystem in the way.

A refusal names the layer that refused and says what to check, and the reason it gives is the one the layer had AT THE MOMENT IT REFUSED. That distinction is not pedantic: a change that fails is followed at once by a flush, to leave the medium consistent, and a flush that succeeds leaves the filesystem reporting no error at all - so a reason read afterwards is about the flush. Until build 180 a refused `mkdir` could therefore print *The medium said: nothing went wrong*, which sent a reader looking in the wrong place. It now prints what the filesystem said when it said no.

Trap. A layer that refuses and names nothing is not a success. If you ever

see *file error 9: the filesystem refused and named no reason*, the fault is in this monitor and not in the medium - the operation did not happen, the medium is untouched by it, and the thing to report is the command you typed. It is a separate sentence precisely so that it can never be read as an ordinary outcome.

A board running build 167 or earlier still answers `ls OVERLAYS` with *volume error 3: that partition is FAT, and FAT is read here at the root directory only*; from build 168 it lists the folder.

`ls` on its own lists the root:

```
pmf> ls
Directory / on the boot medium:
  KERNEL8.IMG   2930784 bytes
  SETTINGS.TXT       412 bytes
  CONFIG.TXT        2170 bytes
  ...
  5 files, 3043821 bytes, 0 directories.
```

`load` with no address goes to the staging address the board just told you about in `map`. With no arguments at all it reports the medium instead of loading anything.

The refusals are worth reading before you need them, because they tell you what was and was not done. If the file would not fit:

```
!! that file is 2930784 bytes long, so loading it there would run over
   the monitor itself, which lives at 00200000 to 004FFFFF.
   Nothing was loaded. Give load an address in one of the payload
   windows - type help to see them - or leave the address out and it
   goes to 00500000.
```

Trap. And the one that is genuinely dangerous, because it happens

after memory has been changed:

```
!! reading the file stopped part way through, AFTER some of it had
   already been written into memory. What is at that address now is
   the beginning of a file and whatever was there before underneath
   it. Do not run it. Load it again, or load something else over it.
```

What `usb tree` says about the link under the stick

`usb tree` lists the host controller, its root ports and what is plugged into them. From build 174 it also prints the link the controller itself is on, which is the first thing to look at when the storage is slower than the medium should be.

The controller line reads `controller link PCI Express 5.0 GT/s x1, spread-spectrum clocking on, controller firmware 000138C0` - the trained speed and lane count as the root complex reports them, whether spread-spectrum clocking was

entered, and the controller's own firmware version. A speed the driver does not have a name for is printed as `speed code <n>` rather than guessed at. On a board with no such link every part of the line is absent rather than invented.

Each connected port then carries `link state U<n>` and `U1/U2 timeouts <a>/<b>` after its speed: the port's current link state (`U0` is active) and the two low-power entry timeouts the controller is holding.

> **Note.** Nothing here changes throughput on its own. Spread-spectrum clocking

is an electromagnetic-emissions measure, not a speed control, and both Linux and U-Boot enable it on this controller straight after the link comes up. It is printed because it was never enabled at all until build 174, and because a link that trained at 2.5 GT/s instead of 5.0 GT/s, or at the wrong width, would cap every transfer on the bus - and until this readout existed there was no way to tell from the board which had happened. The values a real board shows are added here from the first session that runs it.

`usb link` - the same link in raw registers

`usb tree` 's controller line ends with the words `spread-spectrum clocking on` . Those words come from a single flag, and a flag is the one kind of answer nobody can check: it reads exactly the same whether the driver turned spread spectrum on, found it already on, or only believes it did. `usb link` prints the registers underneath instead.

```
pmf> usb link
```

Each row is a register: the name the board gives it, its raw value in hex, every named field inside it with its bit position, and a sentence saying what the drivers that run this part in service leave there. Ten registers are shown - the two spread-spectrum registers on the controller's internal serial bus to its physical layer, the link capability, control and status words, the controller's miscellaneous control and hardware-debug registers, the root capability register, the revision, and the link status word.

Three things are worth knowing before reading the output.

A register nothing answered prints `not read` , never `0` . A zero that means "no bits set" and a zero that means "nobody replied" are different facts, and printing them alike is how a driver gets credited with a bit it never read.

The two serial-bus registers are the values captured at the moment the bring-up sequence read them, not a fresh read. Reaching them means driving a transaction on that bus, which is a change to the link in order to observe the link; the other eight are read live when the command runs.

The field values are the architected codes, not their meanings. A current link speed of `2` is 5.0 GT/s and a negotiated width of `1` is one lane; turning a code into words is what `usb tree` is for.

> **Note.** This command touches nothing. It writes no register, starts nothing

and resets nothing, so it is safe at any prompt and while anything else is running. A board whose USB host does not sit behind a link of this kind answers with a sentence saying so, which is a property of that board and not a failure.

`measure` - the board times its own commands

```
pmf> measure usb read 800 40000000 4000
```

```
pmf> measure load "2:/big file.bin" 40000000
```

`measure` runs the rest of the line exactly as if the word were not there, leaves the command's own output untouched, and then prints what it cost on the board's own tick source, in milliseconds and in microseconds.

> **Note.** Why this is not a convenience. Every speed this monitor had ever

been credited with was measured by a HOST waiting on the network console, and that wait includes the console's round trip - which is neither small nor constant between client programs. One client on this bench measures a floor of about four hundred milliseconds per command; another measures about one. The same three-hundred-megabyte load therefore reads as 0.078 s or as 0.502 s depending on which program held the stopwatch, and the second one looks exactly like a six-fold regression that is not there. With `measure`, the host clock becomes a sanity check instead of the measurement.

> **Trap.** The word is `measure` and not `time`. `time` is already the wall

clock - it sets and shows the date - and taking that word for a stopwatch would have broken the clock on every board at once for the sake of a nicer name.

A bare `measure` with nothing after it runs nothing and says so, and `measure measure <something>` is refused rather than quietly flattened into one measurement of the inner command.

`usb devices` - every device, as it describes itself

`usb tree` answers "what is plugged in". `usb devices` answers "what does it actually say about itself", which is the question somebody writing a driver has. Every device, in address order, with:

```
its address, slot, port path (3, or 1.4 behind a hub), speed and parent
idVendor:idProduct, bcdDevice, bcdUSB
bDeviceClass/SubClass/Protocol with the class in words
bMaxPacketSize0 and how many configurations it has
the manufacturer, product and serial-number strings
the active configuration: value, self or bus powered, remote wakeup,
    bMaxPower in mA, and how many descriptor bytes it published
every interface and alternate setting, in words, with which driver
    has claimed it - or "no driver"
every endpoint: address and direction, type, wMaxPacketSize,
    bInterval, and the SuperSpeed companion's bMaxBurst where there
    is one
```

`usb devices <n>` prints one device and then its raw descriptor bytes - the device descriptor and the configuration descriptor with everything nested under it - as a hex dump. That is what is needed when the decoded view is not enough: a vendor descriptor this monitor does not name, a field somewhere the specification does not put it, a device lying about its own lengths.

> **Note.** A string row never prints blank. A device that HAS no serial number

says `none` and a device that has one and would not hand it over says `not read`, because those are different facts and a blank line for both is how a reader concludes the first when the truth is the second.

> **Trap.** The walk over a configuration descriptor refuses rather than

guessing. A record that reports a length of zero would send a naive walk round forever; a record that claims to run past what the device actually sent would walk off the end of the buffer and print whatever is behind it as an endpoint. Both stop the listing with a sentence saying where. A malformed descriptor is not an exotic case - it is what somebody bringing up a new device is most likely to be holding.

The decoding lives in `Anvil/Bus/usb_core.pbi` and knows nothing about any host controller, so it is the same code on every board Anvil is built for. A board supplies two procedures that carry a control transfer, and nothing else.

Where the USB stack spends its time: `usb timing`

`usb tree` says what is plugged in. `usb timing` says what the waiting cost. It prints one row for every place the USB stack waits - not one row per event - with how many times it waited there, the total, the worst single visit, and when the site was first reached and last left.

```
pmf> usb timing
pmf> usb timing reset
```

Every figure is counted on the board's own 54 MHz architectural counter by the driver that does the waiting. Nothing about the console is in it. That matters more than it sounds: every storage figure this monitor quoted before build 195 was timed by a host waiting on the network console, and a console exchange has a floor of about four hundred milliseconds - so a one-block read and a one-megabyte read measured the same, and the difference between them was noise.

`usb timing reset` zeroes every row. The pattern that answers "what does this one command cost" is: reset, run the command, read the table.

> **Note.** Rows, not a timeline, and that is deliberate. The boot trail is a

timeline and its ring holds 64 entries, while a boot writes about 161 - so the thirteen seconds worth reading are overwritten by the four hundred milliseconds that follow them, and the record of a slow boot is destroyed by the end of that same boot. A fixed row per wait site cannot wrap. A count beside a total also says more than a timeline does: 800 ms of port reset is either one slow port or four fast retries, and those are two different faults with two different cures.

What `usb storage` says about the stick

`usb storage` names the device, its capacity, and then three things about what it is being driven AS. The lines a real board prints are added here from the first session that runs it; nothing is shown that has not been on a board.

`transport` is the interface protocol actually in use - `50` is Bulk-Only Transport - and the length of the configuration descriptor the device published. If the device ALSO advertises USB Attached SCSI, the command says so in full sentences, because a stick that offers UAS and is being driven Bulk-Only is not being asked for everything it can do: UAS queues several commands at once where Bulk-Only pays a round trip per command. Anvil has no UAS transport, so it says which one it is using rather than leaving the reader to infer it.

`burst` is `bMaxBurst` out of each bulk endpoint's SuperSpeed Endpoint Companion descriptor, AS CONFIGURED - which is not always what the descriptor asked for. This controller cannot burst a bulk OUT endpoint of a mass-storage device that sits behind a hub, so such a device is configured with a burst of 0 and the printed value is the 0, not the 15 the descriptor offered.

> **Trap.** Until build 195 this driver asked for only the first 64 bytes of a

device's configuration descriptor and then refused, silently and with no error set, anything whose descriptor was longer. The commonest reason for a longer one is a second alternate setting - which is exactly what a USB 3 stick that also speaks UAS has. Such a stick was reported as "not mass storage" and nothing said why. The descriptor is now read in two steps, the length field is obeyed, and a descriptor that still will not fit is refused by name.

save

```
pmf> save KERNEL8.IMG 400000 18CE24
```

Name, address, length - the last two in hex. The length is bytes. Leave the length out and the file keeps the size it already has; give one and the file is **resized** to match.

What you will see, from a recorded board session:

```
The file KERNEL8.IMG on the boot medium is 1809100 bytes long at the moment.
It is being resized from 1809100 bytes to 1625636 bytes to match what
you asked to write.
Writing 1625636 bytes out of memory, starting at 00400000. This
takes a moment and must not be interrupted.
written.
  Saved KERNEL8.IMG on the boot medium.
  That file now holds what was in memory, and the
  block writer has been disarmed again so nothing else in this monitor
  can reach the medium. Saving does not change the boot-file selection;
  reset still loads the configured boot image, not an arbitrary saved file.
```

> **Note.** `written.` is a protocol string too - the update tool checks for

exactly that word. And notice the last sentence about the block writer: the medium is not writable at rest. `save` arms the writer, writes, and disarms it again, so nothing else in the monitor can reach the medium by accident.

> **Trap.** The failure to memorise:

```
!! the write did not finish. THE FILE MAY NOW BE PART OLD AND PART NEW.
   Write it again before resetting the board, or it will try to boot
   half of one image and half of another.
```

If you see that, **do not reset**. Chapter 8 is entirely about that moment.

Getting an image in over the network

```
net recv <port> <address> [max-length]
```

Port decimal, address and length hex. It opens a TCP listener for exactly one connection, then closes it.

```
pmf> net recv 5001 400000
Listening on port 5001 on every usable addressed interface for ONE connection:
    wired at 192.168.137.2
  Everything it sends goes to 00400000 and the last byte it may
  write is 07EFFFFF, which is 128974848 bytes.
  Send the file and close the connection; the length and the SHA-256 of
  what actually landed are printed here when you do. Any key stops it.
```

And when the host has sent it:

```
  Received 1625636 bytes from 192.168.1.24 to 00400000 in 7676 ms.
      That is 206 KB/s.
      sha256 85370e48df79c7fe06956c2c47a27fef049b3e475d44cbba6395422df6ab463f
      The digest is of THE MEMORY, taken after the transfer, and it took 32578
      ms. Compare it with sha256sum of the file on the host: equal means
      the bytes in this board's DRAM are the bytes in that file.
      Nothing has been booted and nothing has been written to the medium.
      boot mem 00400000 enters it if it is a container; save <name>
      00400000 0018CE24 writes it to the boot medium.
```

> **Note.** A close is how the end of a file is announced. A host that stops

sending without closing looks exactly like a host that has crashed, and the board cannot tell them apart - so a silent peer is reported as an **incomplete** transfer and what is in memory is part of an image. Nothing may be booted from it.

run, go, and boot

```
run [address]        also go, g - enter a raw image
boot mem <address>   enter a PMFBOOT container already in memory
boot <name>          read a container off the medium and enter it
boot <path>          the same for a container in a folder or on partition 2
programs             list the containers on the medium (Pi 4)
boot status          what is armed, and the loop-guard counter
boot ok              clear the loop-guard counter
boot clear           stop booting a file automatically
```

The difference is the container. `run` jumps at an address and trusts you. `boot mem` reads the 96-byte header there, checks it, places the image **at the address the payload was built for**, verifies its SHA-256, and only then enters it.

What you will see when a payload is entered:

```
Starting the payload at 00500000. Control leaves the monitor now, and
```

```
anything printed below this line came from the payload, not from Anvil.
```

and when it comes back:

```
The payload returned to the monitor. Its x0 register held 000000000000000A,
which is whatever it chose to return. A stock image never returns at
all, so seeing this line means the payload was built to come back.
```

Sixteen hex digits, because x0 is a 64-bit register and always was.

> **Trap.** `boot mem` places the image **where the container says**, not where

you put the container. For a monitor image that address is `00200000` - on top of the running monitor. Entering a monitor image with `boot mem` takes the board down with no console. Chapter 3 says this too; it is worth saying twice.

Programs on the medium

`boot` takes a path as well as a bare name. A name with no prefix is a file in the root of partition 1, the FAT partition the board boots from, and that partition takes 8.3 names only. A path starting `2:/` is on the data partition, where folders and long names are fine. Quote a path that has spaces in it:

```
boot SLOW.PMF
boot 2:/TruckGPS/tgdemo.pmf
boot "2:/my programs/slow.pmf"
```

On the Pi 4, `programs` lists every container it finds: the root of partition 2, each folder of that root, and the root of partition 1. A container is recognised by its first eight bytes (`PMFBOOT` and a zero), not by its name. What it prints, quoted from `Anvil/Core/programs_cmd.pbi` (the rows are name, size and path):

```
Programs on the boot medium - type a name to run it:
  <name>            <bytes> bytes   <path>
  Typing a name is the same as boot <its path>. A command of the same name wins.
```

and on a medium with none, quoted from the same `Anvil/Core/programs_cmd.pbi` :

```
No programs were found. A program is a payload container (.pmf) in the root
of partition 2, in a folder of that root, or in the root of partition 1.
```

Typing a listed name runs it: the file name without `.pmf` , in any case. `tgdemo` at the prompt prints `Running 2:/TruckGPS/tgdemo.pmf` and then does exactly what `boot 2:/TruckGPS/tgdemo.pmf` does - the container's own load address, its SHA-256 checked before the jump, the deadman if one is armed. You do not have to type `programs` first; the first name typed in a session builds the list quietly.

> **Note.** A program's name only runs when no command has it. The prompt asks

for programs last, after every command, so a program called `ls` can never take the place of the command. Nothing here starts anything by itself: starting a program at power-up is the `boot.file` setting's job (`setenv boot.file SLOW.PMF` , then

`saveenv`; `boot status` and `boot clear` above).

The deadman

```
deadman <seconds>    1 to 15, decimal
deadman off          also deadman 0
```

It is a one-shot hardware watchdog, armed by the monitor immediately before it enters a payload and stopped immediately after the payload returns. **It is never running at the prompt.**

```
The deadman watchdog is armed for 15 seconds. If the payload hangs and
never comes back, the board will reset itself rather than sit there.
```

```
The payload came back, so the deadman watchdog has been stopped. The
board will not reset itself.
```

Trap. The ceiling is about sixteen seconds and the reported value is the

floor, never the request. Ask for `deadman 30` and it is accepted, clamped, and reported as 15 - because a deadman that fires earlier than it said it would is worse than none at all.

Note. Anvil never pets the deadman for the payload. The payload pets it

in its own loop, and that is the whole reason the deadman makes a wedged board impossible: a monitor that petted a watchdog on behalf of a program it is not running has disarmed the one thing standing between a hung payload and somebody driving to the bench.

What happened to the last payload

```
last run
```

Trap. This command is in **monitor build 152**, which is built and not yet

flashed. Build 151 and older answer it as an unknown word.

The monitor writes down what became of every payload it enters: at the jump, at the return and at a processor exception, in the boot phase record's page, which a reset does not erase. `last run` prints that record - one machine line for tools, then the same facts as sentences:

```
LASTRUN <run> <state> <restarted> <deadman> <x0> <address> <ms> <esr> <pc> <far>
```

`state` is 0 nothing recorded, 1 entered and not back, 2 returned, 3 stopped at a processor exception. `restarted` is 1 when the board has booted since the payload was entered - the record counts boots - which is how "the deadman reset the board"

is something the board says rather than something a host guesses. The run number rises at every jump and at nothing else, so a refused `boot mem` leaves it where it was.

> **Note.** It exists because a return line can be lost on the network while the

payload came back perfectly well. Chapter 6 has the measurement, and the host runner asks this command whenever a return line does not arrive.

> **Note.** The deadman is **one-shot**, as it says: the setting is taken at the

jump, so the payload it was armed for spends it and the next payload runs without one unless `deadman` is typed again. A refused jump leaves it armed. Until build 160 the watchdog was stopped after a return but the setting stayed armed for every later payload (forum 879).

The boot trail

```
trail
```

In the shipped image since build 165. Every step of the configuration half of the boot - the PCI Express and USB host phases, every wait for a USB event and whether it ran out, every SCSI command sent to the USB stick with the status and sense that came back, every block read, and each step of mounting the filesystem - is written with its millisecond into a ring of the last 64 in the boot phase record's page, which a reset does not erase. The ring the previous boot left is kept beside this boot's, and `trail` prints both: `TRAIL 1` lines are the previous boot, the last one being where it got to.

A 15-second watchdog, petted by every one of those steps, covers the configuration half only. A boot that has stopped is reset once, and the boot after it prints `!! THE PREVIOUS BOOT STOPPED` with its last six steps on the screen and the serial line before USB starts. The watchdog is never armed on the boot straight after a stopped one, so it can cost one extra reset and never a board that resets for ever.

A boot that is merely slow is not reset. Every USB wait lasts five seconds at most and records a step as it starts, and every SCSI command and every retry of a stick that is not ready records one, so the longest stretch with no step on a working board is about five seconds against the watchdog's 15. The hardware cannot count past a fraction under 16 seconds, which is why the bound is 15 and why the steps are what keep it safe.

> **Note. If a boot fails, send this:** the `!! THE PREVIOUS BOOT STOPPED`

line and the steps under it (a photo of the screen is fine), the output of `trail` from the boot that did reach the prompt, `version`, and what is plugged into the USB ports. That is enough to say which step a boot stopped in, which is where a fix starts (forum 880). A power cut clears the record, so type `trail` before switching off.

Settings that survive a power cycle

```
settings                list everything, passwords hidden
settings set <name> <value>     the value is the rest of the line
settings remove <name>
```

```
settings reveal <name>   print one in full, hidden or not
settings save            write SETTINGS.TXT to the boot medium
settings load            read it back
settings discard         accept a part-read file so a save is allowed
```

Nothing loads the settings file for you after a reset; `settings load` does it, and bringing the medium up is slow and noisy, which is why it is not done unasked.

> **Note.** Anything whose name contains `password` , `passphrase` or `secret`

is masked in the listing, because that output also goes to the on-screen console and into the next screenshot anybody takes. `settings reveal` is the deliberate keystroke that puts one on the screen - which is why it will not do it for a name you did not type.

> **Trap.** The refusal that saves your configuration file:

```
!! refusing to save. The last load stopped at a bad line, so the table
   in memory holds only the part of SETTINGS.TXT above it, and a save
   rewrites the whole file from the table - it would delete every line
   after the bad one, silently and permanently.
```

What SETTINGS.TXT looks like on the medium

You do not have to write it by hand - `settings save` writes it - but it is a text file and you will read it sooner or later:

```
key=value        one to a line; the FIRST '=' splits, later ones are
                 part of the value
keys             lower case, a-z 0-9 . - _ , at most 31 bytes
values           printable ASCII, at most 95 bytes, and may NOT begin
                 or end with a space
# or ;           a comment line
```

Keys are case-insensitive on the way in and always written back in lower case. Blank lines are ignored. A save writes LF endings; a load accepts LF, CRLF or a lone CR. At most **32 keys and 6144 bytes**, and a UTF-8 byte-order mark is skipped only at the very start of the file.

> **Trap. Your comments are not kept.** A save rewrites the whole file from

the table in memory, so the first `settings save` after you hand-edit the file deletes everything you wrote in it and replaces it with Anvil's own banner. This is also why a part-read table refuses to save at all.

An unknown key is **not** an error - it is simply stored - so a misspelled name becomes a new setting rather than a complaint. The names with code behind them are the network addresses (`net.address` , `net.netmask` , `net.gateway` , `net.server` , `net.dns`), the four Wi-Fi slots (`wifi.N.ssid` , `wifi.N.password.plaintext`), the clock (`clock.utc` , `clock.zone` , `ntp.server`), the autoboot (`boot.file` , `boot.delay` , `boot.maxfails` , `boot.fails`), the fan (`fan.pin` , `fan.mode` , `fan.lowF` , `fan.highF` , `fan.hystF` , `fan.hz`) and the panel (`screen.rotate` , `screen.scale` , `touch.axes`). The fan

thresholds carry their unit in the name because they are Fahrenheit and an older Celsius spelling existed; the old names are read once to migrate and then removed.

> **Note.** A commented example of the whole file, safe to copy, is in the

repository at `Boards/RaspberryPi4/sdcard/SETTINGS.TXT.example`, beside the boot-volume skeleton chapter 3 points at.

> **Trap. A passphrase in this file is plain text**, on purpose: the radio

needs the passphrase itself, and hiding it on a medium whose reader also holds the key would only look like protection. After a successful handshake Anvil also caches the derived key as `wifi.N.pmk.secret` so the next boot skips a ten-second derivation - and **that value joins the network as well as the passphrase does.** Never copy one to another board or into a file you share.

Network and radio

```
net                   show the four addresses and the wire
net address <a.b.c.d> / netmask / gateway / server / dns
net recv <port> <addr>
net up / net down / net forget
net stats             what each network controller counted
dhcp                  ask for an address now
get <name> [address]         fetch over TFTP
put <name> <address> <len>   send over TFTP
ping <host> / dns <hostname>
http serve [port]     default 8080, / and /health only
```

```
wifi                  list stored networks, passphrases hidden
wifi ssid <slot> <name>      the name is the rest of the line
wifi phrase <slot> <phrase>  the phrase is the rest of the line
wifi scan / join / rejoin / ip / forget <slot>
wifi link [on|off]    the keepalive and auto-reconnect
```

Four networks can be stored, in slots 1 to 4, and **the number is the order they are tried in** - slot 1 first.

`dhcp` takes no arguments. What you will see on success:

```
Lease accepted on wired:
  address   192.168.1.25
  netmask   255.255.255.0
  gateway   192.168.1.2
  server    192.168.1.2
  lease     86400 seconds remaining
The lease is runtime protocol state and is not written over saved static
settings. Renewal, rebinding and expiry continue automatically.
```

> **Note.** That last sentence is a design decision worth noticing. A lease is

not a setting. It is never written over the static addresses you saved on purpose, and it renews itself without you.

`net stats` (also spelled `net counters`) answers a different question from `net`. `net` prints this monitor's own tallies - frames in, frames out, refusals. `net stats` prints what the network HARDWARE counted, for every interface, whichever one traffic is currently leaving by. It reads and prints and changes nothing, so it is safe to type twice in a row and compare.

Under THE WIRED ETHERNET it prints the Ethernet controller's own statistics since the interface last came up: pause frames sent and received, good frames received and sent, and then only the error and overrun counters that are not zero, with the number of bring-ups and auto-negotiation restarts beside them, because every bring-up clears the block. The first line always prints:

```
  Pause frames this board SENT: 0.
```

That line must read zero. A non-zero number means the board is telling the machine at the other end of the cable to stop transmitting, and the command says so in capitals. If the controller has never been started, it says that instead and reads nothing.

Under THE WI-FI RADIO it says whether the radio holds an address, and that the radio has none of those counters: the CYW43 keeps its equivalents in its firmware and this driver does not ask for them. An absent number is not a zero. For the radio, `net` and `wifi status` are the readings.

Who the network console belongs to

The network console answers one host at a time. The first datagram that arrives while nobody owns it makes that host the owner, and every other host is answered `BUSY` while the owner's command runs. When the command finishes, the console is nobody's again.

A host that stops talking cannot keep it:

```
an empty datagram           never takes the console, and is not answered
an owner at the prompt      loses it the moment another host sends, if
  with nothing typed        nothing it sent is still waiting
an owner at the prompt      loses it after 60 seconds of silence; the
  with a half-typed line    half line is erased, never run
an owner whose command      keeps it, however long the command is silent
  is running
```

So a host tool that finishes and exits hands the console on to the next host at once. An empty datagram is how a host keeps its own flow alive through a firewall during a long command; it neither claims nor refuses anything. See forum topic 888.

reset

```
pmf> reset
Resetting the board now. The firmware reloads this monitor from the boot
medium and it starts again in a few seconds. Nothing in memory survives.
```

No confirmation prompt. `reboot` and `r` are the same command.

The board itself

```
info               what board this is, its memory, clocks    also board, i
uptime             also up
clock [rate]       rate in DECIMAL
cache              turn the caches on or off, and report
gpio ...           pins in DECIMAL
i2c probe / read / write / speed
usb info / tree / storage / read
mmc info / read / write
fan ...            the PWM fan and the part's temperature
mod ...            driver modules - chapter 9
screen ... / screenshot / shot ...
el3test            one bounded secure-timer interrupt, then one BRK
coretest run|stop|status     chapter 7
```

Trap. Number bases are not uniform and there is a reason for each. Memory

addresses, lengths and CRCs are **hex**, because that is what an address is. GPIO pin numbers, clock rates, `deadman` seconds, `sleep` seconds and `net recv` port numbers are **decimal**, because that is how those things are written on a schematic, in a datasheet and in a socket. When in doubt, `help` states the base for each command.

Note. `el3test` and `coretest` are explicit diagnostics and are never

boot actions. There is no automatic start and no payload form of either. After `coretest run`, restart before changing cache policy or entering a payload - chapter 7 explains exactly why.

info, and how much memory this board has

`info` is the board's own account of itself. It prints what the monitor knows without asking anybody - what it refuses and why, and which exception level it came up at - and then what the firmware says: the board, its revision code, **how much memory is fitted**, the temperature, the clock rates and the I2C bus.

The memory row is decoded from the board revision code the firmware reports, which carries the size in a field of its own. That is a different question from the mailbox tag called "get ARM memory", which answers with the ARM's share of the first gigabyte - about 948 MB on a 4 GB board - and as a memory size would be wrong by a factor of four.

```
board       Pi 4 Model B, BCM2711, 4 GB, rev 1.1   (revision code $00C03111)
RAM         4 GB, from the board revision code the firmware reports.
            An address past it is refused, naming that size. Memory
            runs from zero, stops at the peripheral aperture and
            resumes above four gigabytes; both runs are inside the
            one figure above.
```

Note. Those lines are copied from the monitor's **source**, not from a

recorded board session: the RAM row is new and no board has printed it yet. The `board` row above it is the line `info` has always printed. An image built before this change prints that row and goes straight on to the temperature.

Why the monitor cares. A Raspberry Pi 4 is sold with 1, 2, 4 or 8 GB fitted, and Anvil is the same image on all of them. Until it could size itself, `memory 80000000` on a 1 GB board read an address with no memory behind it, got back whatever the bus left on the wire, and printed it under a heading that says it is the contents of memory. On that board it is now refused, in the same shape every other address refusal takes:

```
pmf> memory 80000000 40
!! memory was refused: 80000000 .. 8000003F
   is above the 1 GB of RAM this board has.
   That space runs 40000000 to FBFFFFFF.
   Nothing was dumped. THIS REFUSAL IS KNOWLEDGE, NOT IGNORANCE:
   the board asked how much memory is fitted and this address is
   past it. Reading it would not stop the board - it would answer
   with whatever the bus left on the wire, which is the worse
   outcome: a plausible value under a heading that says memory is
   indistinguishable from the truth, and a stop at least announces
   itself. info says what the size is and where it came from.
   If you have a reason to touch it anyway, type
       force 80000000
   and then repeat the command on the very next line. That arms this
   one address for one command and nothing else. Typing the address
   twice is the confirmation - there is no flag, because a flag is
   something a script can carry without anybody having decided.
   info lists what this board does know, and where each answer came
   from.
```

The same refusal stands in front of `load`, `save`, `fill`, `copy`, `compare`, `crc32`, `sha256sum`, `mtest`, `readback`, `net recv` and `boot mem` - every command that takes an address from you - and each one names its own verb on the "Nothing was" line.

> **Trap.** **Memory is in two runs on this board, and you can only reach the

first one. **It starts at zero, STOPS at `FC000000` where the peripherals are, and resumes at `100000000` - so a 4 GB board's last 64 MB lives above four gigabytes.** This monitor does not map any of that.** Its page tables cover the low four gigabytes and the PCI Express window and nothing in between, so an address like `103FFFFC0` is a translation fault, and a fault here is not a refusal: Anvil prints one fatal line and stops, the console dies with it, and the board has to have its power removed. That is measured, not theorised - it is how this paragraph came to be rewritten. Every address above four gigabytes except the PCI Express window is therefore refused, and the refusal says the tables rather than the size. A 1 GB or 2 GB board has no second run at all. The one size `info` prints is the memory FITTED, which is why it is a size and not a top address.

Nothing is refused on a board that cannot say what size it is. If the mailbox is down, or the firmware returns an old-style revision code with no memory field in it, the RAM row says which and the guard stands down rather than guessing - the same behaviour every image had before this change.

The size reaches the hardware too. The PCI Express controller's inbound DMA window - the aperture the USB host controller reads your memory through - is programmed to the smallest power of two that covers the memory fitted, which is what Linux and U-Boot do: 1 GB gets size code 15, 2 GB gets 16, and 4 GB and 8 GB both get 17, because the wrapper erratum stops

inbound transfers at 3 GB and a larger window would describe address space nothing can use. You never type anything for this. The one time it says something is when the size is unknown:

```
!! this board could not say how much memory it has, so the PCI Express
   inbound window is programmed at its largest useful size instead of
   being fitted to the memory. That is the safe direction - a window
   bigger than the memory behind it costs nothing - and transfers are
   still refused above the 3 GB the wrapper erratum allows. Type info
   to see why the size is unknown.
```

It is printed once per boot, the first time anything brings the USB controller up, and it is a note rather than a failure: the window is then exactly what every image before this change programmed.

There is no loop command, on purpose

`help` says it in one line:

```
  loop is deliberately absent - it cannot return at all.
```

That is the console's whole philosophy in a sentence. A command that can take the prompt away from you and never give it back is not a convenience.

Writing a payload

A payload is an ordinary program that borrows the whole machine for a while and gives it back. This chapter is the contract.

What a payload is

It is a compiled image. Anvil places it, verifies it, and branches into it with the processor entirely its own. When it returns, Anvil takes the machine back and prints what the payload put in `x0`.

What makes it a payload rather than a boot image is two build switches and the promise that goes with them: it comes back, and it does not touch what it did not claim.

The three addresses

Every image this compiler produces needs three addresses, and you will have to think about all three.

```
--load-addr <addr>    where the image is linked; unless --bss-addr is
                      also given, its bytes do not depend on it
--bss-addr  <addr>    where its variables go
--stack-addr <addr>   where its stack starts, growing down
```

> **Note. The load address is not baked into the image.** The code reaches

its own code, data and variables relative to where it is running, so the same source built `--load-addr 0x400000` and `--load-addr 0x1000000` is byte for byte the same file, and it runs wherever it is staged. `tools/a64/a64_nosizecap_check.py` proves both: it compares those two builds whole, and enters the `0x400000` one at a different staging address and gets its return value back. So do not rebuild when the monitor grows - stage at the address `map` prints.

> **Trap. `--bss-addr` pins an image to its load address.** Give an

explicit BSS address (or a `BssAddress` line) and the variables are at that absolute address while the code still finds them by distance, so the image then runs only at the address it was built for.

> **Trap. The BSS is not covered by any range check the loader makes.**

Leave `--bss-addr` off and the variables go 512 KiB above wherever the image is running. The loader never writes a byte of them, so no window check ever sees them. A payload staged near the top of a window puts its variables past the end of that window at run time - and in the low window's case, that is the payload's own stack corridor. Leave room, or use the high window, where there are three gigabytes of it.

> **Trap. The compiler's default stack top is a hardcoded `$08000000`,**

for monitor and payload alike. Leave `--stack-addr` off and your payload's very first push lands on the monitor's live frames, and returning walks off into the weeds. That is also why the low payload window stops at `$07EFFFFF`: one clear megabyte below that default.

> **Note.** You can put all three in the source instead of on the command

line:

```
LoadAddress  $400000
BssAddress   $480000
StackAddress $3000000
```

A flag still wins where it is the only thing said. If a flag and a source line **disagree**, the build stops and names both values. It never quietly picks one.

The two switches

```
--entry-returns     make the entry point return instead of trapping
--wants-services    ask for the service table in x0
```

A stock image's entry point traps forever after calling `Main()`. `--entry-returns` turns that trap into a real epilogue, so a payload that ends in a return comes back to the monitor - and the monitor can report your `x0`.

`--wants-services` sets a flag in the container header. That flag is the only thing that makes Anvil build its service table and hand you its address.

> **Trap.** `--wants-services` and `--wants-dtb` are mutually exclusive, and

the pair is refused before a byte of the container is read. Pick one.

What you may assume at your first instruction

Everything in this list is guaranteed. **Nothing else is.**

```
x0        the address of the service table. Never 0 for a payload that
          declared --wants-services.
x1..x7    ZERO. Not "unspecified" - zero.
x8..x30   unspecified. You own them.
SP        your own stack, at the address --stack-addr declared, ALREADY
          SWITCHED.
IRQs      masked. Anvil's fault vectors are installed and stay installed.
image     placed at your load address, digest-verified AT THAT ADDRESS,
          BSS zeroed - all before entry.
cores 1-3 parked.
deadman   armed if the operator armed it. YOU pet it, in your own loop.
```

> **Trap.** **The exception level and the cache state are not in that list, on

purpose.** You run at the same level Anvil runs at, whatever that is - and you must not hard-code a number. `SvcMmuState()` answers the cache question: bit 0 MMU on, bit 1 D-cache, bit 2 I-cache.

This is not theoretical. A payload that hard-coded "we must be at EL2" refused to run at all - correctly, having computed nothing - the day the monitor started booting at EL3 through the firmware stub. The refusal was right. The check was wrong. Ask.

The service table

`x0` points at a table. Sixteen bytes of header, then one 8-byte pointer per slot:

```
+0   u32  magic        'ANVS'
+4   u32  abi_major
+8   u32  abi_minor
+12  u32  entry_count
+16  ptr  slot 0
+24  ptr  slot 1
     ...  entry_count pointers, 8 bytes each
```

At ABI 1.3 that is 16 + 192*8 = 1552 bytes. A slot's address is `table + 16 + n * 8`.

> **Trap.** Read those four header fields **a byte at a time**. The fields at

`+4` and `+12` are not 8-byte aligned, and with the MMU off all of DRAM behaves as device memory where an unaligned wide load **faults** - into an exception vector your payload does not install. That is a silent hang from a single read. The slots themselves are 8-aligned and read normally.

Every slot is filled. An unimplemented one points at a procedure that returns `SVC_ENOSYS`. There are no null pointers in the table, so you never have to check one.

The version gate, and it is not optional

```
major differs   DO NOT CALL A SINGLE SLOT. The only thing guaranteed
                across a major version is the sixteen header bytes.
minor older     you may call slots, but refuse with SVC_EABI naming
                both versions, so somebody can act on it.
minor newer     NOT an error. A newer table is fine.
```

The numbering rule behind that: a slot's meaning is fixed for the life of a major version. New services are appended inside their group's reserved block and the minor rises. A group that fills its block gets a **new block at the end**, never a renumbering.

The groups

```
base  group                 reserved   state today
   0  Core and lifecycle        16      all 16 live
  16  Clock                      8      5 live; there is NO clock setter
  24  Console                   16      14 live, where the board has one
  40  Touch                      8      4, or ENOCAP
  48  File and storage          16      14 live
  64  Settings                   8      4 live
  72  Net                       24      EMPTY. Every slot answers ENOSYS
  96  USB                       12      6; the block writer is ENOSYS
 108  GPIO                       8      6 live
 116  I2C                       12      8 live
 128  SPI                        8      5, all ENOSYS today
 136  UART                       8      6, all ENOSYS today
 144  Vehicle link              12      11, all ENOCAP today
 156  GNSS                      12      7, all ENOCAP today
 168  Crypto and entropy        16      ONE live: SvcSha256
 184  Parallel execution         8      3 live - new at 1.3
```

> **Note.** The net group being empty is deliberate and documented as such in

the source rather than left to be discovered. Read that table as the state of this build, not as a promise about the next one - and check `SvcAbiMinor()` rather than this book.

The lifecycle slots you will actually use

```
slot 4   SvcCapGet(cap)               does this board have that group
slot 5   SvcRequireCap(cap, name, why) the gate - returns 1 or 0
slot 6   SvcPet()                     feed the deadman
slot 7   SvcBootOk()                  "I came up" - clears the loop guard
slot 10  SvcMmuState()                MMU and cache bits
slot 11  SvcUptimeUs()
slot 12  SvcErrText(code)             the whole sentence for a code
slot 13  SvcErrDetail()               the fragment for the LAST failure
```

SvcRequireCap returns the sentence; it does not print it

This is the one place the payload ABI deliberately differs from the monitor's own internal gate, and the reason is worth understanding.

Inside the monitor, the capability gate prints its refusal, because the monitor owns the console. Here, **you** own the console. A sentence printed by Anvil would land on a framebuffer you are half-way through composing, or on a serial line nobody is watching in a truck.

So on a refusal it returns 0, sets the detail fragment to the reason string you passed in, and you render it. The wording is the monitor's wording minus the "type help" tail, which means nothing on a touchscreen.

Errors are sentences, and you compose them

`SvcErrText(code)` returns a **pointer** to a complete sentence that keeps the number in it:

```
SVC_OK        0    the operation completed at code 0 (SVC_OK)
SVC_ENOSYS  -100   this monitor has no code for that service at all...
SVC_ENOCAP  -101   this board does not offer that hardware group...
SVC_EARG    -102   one of the arguments was not usable at code -102 (SVC_EARG)
SVC_EBUSY   -103   the resource is already in use at code -103 (SVC_EBUSY)
SVC_ETIMEOUT -104  the operation did not finish in its time bound...
SVC_EIO     -105   the hardware answered, and answered badly...
SVC_EPERM   -106   this monitor refused on purpose at code -106...
SVC_EFLAG   -107   the payload's container did not declare the flag...
SVC_ESTATE  -108   this service was called in the wrong order...
SVC_EABI    -109   this monitor's service table is older than the
                   payload was built against...
```

> **Note.** The number stays inside the text on purpose, so that an old forum

thread, an old log line and an old screenshot still match a message printed today.

The house shape for the sentence you print:

```
<what it was doing> failed at <code> (<name>): <SvcErrText>.
<SvcErrDetail>. Check <first thing>, <second thing>.
```

> **Trap.** `SvcErrDetail()` is **not latched**. It is valid immediately after

the call that failed, and not one call later. Read it before you do anything else.

The deadman

The operator arms it with `deadman <seconds>` before entering you. You keep it fed:

```
SvcPet()       slot 6
```

> **Trap. Anvil never pets it for you**, and that is the whole point. A

monitor that petted a watchdog on behalf of a program it is not running has disarmed the one thing between a hung payload and a board that needs hands.

The numbers: the counter is 20 bits at 65536 ticks a second, so the ceiling is **about 16 seconds**. A request of 1 to 15 seconds is taken as given; anything larger is clamped and **reported as the clamped value**, because a deadman that fires earlier than it said it would is worse than none.

Trap. If your payload includes the safety library and calls its

`SafetyPet()`, that is **not** the same thing and it does **not** feed this hardware counter. A long-running payload that pets the wrong one is reset while perfectly healthy. Call `SvcPet()`.

Note. There is **no message printed when the deadman fires.** There

cannot be: the reset arrives at an arbitrary instruction while your payload owns the machine, and nothing is running that could print. What you see is the board restarting.

And an honest one: arming and stopping the watchdog are proven on this board. **Expiry has never been observed to fire.** The experiment that would settle it is written down in the safety library's own header.

What a payload must not do

Ownership. You may use your declared image, BSS and stack ranges, the processor state handed to you, and devices you explicitly claim. You must preserve Anvil's image and BSS, and you must not reconfigure unrelated devices. A payload that overwrites the resident monitor cannot be recovered by copying a few variables back, and Anvil does not pretend otherwise.

No callbacks, ever. The ABI is call-and-return only. Anvil never calls back into the payload. Where a future service looks like it wants a callback, it wants a **queue** instead - which is exactly why the touch and vehicle services are polls.

No slot from an interrupt handler. The handler would re-enter Anvil's code on top of a call the main line is already inside, and Anvil's drivers keep their state in globals with one writer and no lock. Interrupts are masked at entry and you have no reason to unmask them.

No slot from any core but the one you were entered on. At every version of this ABI. From 1.3 you may run your own procedures on the other three cores - chapter 7 - but a service call from core 2 is not allowed at any version.

Do not include Anvil's ABI source file. That file is Anvil's half: it declares the table, the procedures and the builder, all of which would be compiled into your image as a second, dead copy. What crosses the seam is the **numbers**, and a payload restates the ones it uses - exactly as a program written against any other ABI does.

Do not branch on the board id. `SvcBoardId()` exists so a diagnostic can print what it is talking to. A payload that changes behaviour based on it has broken the entire point of a portable service table, and there is a check that looks for exactly that.

Note. One thing that is **not** forbidden, despite what you might

expect: a `DataSection`. Payloads in this tree use them. The only real constraint is the alignment one above - the emitter aligns the section, and a wide load off an unaligned address with the MMU off faults silently.

Returning a value

Make `Main()` return a value and build with `--entry-returns`. It arrives in `x0`, and the monitor prints all sixteen digits of it:

```
The payload returned to the monitor. Its x0 register held 000000000000000A,
which is whatever it chose to return. A stock image never returns at
```

```
all, so seeing this line means the payload was built to come back.
```

> **Note.** Sixteen digits, not eight. `x0` is a 64-bit register and always

was; eight digits is the low half of it, and printing half a register is how a return value that differs only above bit 31 reads as a success.

The host runner treats a non-zero `x0` as a failed run by default, so the useful convention is: **zero means content**, anything else is your own status code.

> **Trap.** `SvcReturn()` exists in the table and **does not work** in ABI 1.x.

It refuses, and the refusal tells you what to do instead:

```
this monitor cannot unwind out of a payload from inside a service call; return from the payload's own entry point instead, which is what the returns-to-monitor flag is for
```

Where your payload may go

Type `map` on the board. Do not copy an address from anywhere else, including this book.

The low window's bottom edge is the monitor's own extent rounded up to a megabyte, so it moves every time the monitor grows. It was `00400000` on one build and `00500000` on the next. The high window starts at `40000000` and runs to `FBFFFFFF`.

> **Note.** The high window is the right answer more often than people expect.

Three gigabytes means you stop having to think about where your BSS lands, and an image linked there cannot collide with the monitor no matter how large the monitor becomes.

Staging assets at addresses

Large data - a model, a font, a texture, a recording - does not have to go inside your image. Put it in memory at a known address and have your payload read it from there.

Two routes. Over the network, into memory:

```
pmf> net recv 5001 40000000
```

and check what landed with the board's own digest:

```
pmf> sha256sum 40000000 <length in hex>
```

Or off the boot medium, which is much faster the second time:

```
pmf> load MODEL.PMW 40000000
pmf> sha256sum 40000000 <length in hex>
```

> **Note.** That second form is worth setting up for anything large. In one

recorded board session a 324 MB asset was uploaded once, written to the medium, and thereafter read back with `load` in a fraction of the time - and the board's own `sha256sum` confirmed it was the same file, **checked by something other than the tool that wrote it**.

> **Trap.** Check the size before you check the hash. In that same session the

file on the medium was 324,635,776 bytes where the payload expected 324,621,440. The size alone settled it and no hash was needed. Size is cheap; a 324 MB digest is not.

A complete payload

This is a real diagnostic from this repository. It spins for ten seconds on the hardware counter and returns 10.

```
; pi4SlowSpin.pi4 - looks like a hang, but gives up on its own.
;
; Armed with a SHORTER deadman, the two outcomes are distinguishable and
; only one of them can go wrong:
;
;   deadman fires   -> the board resets partway through. PROVEN.
;   deadman broken  -> this returns to the prompt. Board still healthy,
;                      and the failure is reported rather than wedging.
;
; The wait is on the architected counter via timer.pi4, not a spin
; count, so it is the same ten seconds at 600 MHz and at 1.5 GHz.

XIncludeFile "RaspberryPi4/Lib/uart.pi4"
XIncludeFile "RaspberryPi4/Lib/timer.pi4"

Procedure.i Main()
  Define i.i

  PrintNl()
  UartWriteStr("=== spinning for 10 s. a shorter deadman should cut this short. ===")
  PrintNl()
  UartDrain()

  ; One line a second, so the transcript shows exactly WHEN it stopped
  ; rather than only that it did.
  For i = 1 To 10
    Delay(1000)
    UartWriteStr("  t+")
    PrintDec(i)
    UartWriteStr(" s")
    PrintNl()
```

```
    UartDrain()
  Next

  UartWriteStr("=== reached the end. THE DEADMAN DID NOT FIRE. ===")
  PrintNl()
  UartDrain()
  ProcedureReturn 10
EndProcedure
```

> **Note.** Notice what the comment block is doing. This program exists to

prove the deadman works, and it is built so that **if the deadman does not work, the board is still healthy and the failure is reported.** A test whose failure mode is "everything is fine, and you know" is worth more than a purer one that has to be supervised.

Building it

This exact command was run on a desk machine while this chapter was written:

```
PureMetalForge.exe --compile RaspberryPi4/Examples/Diagnostics/pi4SlowSpin.pi4 -t pi4 --load-addr 0x400000 --
stack-addr 0x3000000 --entry-returns -o slow.img
```

What you will see - this is that run's output, word for word:

```
pmfc: BCM2711 --entry-returns; Main()'s value comes back in x0, the loader's stack and DAIF are left as they
were
pmfc: RaspberryPi4/Examples/Diagnostics/pi4SlowSpin.pi4  ->  BCM2711
> Intrinsics for BCM2711: bcm2711_hardware.def
> Loaded Hardware Intrinsics from bcm2711_hardware.def
pmfc: pin aliases from Boards/Raspberry_Pi_4.board
pmfc: 1423 lines of assembly
pmfc: wrote slow.img.pmf - a PMFBOOT v2 container, 5068 bytes
      load $400000  entry $400000  bss $480000..$48609F  sha256
95cd1f698a999a3afa9f56c894851ef996cf131038039afa7b025cd11335b189
      architecture AArch64  target BCM2711  stack $3000000
pmfc: wrote slow.img
pmfc: image 4940 bytes, occupying $400000..$40134B - no maximum;
      the board computes its map from this
pmfc: OK
```

Read the container line. It tells you the load address, the entry point, **where the BSS actually landed** - `$480000..$48609F`, which you did not ask for and now know - the digest, and the stack. That is the line to check against `map` before you send anything anywhere.

The container is 5,068 bytes and the image is 4,940, so the header is 128 bytes. Its fields:

```
+0   magic 'PMFBOOT' and a NUL
+8   version
+12  header length
+16  load address
```

```
+24  entry address
+32  image length
+40  BSS base
+48  BSS length
+56  flags   bit 0 entry-returns, bit 1 expects-DTB,
             bit 2 wants-services
+64  sha256 of the image bytes
+104 stack address
```

> **Note.** The digest is taken **at the load address, after placement** - not

in flight. A container whose bytes were damaged between the host and the board's DRAM fails the check with the target cell untouched, and nothing is entered.

Running it

```
pmf> deadman 15
pmf> boot mem 400000
```

or, from the host, the whole thing in one command:

```
python tools/board_run.py slow.img.pmf --console-ip <board-ip> --deadman 15 --expect-x0 10
```

Chapter 6 is that tool in full.

A checklist before your first run

```
1. map on the board. Write down the staging address and both windows.
2. Does your BSS land inside the window you chose? The compiler's
   container line tells you where it went.
3. Is --stack-addr set, and is it somewhere you own?
4. Both switches: --entry-returns, and --wants-services if you call
   a single slot.
5. Does Main() return a value, and is zero your success?
6. Do you pet with SvcPet() in every loop and every wait?
7. Did you ask SvcMmuState() rather than assuming the cache state?
8. Arm a deadman. Every time.
```

Optional Pi4 Bluetooth transmit buffering

The PL011 Bluetooth byte pipe remains blocking by default. A Pi4 payload which owns its HCI stack can opt in with BtuBufferedTx(1) after opening the link. Each successful put has copied its byte into a bounded 4095-byte FIFO; it has not necessarily reached the controller yet. Continue HCI polling: BtuGet drains only currently free hardware FIFO slots before reading RX. Keep all queue accesses on the same primary core. No background worker, interrupt ownership change or retained caller pointer is introduced.

BtuBufferedTx(0), baud changes and close flush pending software bytes with a 250 ms bound. Failure returns zero where a result is available, increments btu_txTimeout and records BtuWhy(). Close always restores the console and clears the queue

even when draining fails. Startup/open resets the mode; other boards and existing clients retain their previous blocking behavior. Queue peak occupancy is available as btu_txPeak for bounded diagnostics.

Running payloads from the host

You do not type payloads in at the prompt. One command sends it, proves it landed, runs it, and brings back the answer and a picture.

The tools, and what each is for

```
tools/board_run.py     one complete run: stage, verify, boot, return,
                       screenshot. The one you will use.
tools/board_model_run.py
                       a model run from a manifest: assets sent only
                       if not already in memory, run, readback, WAV.
tools/pi4_upload.py    bulk image transfer over the network, with the
                       hash verdict. Used on its own to put a file on
                       the board or on its boot medium.
tools/anvil_update.py  replace the monitor itself. Chapter 8.
tools/anvil_putfile.py put arbitrary files on the boot medium, with a
                       read-back check.
tools/anvil_wifi.py    the interactive network console. Chapter 3.
tools/pmfshot.py       a screenshot over the serial line.
tools/anvil.py         the shared serial driver the others import, and
                       a small command-line front end.
```

Trap. Do not run `python tools/anvil.py --help`. It has no argument

parser: its entry point constructs its serial driver for **any** argument, so `--help` opens the serial port. A bare `python tools/anvil.py` with no arguments is safe and prints the usage. Every other tool in the list parses its arguments before touching anything.

Ask the board where to put things

None of these tools has a built-in staging address, and that is deliberate.

Every tool in this tree used to carry `0x400000` as a constant of its own - four copies of one number on the far side of a cable, none of which could find out it had changed. The monitor has no fixed size; the first free address above it moves with every build. So the tools ask, with `map`, and one parser does it for all of them.

Note. If a monitor is too old to have `map`, the parser returns nothing

rather than guessing. A fallback to `0x400000` there would be reintroducing the constant with an extra step. Each caller says what it needs and stops.

One complete run

```
$ python tools/board_run.py --help
usage: board_run.py [-h] [--console-ip CONSOLE_IP]
```

```
                    [--console-port CONSOLE_PORT] [--board-ip BOARD_IP]
                    [--port PORT] [--addr ADDR] [--out OUT] [--name NAME]
                    [--tier {dma,v3d,cpu}] [--deadman DEADMAN] [--trace TRACE]
                    [--expect-x0 EXPECT_X0] [--timeout TIMEOUT]
                    [--ask-seconds ASK_SECONDS] [--shot-timeout SHOT_TIMEOUT]
                    [--settle SETTLE] [--twin] [--no-shot]
                    payload

positional arguments:
  payload               the .pmf container to run
```

The positional argument is the `.pmf` container - the one chapter 2 explained, and the one chapter 5 tells you how to produce. Not the raw image. The container carries the load address, so the tool does not have to be told it.

What the options mean, from the tool's own description:

```
--console-ip A     the board's UDP console. Default from --board-ip.
--console-port N   5555 unless somebody has changed it.
--board-ip A       where the image is streamed to. Defaults to
                   --console-ip.
--port N           the one-shot TCP listener's port. Default 5001.
--addr HEX         where the container is staged. Default: ask the board
                   with `map`.
--out DIR          where the run's files go. Default runs/<name>.
--name NAME        what to call them. Default the payload's stem.
--tier dma|v3d|cpu send `screen <tier>` before the run.
--deadman S        arm the deadman watchdog for the payload's window.
--trace HEX[:LEN]  read this many bytes back (default 256, at most
                   4096) after the run, with `readback`.
--expect-x0 N      accept this unsigned 64-bit return value.
--timeout S        how long the payload may take. Default 600.
--ask-seconds S    when no return line came, how long to wait for the
                   board to answer before calling the payload still
                   running. Default 60.
--settle S         twin mode only: how long to let it draw. Default 20.
--twin             the payload never returns.
--no-shot          do not arm a capture on return.
```

A real invocation, from this project's own records:

```
py -3 tools\board_run.py <path>\myPayload.img.pmf --console-ip 192.168.1.102 --tier dma --deadman 15 --trace
5A6000:280 --timeout 90 --shot-timeout 90 --out <path>\silicon-run --name my-run
```

Trap. **Never wrap this tool in an outer `timeout`, and never pipe it

through `tail` while it runs. **Its own `--timeout` is the bound on the payload and nothing else needs to impose one; a board that refuses to enter a payload answers in the round trip rather than running that bound out. An outer `timeout` kills the process where it stands, and `tail` prints nothing until the tool exits - so a long run that is working looks exactly like a hang, which is the state somebody kills a run in. Worse, the shell then reports** tail's** exit status and not the tool's.

That is not a hypothetical. On 2026-09-16 a run was driven twice as `timeout 600 python tools/board_run.py ... | tail -30`, about ten minutes each, and produced no output at all, an empty run directory, and `EXIT=0`. The board had refused the payload within milliseconds of each `boot mem` - its load address was inside the monitor - and had been idle at its prompt ever since.

Send the output to a file if you want to keep it, and put the run in the background if it is long. Do not put a clock or a pager in front of it.

What it leaves behind, while it is still running

The record and the transcript are written after **every** step, each carrying a `stage` field naming the step just reached:

```
map -> listener armed -> sent -> verified -> deadman -> boot ->
return -> trace -> picture
```

and the terminal stages `listener refused`, `no transfer verdict`, `upload did not verify`, `deadman refused`, `boot refused`, `still running`, `deadman reset`, `restarted`, `never entered`, `processor exception` and `outcome unknown`. A record that also carries a `finished` timestamp is one the tool wrote the ending of; one without it was killed where its `stage` says it was.

A payload whose return line was lost, but which the board says came back, goes on through `return`, `trace` and `picture` like any other, and its record carries `return_line_lost: true` and an `outcome` naming the evidence. There is no stage called "never returned" any more - see the next two sections for why that sentence was retired.

> **Note. An empty run directory is not something this tool can produce.**

The directory is not created until the payload has been read and the staging address is known, so a run that cannot start leaves nothing at all - and a directory is therefore evidence that a board was spoken to.

The exit codes say which of the three happened:

```
0   the payload returned what was expected and its picture is on disk
1   the run started and failed. The directory says where it stopped
2   the run never started. Nothing was sent to the board and nothing was
    written - not even the directory
```

A refusal is an answer, and it arrives at once

`boot mem` refuses for eight different reasons - the address lands inside the running monitor, the last transfer did not arrive cleanly, the resident secondary cores own the board until it restarts, the word after the address was not `shot`. Every one of them prints its sentence and goes straight back to the prompt, and **none of them is the payload's return line**.

So the boot step ends on any of three things: the return line, the processor-exception marker, or the board's prompt coming back after the echo of the command. The third is a refusal, and the tool quotes the board:

```
!! THIS RUN FAILED:
   the board's prompt came back 0.1 s after `boot mem 500000 shot` and the payload's return line never did, so
the board REFUSED TO ENTER THE PAYLOAD and nothing ran. The board said:
```

```
   !! resident secondary cores own this monitor until restart, so no
      payload was entered. Restart Anvil before run, go or boot mem.
```

> **Note.** The tool keeps **no list of refusal strings**. What is generic is the

shape - the prompt came back and the line being waited for did not - and what is specific is quoted verbatim from the board. A host tool that matched on the monitor's wording would go silent again the first time one of those sentences was reworded.

> **Trap.** The echo is what makes the prompt usable. The reply to any command

opens with the prompt it was typed at, so a bare `pmf>` would decide that every command had finished before it began - the opposite failure, and just as quiet. Only a prompt after the echoed line belongs to the command.

Nothing is asked of the board after a refusal. There is no new capture to read and no trace to dump, and streaming the previous run's picture back is exactly how a refused run would acquire a screenshot it has no right to.

When nobody is listening on the console port

A UDP console has no connection to refuse, so the refusal arrives as an ICMP port-unreachable that the operating system reports on the next receive:

```
!! THIS RUN NEVER STARTED, so nothing was sent to the board and nothing was
   written:
   nothing is listening on the Anvil console at 192.168.137.1:5555: this machine's datagram came back
unreachable ([WinError 10054] An existing connection was forcibly closed by the remote host). Either the board
is not powered or not on this network, or the address or the port is not its console's - check the board's own
`net` output for the address it answers on, and that UDP 5555 is the port. Nothing was sent to any board and
nothing was written.
```

> **Note.** "An existing connection was forcibly closed" is a strange thing to be

told about a protocol with no connections, which is why the number stays in the sentence: 10054 is what you will find if you search for it, and the sentence is what tells you it means nobody answered.

A long silence, and the host's firewall

The network console is UDP, and the PC's own firewall is stateful: it lets the board's datagrams in because this socket recently sent the board something, and it **forgets** that after a while of silence. A payload that computes for minutes and prints nothing is exactly that silence, and the return line it prints at the end is then dropped on the way in.

That was measured on the bench Pi 4 on 2026-09-16, with no payload at all: `sleep N` typed at the prompt, the host silent until the prompt should come back.

```
sleep 60     reply arrived
sleep 120    reply arrived
sleep 150    reply LOST
sleep 300    reply LOST
```

```
sleep 300    reply arrived - other interpreter, allowed on this profile
sleep 300    reply arrived - empty datagram to the board every 20 s
```

The cable was a Public network to the PC, and the Python it ran under had an inbound allow rule for Private networks only. That interpreter's replies depended on the firewall's memory of the flow; the one allowed on Public did not. On the same day three Kokoro runs of 158 s to 328 s had been reported as "never returned" while a 107 s one was not. A 7.7 KB payload that waits silently reproduced it: 300 s, returned, result block complete, capture taken on return - and not one datagram from any address reached the host socket between the boot line and the tool's next command.

So every wait in `board_run.py`'s console now sends an **empty** datagram to the board when nothing has gone to it for 20 seconds. The monitor takes an empty datagram as no keystrokes at all - the `sleep 300` above was not cut short by fourteen of them - and it keeps the flow alive. The same 300 s payload, through the fixed tool, on build 151:

```
  deadman 15 s
  boot mem 500000 shot
  returned expected x0 = 000000000000012C after 300.7 s
  picture 1280 x 800, capture number 3, v3d tier, 90 degrees, the monitor took it on return
Run complete: the payload returned expected x0 000000000000012C and the picture it left behind is on disk.
```

> **Note.** Allowing the interpreter on the network's profile also fixes it on

one machine. The keepalive fixes it for every machine this tool is run on, which is why the tool carries it rather than a setup step.

A missing return line is asked about, not believed

A line on a network can always be lost. What the host cannot do by itself is find out afterwards which of these happened:

```
the payload returned, and only its line was lost
the payload is still running
the deadman reset the board
the payload stopped at a processor exception
the monitor never entered the payload
```

So when no return line arrives inside `--timeout`, the tool asks the board. First it waits up to `--ask-seconds` for the board to answer at all - a payload that still holds the processor cannot, and nor can a board part-way through the boot after a deadman reset. No answer is the first outcome:

```
no return line arrived inside 600 s and the board has not answered its console for a further 60 s, so THE
PAYLOAD IS STILL RUNNING: it still holds the processor, which is the one state in which the monitor cannot
answer. A 15 s deadman was armed, so the payload is petting it and has not hung. Nothing was sent to the
board. Wait, or reset it.
```

If the board answers, the tool reads the monitor's **run record** with `last run` - which it also read just before `boot mem`, so a run number that did not move means nothing was entered. The record is written by the monitor at the jump, at the return and at a processor exception, in a page a reset does not erase, and it counts boots, so a reset under a running payload is a fact the board states rather than a guess the host makes. A payload the record says returned goes on to its trace and picture with the x0 the record holds.

> **Trap.** `last run` is in **monitor build 152**, which is built and not yet

on the board. On build 151 and older the word is refused, and the tool does the one honest thing left: if the capture on return was armed and its number moved, the payload returned and the capture's x0 is used; otherwise it says the outcome is **not known** - and that this is not evidence the payload never returned.

The sentences in this section are the tool's own wording at its default bounds, taken from the tool and exercised by its desk gate against a scripted board. The board-side proof of each outcome waits for build 152 on the board.

What a run actually does, in order

```
1. ask the board where to stage           map
2. arm a one-shot listener and stream     net recv <port> <addr>
3. prove the bytes landed                 the board's own length and
                                          SHA-256 of the MEMORY, compared
                                          against the file on disk
4. pick a renderer, if asked              screen <tier>
5. arm the deadman, if asked
6. note the capture sequence number       shot status
7. note the run number                    last run
8. run it                                 boot mem <addr> shot
9. wait for the return line and its x0,
   and if it does not come, ask           last run
10. read the payload's trace, if asked    readback <addr> <len>
11. read the picture                      shot
```

It writes three files into the output directory: `<name>.png`, `<name>.json` - the return value, the timings, the capture header, the digest of the decoded pixels and the transfer verdict - and `<name>.txt`, the whole console transcript. It exits **non-zero** if the return value is not what you expected, or if no picture came back.

A clean run ends like this:

```
Run complete: the payload returned expected x0 0000000000000000 and the picture it left behind is on disk.
```

A real result record from this repository, to show what the JSON holds: console `192.168.137.1:5555`, stage `00500000`, payload 801,108 bytes, expected and actual x0 both `0000000000723698`, transfer 801,108 bytes in 87 ms with its SHA-256, and a capture header saying sequence 2, 1280 by 800, rotation 90, tier `v3d`, source `dsi`, taken by "the monitor took it on return".

Step 3 is the one that matters

The board hashes **the memory**, after the transfer, not the bytes as they came off the wire. This tool hashes the local file. Equal means the DRAM that is about to be entered holds the file on your disk.

Unequal - by one bit - and the tool exits non-zero and boots nothing.

There are three ways step 3 can end badly, and each says something different:

```
!! the board did not arm a listener, so nothing was sent. It said:
```

```
!! the board never printed a length and a digest, so this transfer HAS NO VERDICT AT ALL - which is not the
same as a good one. Nothing was booted.
```

```
!! THE UPLOAD DID NOT VERIFY, so nothing was booted.
   host  <N> bytes <sha>
   board <N> bytes <sha>
```

Note. Read the middle one twice. "No verdict at all" is not a pass. A

tool that treated a missing answer as a good one would be exactly the silent-wrong-answer failure this whole project is built to avoid.

What a refusal looks like

Stage a payload on top of the running monitor and the tool stops before sending a byte:

```
!! REFUSING TO SEND: staging 801108 bytes at 00200000 would land on the monitor that is running this run
(00200000..004B144B). Leave --addr out and it goes to 00500000, which is where this board says to put it.
```

The upload tool's version of the same refusal names the overlap in bytes and says why it would be so hard to diagnose:

```
!! REFUSING TO SEND: staging <n> bytes at <addr> would overwrite the monitor that is
   running this upload. Its image is <lo>..<hi>, and the <n> bytes would land
   on <addr>..<end> - an overlap of <k> bytes.
   The board would keep answering and stop understanding commands, because its
   command names are strings near the end of the image. Nothing was sent.
   Leave --addr out and it goes to <stage>, which is where this board says to
   put it.
```

Trap. That is measured behaviour, not caution. A board that had its last

few kilobytes overwritten **kept its address, kept answering, and could not execute a single command** - because the command names are strings near the end of the image. It looked alive. It cost somebody a trip to the power lead.

Note. There is no override flag on that refusal. `--addr` already is

the way to say where something goes; a force flag beside the refusal would be a second path to the same place, which is how a safety check becomes decoration.

The monitor refuses the same thing from its own side. This is a recorded board session:

```
pmf> net recv 5001 200000
!! that range runs over the monitor itself, which lives at 00200000
   to 003FFFFF. Receiving there would overwrite the running
   monitor and take this prompt and this connection with it, so nothing
```

```
   was armed and no port was opened.
   Send the image to 400000 for the low window, or to 40000000 for the
   high one.
pmf>
```

Every run comes back with a picture

This is a rule, not an option, and it exists because of one afternoon: a payload ran four times and came back clean every time while nothing reached the panel - and then drew a garbled mess that nobody had been watching for.

The picture cannot be taken afterwards. When a payload returns, the monitor prints, and printing goes through the console onto the surface the payload left behind. The frame you want to look at is destroyed by the monitor's own report that the payload returned, before a prompt exists to ask for it. So the frame is copied out of the way **while it still exists**, into a reserved capture area, and read back from the copy.

Two ways it gets filled:

```
the payload asks   SvcScreenCapture(), after SvcFrameEnd. It copies and
                   does not present. This is the only way a payload's
                   own picture can be kept.
the monitor takes  shot arm arms one capture for the next payload entry;
it on return       boot mem <addr> shot arms and boots in one line. The
                   copy is the FIRST statement after the payload
                   returns, before a single character is printed.
```

The arm is one-shot. An arm that stayed set would overwrite the picture somebody was still reading with the next run's - and the next run is the one nobody was watching. A refused jump does not spend the arm.

The two checks that make a picture evidence

> **Trap. The stale-capture check is the one you need to understand.** The

capture area keeps the last picture until something overwrites it. A run whose capture silently failed would stream the **previous** run's picture - perfectly well formed, correct dimensions, no error anywhere. So the tool reads the sequence number before the run and fails a picture whose number did not move:

```
the picture that came back is number 2 and the board already held number 2 before this run, so no new capture
was taken. What was streamed is the PREVIOUS run's screen; this run has no picture.
```

The second check is the run count. The board counts the encoded runs it emitted and says so at the end of the stream; a lost datagram takes runs with it and the two numbers stop agreeing. A decoder that ignored the count would write a plausible picture with a band missing.

Everything the decoder can refuse says why, and says that nothing was written:

```
the stream stopped before its END marker; <n> runs arrived. Nothing was written, because half a picture that
decodes looks exactly like a display fault.
```

One command for a model run

A model run is more than one payload. The weights and the other inputs have to be in the board's memory first, and the answer is a region of memory - a waveform - that has to come back and become a file. Driven one tool at a time, that is eight steps with a wait between every two of them, and on 2026-09-16 a payload had returned on the board's screen minutes before its WAV existed. `tools/board_model_run.py` does the whole run from one manifest, in one console connection, with nothing typed in between:

```
python tools/board_model_run.py <manifest.json> --console-ip <board> --lease-file <BOARD-SHARE.md> --lane "
<name>"
python tools/board_model_run.py <manifest.json> --console-ip <board> --no-lease
```

The manifest names the payload and the value it must return, the deadman, every asset with its address and SHA-256, the regions to read back, and what to make of them. This is the Kitten TTS one from the ONNX compiler repository's `examples/pi4/kitten-rosie.run.json`, digests shortened:

```
{
  "format": "anvil-board-model-run 1",
  "name": "kitten-rosie",
  "base": "kitten-nano-0.8",
  "out": "pi4-rosie/board-model-run",
  "payload": {"file": "pi4-rosie/kitten-rosie.img.pmf",
              "sha256": "6e0c2bd0...", "stage": "0x03000000",
              "expect_x0": "0", "timeout_seconds": 900},
  "deadman_seconds": 15,
  "assets": [
    {"name": "weights", "file": "pi4/kitten.pmw",
     "address": "0x40000000", "sha256": "173baffd..."},
    {"name": "noise", "file": "pi4-rosie/kitten-rosie-noise.f32",
     "address": "0x54000000", "sha256": "9fcf6549..."}],
  "results": [
    {"name": "wave", "address": "0x57000000", "bytes": 2382400,
     "board_sha256": true},
    {"name": "trace", "address": "0x57E00000", "bytes": 256}],
  "post": {"wav": [{"from": "wave", "rate": 24000,
                    "file": "kitten-rosie.wav"}]}
}
```

Paths are relative to `base`, and `base` to the manifest's own folder; `--base` replaces it. A key the tool does not know is refused rather than ignored, and so is an asset whose file on disk no longer has the manifest's digest. `post` may also carry `compare`, a command run after everything else, with `{python}`, `{base}`, `{out}`, `{result.NAME}` and `{wav.NAME}` filled in.

What one run does, in order:

```
lease      the live-holder callout and a ledger row, re-read to confirm;
           released with a row at the end, whatever happened
identify   version, map, coretest status, cache (on for the digests)
assets     sha256sum <addr> <len> of what is ALREADY in memory: equal to
           the manifest, "resident" and no upload; otherwise net recv,
           the file, and the board's SHA-256 of where it landed
payload    net recv to its stage address, verified
```

```
run         deadman, last run, boot mem - and the moment it returns:
wave        readback of the regions a WAV is made from, checked by crc32
            and by the board's own sha256sum, then the WAV and the line
            "WAV ready: <path>"
the rest    the other regions, deadman off, coretest status, the cache put
            back, the lease released - then compare, if there is one
```

The second of two runs on build 160, with both assets already in memory from the first (a board transcript, run folder shortened):

```
[    2.1 s] lease: taken (2.0 s waiting), confirmed by two re-reads.
[    4.2 s] board: build 160 date 20260916 time 223809; coretest started=0 stopped=0 runs=0 nonce=285 error=0
core=0
[    4.6 s] asset weights: asking the board for sha256sum 40000000 3575C80
[   36.5 s] asset weights: resident - the board's digest of 40000000 equals the manifest's (31.9 s), no upload
[   36.5 s] asset noise: asking the board for sha256sum 54000000 1522950
[   49.4 s] asset noise: resident - the board's digest of 54000000 equals the manifest's (12.9 s), no upload
[   50.8 s] payload: 1404264 bytes at 03000000, verified (1.00 s)
[   52.2 s] run: `boot mem 3000000`, deadman 15 s; waiting up to 900 s for the return line
[  121.5 s] returned: x0 = 0000000000000000 after 69.3 s; reading back now
[  126.5 s] readback wave: 2382400 bytes from 57000000 in 4.97 s, crc32 and board sha256 equal verified
WAV ready: <run folder>\kitten-rosie.wav  (5.1 s after the payload returned)
[  126.9 s] readback trace: 256 bytes from 57E00000 in 0.21 s, crc32 verified
[  127.1 s] readback ticks: 131072 bytes from 57D00000 in 0.28 s, crc32 verified
[  127.7 s] readback bss: 334080 bytes from 01000000 in 0.57 s, crc32 verified
[  128.6 s] lease: released with a ledger row.
```

The first run, straight after a reset, found neither asset in memory and uploaded both: 180.0 s of board time. The second, above, skipped both: 126.5 s. Both waveforms were the same 2,382,400 bytes, sha256 `6e3b3646...`, and so were their WAVs.

> **Note. Checking costs a digest; uploading costs a digest too.** The board

hashed 56 MB in 31.9 s and 22 MB in 12.9 s on that run. A resident asset costs only that check. One that has to go up costs the check and then the upload, whose own verdict is the same digest over again: 37.6 s and 14.9 s here. After a reset every asset pays both.

> **Trap. Never wrap it in an outer `timeout` or pipe it through `tail`**, for

the reasons above - and one more: this tool holds the shared board's lease, and releases it in a `finally`. A process killed from outside never reaches that `finally`, so the ledger says the board is still held.

The exit codes are the same three: `0` the payload returned what was expected, every region verified and the WAVs are written; `1` the run started and failed, and `<name>.json` in the run folder says where; `2` nothing was sent to the board.

Uploading without running

`tools/pi4_upload.py` is the bulk transfer on its own. It drives the board's `net recv`: the console arms a one-shot TCP listener, the image goes over that connection, and the board prints how many bytes landed and the SHA-256 of the memory they landed in.

```
python tools/pi4_upload.py <image> [options]

  --to ram        leave it in memory (the default)
  --to card       and then `save <name> <addr> <len>` onto the boot medium
  --boot          and then enter it: `boot mem <addr>`, or `boot <name>`
                  for --to card
  --reset         and then reset the board
  --addr HEX      where in DRAM it goes. Default: ASK THE BOARD, with `map`
  --port N        the port the board listens on. Default 5001
  --name NAME     the file --to card writes. Default KERNEL8.IMG
  --board-ip A    the board's address. Default: ask it, with `wifi ip`
  --console serial|net      how the commands get there. Default serial
  --com PORT      the serial port. Default COM7
  --console-ip A  the board's address for the network console
  --console-port N  the network console's UDP port. Default 5555
  --cache         send `cache on` first
  --keep-cache    leave the caches on afterwards
  --corrupt [OFF] the bench instrument - see below
```

A real invocation, putting the firmware stub on the boot medium:

```
python tools/pi4_upload.py _work/armstub8.bin --to card --name ARMSTUB8.BIN --console wifi --board-ip <ip>
```

> **Note.** `--cache` is worth knowing about. On this part SHA-256 runs at

about 61 KB/s with the caches off and about 4,041 KB/s with them on, so the digest is otherwise the slow half of the run. The default puts the caches back the way it found them.

The negative control

`--corrupt` flips one bit of one byte on its way out, while still comparing against the digest of the real file. **The run must fail.**

> **Trap.** This is not a debugging aid you will never need. It is the reason

to believe the verifier at all: a verifier nobody has ever seen refuse is a verifier nobody has any reason to believe. This is what it looks like when it does its job, from a recorded board session:

```
  !! --corrupt IS ARMED: bit 0 of byte 812818 is flipped ON THE WIRE, and the
     comparison is still against the digest above. THIS RUN MUST FAIL.
  net recv 5001 400000
  1625636 bytes on the wire in 7.68 s (207 KB/s at this end)
    Received 1625636 bytes from 192.168.1.24 to 00400000 in 7676 ms.
      sha256 7945d151c6041f9f72ac300d5a063e2dc53d0fb4a0fdf862c0758aaaab1780f7
!! THE UPLOAD DID NOT VERIFY, so nothing was saved and nothing was booted.
   the digests DIFFER:
     host  85370e48df79c7fe06956c2c47a27fef049b3e475d44cbba6395422df6ab463f
     board 7945d151c6041f9f72ac300d5a063e2dc53d0fb4a0fdf862c0758aaaab1780f7
   What is in the board's memory at 400000 is NOT this file. Do not boot it.
```

Reading things back

There are two different readback paths and they answer different questions.

Over the console. `load <NAME> <addr>` reads a file off the boot medium into memory, and `crc32 <addr> <len>` or `sha256sum <addr> <len>` hashes it there. This is the one to use after writing a file, because it asks the filesystem rather than the command that did the writing:

```
pmf> load ARMSTUB8.BIN 400000
pmf> crc32 400000 200
```

> **Trap.** `anvil_putfile.py` verifies every file twice, and the second time

is the one that counts. The transfer's own CRC proves the wire was clean and proves nothing whatever about what reached the filesystem. So the file is loaded **back off the medium into a different address** and hashed there. A firmware image wrong by one byte does not fail loudly; it makes a radio that almost works, which is far more expensive to debug than a refusal.

> **Trap.** And one parsing trap worth knowing, because it bit this project:

when you read a CRC out of the console's reply, anchor on the sentence that carries the answer. A looser match once read `EDB88320` - the CRC-32 **polynomial**, which the command helpfully prints two lines below the result - and reported a perfectly good file as corrupt.

Over TFTP. The monitor has `get` and `put`:

```
get <name> [address]
                fetch a file from the TFTP server into memory.
put <name> <address> <length>
                send that many bytes of memory to the server as a
                file. Both numbers hex, the length in bytes. Most
                servers refuse to be written to until they are told to
                allow it, and the server's own refusal is printed here
                word for word when that happens.
```

Over the console, for a machine. `readback <addr> <len>` sends the range as base64 with a byte count and a crc32 on its last line, to the console that asked and to no other (chapter 4 has the format). Every host tool in this repository that takes bytes off the board uses one reader for it, `tools/anvil_readback.py`:

```
from anvil_readback import read_range
data, stats = read_range(console, 0x57400000, 1812000)
```

It asks in requests of up to 256 KiB - fewer if the console socket's receive buffer is smaller - anchors on the header that names this address and length, stops at the closing line rather than waiting for the console to go quiet, and accepts a request only when the decoded length and `zlib.crc32` both equal what the board says it sent. A request that fails is asked for once more - `readback` changes nothing on the board - and a second failure is refused in a sentence that quotes both. `board_run.py`'s `--trace`, `pi4_svcprobe_proof.py` and the compiler repository's single-tensor readback tool all read through it.

Trap. Why it exists, measured. The console readback used to be 4096-byte

`memory` dumps: 1,812,000 bytes took 1,048 s, 1,729 bytes a second, on a cable that carries TFTP at 965 KB/s. The time was not the network. 1.762 s of each 2.365 s chunk was the board's serial port, which every character of the hex went through first; 0.600 s was the host waiting for silence before sending the next command; the 261 datagrams a chunk cost almost nothing. Neither of those survives in `readback`.

Trap. A burst the host's socket cannot hold is dropped without a word.

On 2026-09-16 a 1 MiB request lost 345,600 of its 1,048,576 bytes, twice: the board sends a request at the link's speed, and the host socket's default receive buffer - 64 KB on Windows - overflowed whenever the reader fell a moment behind. The reader now raises the console socket's receive buffer to 8 MiB once, before the first request, and caps each request so its whole reply fits in half of what the operating system actually granted. `tools/anvil_readback_burst_check.py` sends a whole reply before reading a byte of it, on loopback: the old configuration must lose it, and 2,382,400 bytes read the new way must arrive complete.

Note. `board_run.py` does **not** use TFTP. Its trace comes back with

`readback` over the console, and its picture comes back over the console stream. TFTP is a monitor feature you drive yourself when you want a large region off the board as a file.

Trap. `tftpboot` and `tftpput` are deliberately **not** accepted as

aliases. U-Boot takes the address first and the name second; this monitor takes the name first, to match its own `load` and `save`. An alias whose arguments mean something else is worse than no alias - a line pasted out of a U-Boot session would run, and fetch a file called `0x400000`.

How fast a transfer goes, and the rule behind it

A `get` or a `put` is **stop-and-wait**: the board sends one 512-byte block, the server acknowledges it, and only then does the next block go out. Nothing is in flight but one block, so a transfer's speed is decided entirely by how quickly the board notices the acknowledgement. Notice it in a millisecond and 512 bytes cost a millisecond. Notice it in two hundred and the same 512 bytes cost two hundred.

That is not a theoretical range. Until build 146 the transfer loop used the monitor's **fair** receive pump, which polls every interface without waiting and then blocks its whole 200 ms slice on one of them, taken in turn. Fairness is right for a listener with no expectation - an idle prompt, a passive TCP open - because no interface is likelier than another to hold the next frame. It is wrong for a loop that has just sent something and knows exactly which wire the reply is coming back over.

A Raspberry Pi 4 running this monitor usually holds **two** addresses at once - one on the cable and one on the radio - and both interfaces are then usable, so that blocking branch is live. Follow one block through it:

```
turn 1   the poll takes the acknowledgement off the cable, and the
         rotation moves on to the radio. The next block goes out.
turn 2   the poll runs microseconds later; the server has not answered
```

```
         yet, so nothing is anywhere - and the slice is spent BLOCKED
         ON THE RADIO. The acknowledgement lands on the cable a
         millisecond in and sits there for the other 199.
turn 3   the poll finds it, and it all happens again.
```

Trap. The rotation never rescues it. Taking a frame off the cable sets

the cursor one past the cable, and the idle slice that follows walks it round and back, so the wait lands on the radio after **every** block rather than half of them.

Measured on a board, both interfaces up, before and after:

```
                          before            after
98,304 bytes, 193 blocks  36.35 s           0.106 s
                          2.70 KB/s         930 KB/s
                          188 ms a block    0.55 ms a block

1,812,000 bytes,          ~666 s, and       1.877 s
3,540 blocks              REFUSED at the    965 KB/s
                          ten-minute
                          ceiling

retransmissions           0                 0
duplicates ignored        0                 0
gaps                      0                 0
datagrams for us          194               194
```

Trap. Read those counters again, because they are the interesting part.

Nothing was ever being lost. Every acknowledgement arrived and was taken, on both builds - the counters are identical and only the clock moved. A transfer that is slow for this reason looks perfectly healthy in every tally the monitor prints, which is exactly why it went unnoticed: there is no error to find, and the only symptom is the wall clock.

The rule, from build 146: **a stop-and-wait transfer's blocking wait belongs on the wire its answers arrive on.** The loop still polls every usable interface before it waits anywhere - ARP requests, pings, the DHCP service the board runs on the cable and console keystrokes on the radio are all still answered during a transfer, including the keystroke that cancels one - but the slice itself is spent on the interface the transfer was routed onto, and on no other. A block then costs one host round trip, a millisecond or two, instead of a whole slice.

Note. The slice is still 200 ms and that is deliberate. A receive

deadline is a **ceiling, not a cost**: the call returns the instant a frame arrives, so once the wait is on the right wire the 200 only decides how long a *dead* transfer sits between its own retransmissions. Shrinking it would buy nothing and would risk a slice shorter than the time a slow link needs to assemble one frame, which turns a working transfer into a stream of runts.

Trap. This is the rule to reach for whenever a loop sends and then

waits, and it applies to more than TFTP: the address resolution a transfer does first has the same shape, and takes the same treatment. Any future request-and-reply command on this board should ask, before it waits, *which wire is the answer coming back on* - and wait there.

Screenshots on their own

```
python tools/pmfshot.py COM7 anvil.png
```

That drives the monitor's live-screen command over the serial line, decodes the run-length stream it sends back, and writes a PNG plus the raw stream beside it.

> **Note.** It run-length encodes first for a reason worth appreciating. The

framebuffer is 1824 by 984 by 4 bytes - about 7.18 MB, or 14 MB as hex, or roughly twenty minutes of wire time at 115200. A console screen is overwhelmingly one background colour in long horizontal runs, and collapses to a few tens of kilobytes.

What none of this proves

The gates behind the capture path execute emitted code over a modelled panel. They cannot show that the bytes in the scanout region are the bytes the glass is lit with, that a 12 MiB capture region is really free on silicon, that a 4 MB copy fits in the window between a payload returning and the console repainting, or that a picture survives the trip over UDP. Those are board facts, and the repository records them as owed rather than assumed.

Using all four cores

A Pi 4 has four processors. Anvil will run one of your procedures on all of them, inside a bound, with results that are bit-for-bit what one core would have produced.

What the service does, in one paragraph

You hand Anvil one procedure address and N argument blocks. Anvil calls that procedure once per block - on cores 0, 1, 2 and 3, with the core you were already on taking a block too - waits for all of them inside a bounded deadline, and returns. That is the whole service. It is not a scheduler, there are no threads, nothing is queued, and nothing runs in the background afterwards.

The three slots

Group 15 of the service table, new at ABI 1.3:

```
SvcParallelCores()                                slot 184
SvcParallelRun(entry, argsBase, argsStride, count)   slot 185
SvcParallelStatus(index)                          slot 186
```

`SvcParallelCores()` tells you how many partitions this monitor will run at once. It returns 4 or 1, and **it is never an error**. One is a real answer, not a failure: it means "this Anvil will run your work on the core you are already on". A monitor older than ABI 1.3 gives you the same answer by not having the slot at all. So your payload has exactly one fallback path - the single-core code you already have - and not two.

`SvcParallelRun` starts the run. `entry` is the procedure address, `count` is 1 to 4, and the call Anvil makes for partition `i` is:

```
entry(argsBase + i * argsStride)
```

Nothing else is defined. One pointer goes in. Your procedure's return value is kept and readable afterwards through `SvcParallelStatus(i)`; zero conventionally means it is content. It must return.

> **Trap.** Your entry gets a pointer and **nothing else**. No core index, no

partition number, no base, no count. If your worker needs to know which slice it owns, put that in the block - which is what every real user of this service does. A four-field block of `{ shared args pointer, first, last, status }` at a stride of 32 bytes is the shape the desk gate and the first real user both settled on.

Ask for the count at the right moment

Call `SvcParallelCores()` **after** your payload has enabled its own MMU and caches, not before.

The count is not a constant for the life of your payload. The service answers 4 only when the calling context is right, and one of the conditions it checks is that your translation regime and caches are on. Ask too early and you get 1 - correctly, and silently, because 1 is a legitimate answer - and your payload quietly runs everything on one core while believing it asked.

This is not hypothetical. It happened to the first real user of the service: the count read 1 because the question was asked before the payload's own cache bring-up, and the run took the single-core path for an entire board session before anybody noticed the timings had not moved.

Partitions are yours, and that is the point

Anvil does not split your work. You do.

You split your output into ranges, one per partition, and those ranges must be **disjoint**. No output element may be written by two cores. This is the whole design, and it is what buys you the property that matters:

> **Note.** A kernel that is bit-identical to its scalar reference on one core

stays bit-identical on four, **by construction rather than by measurement**. Because the partitions are disjoint output ranges, no result is written twice and no result's accumulation order changes. You are not hoping the answer comes out the same. There is no path by which it can differ.

That claim held the first time it was tested on real work. Two reference waveforms produced by a four-core run came back byte-for-byte identical to the ones the single-core build had produced, while the run itself went from 265.5 seconds to 107.3 (recorded board session, 2026-09-16, build 142).

Two mechanisms hold the property up, and both are worth knowing because both can bite you.

The floating-point control register must agree

Rounding mode and flush-to-zero live in a register that is **per processor**. One core rounding differently from the others is a silently different answer, which is exactly the failure this whole design exists to make impossible.

So every secondary publishes its floating-point control register when it reports ready and at every job acknowledgement, and the primary refuses the job unless all four agree with its own. You do not have to do anything about this; you just need to know that a refusal citing it is real and is not a false alarm.

Do not flush caches at the join, and this is not a performance note

> **Trap. Cache maintenance at the join would be WRONG, not merely slow.**

Splitting a computation by output range puts two cores' outputs in the same 64-byte cache line at each boundary. On the memory type these buffers live in, the hardware's coherency protocol resolves that correctly at byte granularity. A clean-and-invalidate over the output at the join takes a line that two cores have each partly written and writes one core's copy of it over the other's - destroying exactly the results you just computed. Anvil's own join loop carries this rule as a comment and does no maintenance there. Do not add any in your payload either.

What your worker procedure may not do

Cores 1, 2 and 3 are running your code in a context that has none of Anvil around it. From those cores your procedure may not:

```
- call a service slot
- print anything
- touch a device
- pet the deadman
- unmask interrupts
- reach Anvil's state in any way
```

Pure computation over memory you own. That is the entire permitted surface.

It works because on this backend a procedure's parameters and locals live in its own frame, so four invocations on four stacks do not collide. What they must not do is share anything mutable that Anvil owns.

> **Trap.** Each secondary gets a 4 KiB page of stack, and the lowest 256

bytes of it are a poison guard, so your **usable depth is 3840 bytes**. Run off the bottom and the guard is overwritten, the run is refused, and the refusal says plainly that this job's outputs cannot be trusted. A deep recursion or a large local array in a worker is not a small mistake here.

The five-second bound

`SvcParallelRun` is bounded at five seconds. The bound is armed before anything is dispatched, and it is measured off the board's wall counter.

Five is chosen against the 15-second deadman, with your payload's own pet immediately before the call. A kernel whose partition cannot finish inside five seconds is one you split into more calls. **The bound is never widened toward the deadman**, because a bound that approaches the deadman stops being a separate safety net.

The bound exists for one specific failure the deadman cannot see: a wedged secondary beside a healthy primary that keeps petting. Nothing else covers that case. The board looks alive, the deadman is fed, and one core is simply gone.

> **Note.** What the bound does NOT cover, said plainly: **your own

partition**. Partition 0 runs on the primary as an ordinary call into your code, and nothing can interrupt it. A worker that never returns on the primary is your payload's own infinite loop, and the deadman is its answer exactly as it would be on one core. The bound is armed at entry so its five seconds are measured from before that call rather than after, which keeps the arithmetic honest - but it cannot make a non-returning primary return.

When a core wedges

On expiry the service latches a fault, names the core, and returns `SVC_ETIMEOUT` (-104). Your payload should print the reason by name rather than a bare code; `SvcErrText` and `SvcErrDetail` give you both halves of the sentence, and the detail for this one reads:

```
a secondary core did not finish its partition inside the five-second bound; a payload entry that faults on a secondary parks it silently, because a secondary has no exception vectors of its own
```

That second clause is the important part. A secondary has no exception vectors of its own. If your worker faults on core 2, core 2 does not report an error - it takes an exception to a vector base it does not own, and it is gone. There is nothing to catch. The timeout is how you find out.

> **Trap.** A wedged core **latches the service failed for the rest of the

boot**. Every later call refuses with `SVC_ESTATE` (-108), and `SvcParallelCores()` starts answering 1 so that a well-written payload falls back to one core instead of looping on refusals. There is no re-release and no warm rejoin, because a core that stopped answering cannot be proven safe to re-use.** Recovery is a reset.** This is accepted policy, not an unfinished corner.

The full list of refusals

Every one of these is a sentence, and every one names what to check:

```
SVC_EARG     (-102)  count must be 1 to 4, entry must be a non-zero
                     word-aligned address, and argsStride must not be
                     negative
SVC_EPERM    (-106)  a parallel run may only be started from the core
                     the payload was entered on, at EL3, with interrupts
                     masked
SVC_EBUSY    (-103)  the secondary cores could not be started because
                     something else already holds them
SVC_ESTATE   (-108)  a secondary core has an unfinished job outstanding
                     or was permanently parked, and neither is
                     recoverable without a reset
SVC_ETIMEOUT (-104)  a secondary core did not finish its partition
                     inside the five-second bound
SVC_EPERM    (-106)  a secondary core is not rounding the way the
                     primary is, and bit-identical results need every
                     core to share one floating-point control register
SVC_EPERM    (-106)  a partition ran off the bottom of its secondary
                     stack and overwrote the guard below it, so this
                     job's outputs cannot be trusted
SVC_EIO      (-105)  the payload's own partition procedure reported
                     failure; the value it returned is in
                     SvcParallelStatus and no Anvil state is at fault
SVC_ENOSYS   (-100)  this board does not have the parallel capability
```

That last distinction is worth a second of your time. `SVC_EIO` with that detail means **your** worker said no. Nothing in Anvil failed. Go and read `SvcParallelStatus(i)` for the value your own code returned.

coretest: proving the cores before you trust them

`coretest` is a bounded acceptance instrument, not a scheduler and not a payload. It starts cores 1 to 3 and runs sixteen verified arithmetic rounds on each, so you can establish that the machinery works before you blame your own kernel for a silence.

```
coretest          report ownership, run count, nonces, per-core state
coretest status   the same thing, spelled out
```

```
coretest run       start cores 1-3 if needed, run 16 verified rounds
coretest stop      publish stop requests, wait for permanent park
```

Extra arguments and unknown subcommands are refused. There is no automatic start and no payload form.

What you will see. These lines are from a recorded board session on 2026-09-16, build 142 - the first time this machinery ran on a board. Cold, before anything else:

```
pmf> coretest
coretest started=0 stopped=0 runs=0 nonce=0 error=0 core=0
  leases: secondary=0 gic=0 watchdog=0
```

Then the run. `coretest run` always prints the status block after its verdict, so you get the verdict, the counters, the leases, and one row per core:

```
pmf> coretest run
coretest PASS: 16 nonce rounds, three resident EL3 workers.
coretest started=1 stopped=0 runs=1 nonce=16 error=0 core=0
  leases: secondary=1 gic=1 watchdog=1
  core=1 ready=2 request=16 ack=16 result=25167481 status=1
  core=2 ready=2 request=16 ack=16 result=25236842 status=1
  core=3 ready=2 request=16 ack=16 result=25306203 status=1
```

`request` and `ack` matching at 16 is the proof: every nonce was asked for and every one was acknowledged, with no refusal and no timeout.

A failure says so and tells you where to look:

```
coretest REFUSED/FAILED; see error and core below.
```

and in a latched state, the status block adds:

```
Failure latched. No re-release or warm rejoin. Try stop only after jobs finish.
If stop refuses, retain diagnostics; controlled restart required for recovery.
```

The lifecycle limit you will hit, and it is still open

This is the one thing about the four-core service that will surprise you, and it is worth reading twice.

Once anything has started the secondary cores, this monitor will not enter another payload until you reset the board. Not another four-core payload - not any payload at all, including a plain single-core one.

You will see exactly this:

```
pmf> run
!! resident secondary cores own this monitor until restart, so no
   payload was entered. Restart Anvil before run, go or boot mem.
```

`run`, `go` and `boot mem` all refuse the same way.

The reason is sound. A released secondary is executing code and reading rows inside the resident monitor image. Handing the machine to a payload would disable the translation regime those addresses were handed out under, and every address the workers were given would stop meaning what it meant. Stopped workers are permanently parked **in that same resident code**, so `coretest stop` does not help - it parks them and keeps ownership by design. Only a restart clears it.

> **Trap.** The trap is that your payload springs this itself. `SvcParallelRun`

starts the cores lazily, on first use. You do not type a command to enter this state; your payload's first parallel call enters it for you. So the shape of a four-core session today is: reset, upload, run one payload, read your results back, reset again. Budget for the re-upload of anything large.

> **Note.** To be exact about what the limit is and is not: a single payload

may call `SvcParallelRun` as many times as it likes. The first real user made 252 dispatches in one payload with no refusals and no timeouts. The limit is one **payload** per boot once the secondaries are up, not one call.

Lifting this is open work as of 2026-09-16 and has only just started. It is not built, and nothing in the current build 142 changes it. When it lands, `docs/MULTICORE_STATUS.md` is where it will be recorded first.

One more thing, if your four-core run is long

A long, silent payload can outlast the host tool's patience on the console peer. During one run the host tool reported that the payload never returned - and it had in fact completed, in 265.5 seconds. The result was recovered from the board's own trace and from memory.

So: if a long run comes back as "never returned", do not assume the board is wedged and do not reset it reflexively. Check the board first. Chapter 6 covers what the host tool records and how to read it back.

Updating the monitor in place

The running monitor receives its own replacement, writes it to the file it boots from, and resets into it. Two moments in that sequence are the ones that can cost you the board.

The idea

No second machine, no pulling the medium, no external programmer. The monitor that is running takes delivery of the next monitor, writes it over `KERNEL8.IMG`, and resets.

That works because of one property: **between the save and the reset, the old monitor is still running and still has the console.** If anything is wrong, you are still talking to a working board. Everything in this chapter is built around protecting that window.

The sequence

```
1. confirm you have a prompt
2. ask the board where to stage        map
3. transfer the image                  receive <addr> <len> <crc>
4. write it to the boot file           save KERNEL8.IMG <addr> <len>
5. read the directory back             ls
6. only then                           reset
```

One command does all six:

```
python tools/anvil_update.py build/pi4/anvil.img
```

It takes **one optional positional** - the image, defaulting to `build/pi4/anvil.img` - and **one option, `--fast`**. Anything else is refused before a byte is sent:

```
!! not an option this tool knows: --verbose
```

> **Note.** That strictness was earned. The fast flag used to be matched by

scanning the whole argument list, so a typo like `--fst` was silently dropped and the write went ahead at the slow rate while the invocation looked like the fast one. Now anything starting with a hyphen that is not `--fast` stops the tool before it sends anything.

Step 2, and why it is step 2

The staging address is asked for, not assumed - and it is asked **before** the transfer rather than after:

```
staging at 00500000, which is where this board said to put it
```

Note. A monitor that will not say where to stage is a run that must not

start. The alternative is sending two megabytes and then discovering there is nowhere to put them.

The baud rate is left alone

By default the transfer runs at whatever rate the board booted at. `--fast` opts into 1.5 Mbaud, which is roughly a minute faster on a megabyte-and-a-half image.

Trap. The default is the slow one deliberately. Two saves in a row once

died at "SAVE DID NOT COMPLETE" with the fast rate in the middle of them. The default is the rate that finishes.

What you will see

From a recorded board session, redacted only for the network name:

```
RaspberryPi4/Monitor/anvil.img: 1625636 bytes
115200 - the baud is not touched (pass --fast to change it)
received 1625636 bytes in 153.8 s, CRC verified on the board

save KERNEL8.IMG 400000 18CE24
The file KERNEL8.IMG on the boot medium is 1809100 bytes long at the moment.
It is being resized from 1809100 bytes to 1625636 bytes to match what
you asked to write.
Writing 1625636 bytes out of memory, starting at 00400000. This
takes a moment and must not be interrupted.
written.
  The file on the boot medium now holds what was in memory, and the
  block writer has been disarmed again so nothing else in this monitor
  can reach the medium. Type reset to boot into it.

resetting into the new build ...
reset
Resetting the board now. The firmware reloads this monitor from the boot
medium and it starts again in a few seconds. Nothing in memory survives.
```

and then the new monitor's banner, and a prompt.

Note. The file is **resized to the image**, not padded into a fixed

slot. The length is passed to `save`, so the boot file grows or shrinks to the exact image size and the filesystem does the resize in place.

Stop one: the save did not complete

```
SAVE DID NOT COMPLETE - not resetting.
The file may be part old and part new; write it again
before the board is reset, or it boots half an image.
```

Trap. Do not reset to try again. Write it again first. The old

monitor is still running and still has the console; that is the whole asset here and a reset spends it. A boot file that is part of one image and part of another does not fail in any recognisable way.

Stop two: the directory disagrees

After the save, the tool asks the **filesystem** what is there. Not the command that did the writing.

If the file is not in the listing:

```
!! KERNEL8.IMG is not in the directory listing after the save, so
   NOT resetting. The board is still running the old image
   and the stick still has whatever it had. Type ls on the
   console to see what is actually there.
```

If it is there and the length is wrong:

```
!! KERNEL8.IMG is 1809100 bytes and the image sent was 1625636 bytes, so the file on
   the stick is NOT the image that was just sent. NOT
   resetting - write it again before the board is reset, or
   it boots part of one image and part of another.
```

Note. Why the readback exists at all: `save` saying `written.` is that

command's own account of itself. The one question that actually matters afterwards - **is the file the firmware will load now the length of the image we sent** - is answered by the directory and by nothing else. It costs one command and one line of output, and the alternative is resetting into a file nobody has looked at.

It was earned on a specific day. The tool crashed printing a non-ASCII byte out of the board's reply, **after** the save had landed and **before** the reset - and the operator could not tell which, because there was nothing on the screen but a traceback.

Prove it harder, when it matters

For a monitor you care about, the directory length is the minimum. The stronger check reads the file **back off the medium into a different address** and hashes it there. This is a recorded board session from 2026-09-16:

```
pmf> save KERNEL8.IMG 500000 2CB860
written.
pmf> ls
  KERNEL8.IMG   2930784 bytes
```

```
pmf> load KERNEL8.IMG 01000000
pmf> sha256sum 01000000 2CB860
The sha256sum is ec9abb8418344b38b3638b6d6980e0e7c0d81ba52eb967bbbb6b47c0a25ce439
```

> **Note.** Three things make that strong. It reads the **file**, not the

staged copy. It reads it to a **different address**, so a stale buffer cannot answer for it. And the digest is computed by the board, which is something other than the tool that did the writing. The hash matched the host's before anything was reset.

Then, and only then:

```
pmf> reset
pmf> version
build 142 date 20260915 time 220310
```

> **Note.** That `version` check is the whole reason the build number moves

with every build. You asked for build 142; the board says 142. If it says something else, you have just learned something important while you still have a console.

Recovery, in the order to reach for it

1. Before the reset - just write it again. Between `save` and `reset` the running monitor is still the old one. This is the cheapest recovery and it is why the two stops do not reset.

2. After a bad reset - the medium. `KERNEL8.IMG` is an ordinary file in the FAT boot partition. A monitor that will not come up is recovered by putting the medium in another machine and copying the previous image back. **Which means: keep the outgoing image.** Before flashing, copy the board's current `KERNEL8.IMG` off, or keep the previously flashed artifact and its hash, so there is a known-good file to restore.

3. Better still, keep a spare on the medium itself under its own name, so the recovery is three commands from the prompt rather than a trip with a screwdriver:

```
pmf> load SCALEOK.IMG 800000
pmf> save KERNEL8.IMG 800000 20C940
pmf> reset
```

> **Trap.** Stage at `800000`, not at the default. `load` with no address goes

to the staging address, and a large image there can land on the tail of the running monitor. And `save` takes a **length**, in hex, not an end address.

The guard that makes a boot loop impossible

This one is automatic and it protects payloads rather than monitors, but it belongs here because it is the reason a bad automatic boot does not take the board away from you.

Anvil counts failures in `boot.fails` against `boot.maxfails`, default 3. **The counter is written to the medium BEFORE each attempt** - a counter written after a good boot is never written in the case it exists to catch. A payload that hangs lets the deadman fire; the failure is already recorded; and after the limit the prompt comes back naming the guard instead of booting the thing that hung:

```
!! NOT booting APP.PMF automatically, and this is the reboot-loop
   guard rather than a fault with the file. It has been attempted 3
   times in a row without anything ever confirming that the payload
   reached a working state, and boot.maxfails is 3.
   A payload that hangs or resets gets re-entered on every start, and
   there is then never a prompt long enough to type the fix at. This is
   that prompt.
   Type boot ok to clear the counter once the payload is known good, or
   boot clear to stop booting this file at all. boot status shows all
   four settings.
```

> **Note.** "There is then never a prompt long enough to type the fix at. This

is that prompt." That sentence is the design. A payload marks itself good by getting the counter back to zero - the operator types `boot ok`, or the payload calls the service that does it when it reaches its own steady state.

> **Trap.** Do not call `boot ok` at the start of your payload. Saying it on

every boot regardless turns the guard off, which is a decision rather than a habit worth falling into. Say it when the payload is actually working.

Why a payload cannot write the two counters

The service a payload uses to persist a setting refuses `boot.fails` and `boot.maxfails`, and so does the one that deletes a setting: both answer `SVC_EPERM` and change nothing. A payload that could write those two could switch the guard off silently, so clearing the counter is a separate service of its own - the one `boot ok` also calls - and it is the only sanctioned way to do it. Everything else in the store, `boot.file` included, a payload may set freely; that is how a payload updates firmware.

> **Note.** Setting names are **case-insensitive** everywhere - the store folds

every name to lower case before it writes one or looks one up, so `BOOT.FAILS` and `boot.fails` are one setting - and the refusal folds exactly the same way, through the store's own comparison rather than a second copy of the rule. Spelling either name in upper or mixed case gets the same `SVC_EPERM`, because it would otherwise reach the same stored setting by a name the guard did not recognise. Names that merely resemble the guarded ones - `boot.file`, `boot.fail`, `boot.maxfail` - are not guarded and stay writable.

What this chapter is not

The Raspberry Pi 3 uses a **different** update mechanism: the firmware boots the complete monitor from `kernel8.img` after running a Pi 3-specific stub. Its staging, backup and direct-file replacement procedure is in `docs/PI3_SELF_UPDATE.md` and

none of it applies here.

> **Trap.** And the sharpest edge of all, which is not about updating but is

about this file: **never enter a monitor image as a payload.** `boot mem` places an image at the address it was built for, and for a monitor that is `$00200000` - on top of the running monitor. Do that while a transfer is in flight and the board goes down with no console and no serial, and somebody has to pull the medium. It has been avoided once by the margin of a single command.

Driver modules

Not every board has the same hardware. A driver that is not on the path that loads modules does not have to be compiled into the monitor.

What a module is meant to be

A `.MOD` file on the boot medium: a driver, verified against its own digest, placed in a reserved area of memory, relocated, matched to a device the board declares, probed without side effects, and asked to publish the service it implements.

The argument for it is simple. The Pi 4 image today contains drivers for hardware that a given board may not have - a particular touch controller, a particular panel, a particular radio. Every one of those is weight that every user carries. A module is that driver as a separate file.

Read this before you plan anything around it

The published architecture is explicit, and it is the thing to plan against:

```
The engine is load-once placement infrastructure, not a working driver
loader. READY means verified, relocated, zeroed and cache-synchronised
only. The files are not wired into either board composition and no
board currently supplies a module arena.
```

and the list of what is **not** implemented in the published export:

```
- board arena reservation or module-file ingestion
- MODULES.TXT parsing
- hardware discovery and compatible-id matching
- module global-initializer, probe, init, quiesce, or service binding
- mod list, mod info, mod load, or mod reload
- stable service trampolines, in-flight accounting, transactional reload
```

> **Trap.** The working tree is ahead of that text, and this is the one place

in this book where two true statements sit next to each other. The console command set described below exists in the source, and a real board session has loaded a real module. The **published** boundary is the one quoted above. If you are reading this against a checkout, check what `mod` actually does on your own board before designing around it - and treat `docs/MODULE_FORMAT.md` and `docs/MODULE_PIPELINE.md` as the authorities, because they move with the code.

The container

PMFMOD v1 is a little-endian AArch64 container. A fixed **160-byte header**, then the image bytes, zero padding to an eight-byte boundary, fixed 16-byte relocation records, and one to eight fixed 64-byte compatible-id records.

The magic is the ASCII bytes `PMFMOD` followed by two zeros, at offset zero.

```
+0    magic - 'PMFMOD' and two NUL bytes
+8    version              (1)
+12   header length        (160)
+16   ABI major
+20   ABI minor
+24   seam count
+28   flags                (no v1 flag promises reload)
+32   image bytes
+40   BSS offset
+48   BSS bytes
+56   init offset
+64   relocation count
+68   relocation offset
+72   alignment
+76   architecture         (1 = AArch64)
+80   seams                (4 cells; unused cells are all-ones)
+96   sha256               (32 bytes)
+128  match count
+132  match offset
+136  probe offset
+144  quiesce offset
+152  reserved
```

> **Note.** The digest covers the **complete container except header bytes 96

to 127**, which hold the digest itself. That detects accidental or malicious damage in transit. It does** not** authenticate a publisher, establish trust, or sandbox native code, and the format document says so in its own opening paragraph rather than letting anyone assume otherwise.

Compatible IDs are 1 to 63 printable ASCII bytes, then a NUL and an all-zero tail. Space is allowed; **slash and backslash are forbidden**, so an ID can never be read as a path. IDs within one container must be unique.

> **Note.** The numbers above live in `Anvil/Hal/module_format.pbi` and the

format document deliberately does not repeat them. Compiler writers and loaders consume a byte-identical copy of that one file rather than maintaining their own tables. Two copies of a numeric contract is one copy too many.

What is validated, and when

The engine validates the **complete** container - exact layout, ABI, architecture, flags, reserved fields, entries, matches, relocation sites, targets, opcodes and ranges, and the digest - **before it changes the arena**. Call targets must be aligned instructions inside the emitted image. Data relocations may target image or BSS but not the gap between them. Relocation destination spans may not overlap.

> **Trap.** A digest is never a substitute for structural validation. It

proves the bytes are the ones somebody produced; it does not prove that what they produced is safe to place. Both checks exist and neither replaces the other.

The board's half of the contract

A board that wants modules supplies six functions:

```
HwModArenaBase()            4 KiB-aligned first byte, or 0 for "none here"
HwModArenaBytes()           byte length of a reserved, guarded range
HwModArenaRegion()          which monitor-region index IS the arena, or -1
HwModStageAddr()            where one container is read and judged
HwModStageBytes()           the largest container this board accepts
HwModCodeSync(base, bytes)  clean D-cache, invalidate I-cache
```

> **Note.** The arena is **checked, not trusted**. The engine walks the

board's own reserved-region list and payload windows and refuses an arena that overlaps one, skipping only the index the board names as the arena's own. That check exists because an earlier design reserved a fixed range which is now the DSI framebuffer - and a paragraph saying "do not use this range any more" is not a mechanism. A board that offers no arena answers 0 and gets a **different sentence** from a board whose arena is misaligned.

And a device table, so a driver can be matched to something real:

```
HwDevCount()        how many devices the board will name
HwDevIdAddr(i)      a NUL-terminated ASCII compatible id
HwDevSeam(i)        the capability group it belongs to
HwDevToken(i)       the handle a driver needs - a register base, usually
HwDevSay(i)         prints the device's own name, in the board's words
HwSeamCore(id)      does the core reach this seam with no module at all
HwSeamPossible(id)  is the hardware here, whether or not anything reaches it
```

Building one

```
PureMetalForge.exe --compile RaspberryPi4/Modules/thermal_avs.pi4 -t pi4 --module -o THERMAL.MOD
```

Then the file goes on the boot medium, and a line naming it goes in `MODULES.TXT`.

The manifest

`MODULES.TXT` lives in the **root of the boot medium**, beside `SETTINGS.TXT`. Not in a subdirectory - the filesystem reader descends into none, and one place for the monitor's files is easier to explain than two.

The file in this repository is its own documentation:

```
# Anvil driver modules, loaded in this order at boot.
#
# COPY THIS FILE TO THE ROOT OF THE BOOT MEDIUM, beside SETTINGS.TXT, and
# copy each .MOD it names there too.
```

```
#
# A line is a FILE NAME and an optional SEAM NAME. The seam name can only
# ever NARROW a module to part of what its own header declares; naming a
# seam the header does not carry is refused, and nothing can widen a
# module past the list its digest covers. `mod seams` prints the names.
#
# A line that cannot be acted on STOPS THE WALK - the modules after it are
# not loaded, and the console says which line stopped it. Acting on half a
# manifest is the confident wrong answer this monitor refuses to give.
#
# `#` and `;` start a comment. Blank lines are ignored.
THERMAL.MOD
```

> **Note.** "A line that cannot be acted on stops the walk." That is worth

repeating because it is the opposite of what most loaders do. Loading the modules after a failure would leave you with a system that is partly what the manifest asked for, and no way to tell which part.

The commands

```
mod [list]              what is loaded, where, and in what state
mod info <file>         the header, the digest, the seams, the device
mod devices             what this board offers a driver to bind to
mod seams               the published seams and who answers each
mod load <file> [seam]  read, verify, place and activate one module
mod unload <file>       refused while a consumer still holds a seam
mod detach <seam>       ask those consumers to let go first
```

> **Note.** `unload` being **refused** while anything still uses the service

is the interesting half. `detach` asks the users to let go first, and they fall back to whatever they used before the module was loaded. An unloaded module's memory stays reserved until a reset - it does not come back to the pool, and the state table calls that out by name.

The lifecycle states

Each state says exactly what it does **not** mean, which is how you read this table:

```
READY     verified, placed, zeroed, relocated, cache-synchronised
          -- but nothing has looked at hardware
MATCHED   a device the board declares answers one of the container's
          compatible ids
          -- but the device is not known to be responding
PROBED    the driver looked at that device and accepted it, changing
          nothing
          -- but it has acquired nothing
ACTIVE    init returned 0 and published at least one service
          -- but nothing is necessarily using it
```

```
FAILED    a step refused; everything it had published was withdrawn
          first
          -- the bytes did not go away
STRANDED  unloaded cleanly; the arena it used is not reusable until a
          reset
          -- the memory is not free
```

The example driver

`RaspberryPi4/Modules/thermal_avs.pi4` is the part's own temperature monitor. It matches a device id, fills the thermal seam's read function, and **contains no address of its own** - the board hands it the block's base as an argument.

With it loaded, `fan` reports the temperature as coming from a sensor register the board reads directly. Without it, the same command reports the firmware property interface and everything still works.

> **Note.** That property is what made the conversion safe to do with no board

attached: the thing being replaced kept working. A driver conversion where the old path stops existing is a conversion you cannot test incrementally.

The ABI boundary a module may not cross

```
Anvil/Hal/seams.pbi is the published contract: the seam ids, the
function-index vocabulary per seam, the two slot numbers, and nothing
else. A module includes that file and no other Anvil file. If a module
ever needs a second one, the contract has leaked.
```

What is proven

Two gates. One compiles the real driver as a module, mutates copies of the container into the negative cases, compiles the real core sources against a modelled board and medium, and runs the emitted instructions in this project's own interpreter. The other is the container and relocator gate, covering all eight relocation kinds at two placements.

> **Trap. Neither is a hardware claim.** There is no MMU, no cache, no DMA

and no firmware in either; the device register is interpreter memory and cache maintenance is a stub. Silicon acceptance is a separate claim and is recorded separately.

What the modules are for, eventually

The planned first conversions are the fan controller, the touch controller, the panel power and backlight controller, and the radio. Until the loader lands, they remain compiled into the image where included.

Proving things

A listing is not proof. A gate that examined nothing has not passed. And a host test is not a hardware claim, however green it is.

The shape of the argument

This project's tests do not read code and conclude things about it. They **compile** the real source, **execute** the emitted instructions, and report how many instructions ran.

That last number is not trivia. It is the anti-vacuity number: a gate that reports two hundred assertions and fourteen million instructions did something, and a gate that reports a pass and nothing else might have done nothing at all.

The house rule

```
Trust the disassembled hex, not the listing.
```

It was earned by the worst bug in this project's history: a compiler listing showed stack frame setup that the actual binary did not contain. Everybody reading the listing agreed the code was correct. It was not.

> **Note.** There is a subtler second half. Some transformations happen inside

the assembler, **after** the listing is written - so a correct build can produce a listing that legitimately disagrees with the binary. It is not wrong; it is **earlier**. Somebody verifying a fix by searching the listing concludes the fix did not work. The rule points the same way in both cases: go to what executes.

Inside this repository the rule shows up as one line at the top of a check script:

```
#  A CONSTANT IS NOT PROOF. Neither is a promise in a comment.
```

and as one line in the README:

```
Host builds and emitted-code tests are valuable gates, but they are not
a substitute for hardware proof.
```

The interpreter

`tools/a64/a64_interp.py` is a standalone AArch64 instruction interpreter. It models the integer and system subset this backend emits, and it is what every emitted gate runs the compiled image in.

> **Note.** It **predates the assembler**, and that ordering is the whole

point. The house rule for a new processor is: write and self-test the interpreter against hand-assembled bytes **before** the assembler exists. Then cross-check the assembler against the interpreter, and the code generator against that. When the interpreter and the assembler later disagree, **the assembler is the suspect** - because the interpreter could not have inherited

its assumptions. An interpreter written afterwards, by the same person, on the same afternoon, would have agreed with the assembler for exactly the same wrong reason.

That rule has caught a real bug every single time it has been applied.

> **Trap.** The interpreter holds the one copy of the alignment rule, and it

is a rule about real silicon. With the MMU off - which is how a payload runs unless it says otherwise - a wide data access whose address is not a multiple of its size is **fatal on the part, and silent**. The model serviced such accesses happily until that was found. Any gate that installs its own memory closures has to call the alignment guard, or it is modelling a machine that is kinder than the one you have.

The interpreter is handed to gates by path, through `PMF_A64_INTERP` or `--interp`, and loaded by path rather than imported - so a private copy on somebody's machine cannot substitute itself silently. There is a separate gate for exactly that: it runs in an isolated interpreter process, executes one floating-point instruction, checks the arithmetic bit-exactly, and asserts that the helper it resolved is the repository-local one.

Running a gate

There is no runner. Gates are invoked one at a time:

```
python3 tools/module_engine_check.py
python3 tools/a64/interpreter_dependency_check.py
python tools/screen_shot_emitted_check.py --compiler <compiler> --interp tools/a64/a64_interp.py
python tools/board_run_parse_check.py
python tools/board_run_refusal_check.py
python tools/board_run_outcome_check.py
```

> **Note.** That is a real limitation and it is stated rather than papered

over: there is no `run_all_gates`, no make target, no shell wrapper. Roughly 180 check scripts live in `tools/` and about 50 more in `tools/a64/`, and a lane runs the subset that its change could possibly affect.

What you will see. This is a real desk run:

```
$ python tools/board_run_parse_check.py
board_run_parse_check: PASS - 53 assertions over 14 recorded board replies, including four streams that must
be refused
  the live `p` stream and the kept `shot` stream decode to the same
  pixels, which is what lets one host decoder serve both.
  Recorded replies, not a board. This proves the decoder, not a panel.
```

> **Note.** Read the last line of that output. The gate finishes by saying

what it did **not** prove. That is the house style and it is what makes a green result worth anything.

A gate can also prove that a tool FAILS correctly, which is harder and more useful. This one drives the real host console over a socket that answers like a monitor, and asserts that a refused boot is reported in the board's own words inside the round trip rather than after the ten-minute default, that a run killed mid-step has already written a record naming that step, and that a console with nobody behind it is a sentence and not a traceback:

```
$ python tools/board_run_refusal_check.py
board_run_refusal_check: PASS - 64 causal assertions; no board
  a refused boot ends in the round trip, not in --timeout;
  a killed run leaves a record that names the stage it died at;
  an unlistened console is a sentence naming the address and port.
```

Its sibling does the same for a long silence. The scripted board sits behind a scripted firewall that drops the board's datagrams once the host has been silent too long - the failure measured on the bench - and each of the five ways a run without a return line can end must be reported as itself. Five mutants of the tool, one per decision, must each fail it:

```
$ python tools/board_run_outcome_check.py
board_run_outcome_check: PASS - 36 causal assertions over eight scripted boards behind a modelled firewall;
5/5 mutants caught; no board
```

> **Note.** "No board" is the whole of what that proves: the host tool's

decisions. That the monitor writes its run record at the jump, the return and an exception, and that the empty keepalive datagram is harmless to it, are board facts, and a board run is where they are proven.

A gate that needs the compiler and cannot find it says so and does not pretend:

```
$ python tools/module_engine_check.py
module engine gate: set PMF_COMPILER, or provide PureMetalForge.exe in the repo
```

The one gate every lane runs

Almost every gate above compiles something. One does not, and it is the one to run before any commit, whatever you changed:

```
python tools/layering_check.py
```

It is the layering gate. A library exists once and boards differ in the interface under it, and this refuses the ways that rule gets broken: a shared file calling a name only one board defines, a shared file including a board path, a board-extension file appearing under `Anvil/`, a second implementation of a protocol the tree already owns, a call into another file's private namespace, and one name defined twice in files that can be linked into one image.

It resolves names rather than walking includes, because the include model here is flat: a shared file calls a board procedure with no include edge to find, and a single non-test downward include sits in front of nine hundred downward calls. It needs no compiler and no board and finishes in a few seconds.

> **Note.** Today's findings are recorded in `tools/layering_ratchet.json` and

the gate fails on a NEW one, not on the existing ones. It is a ratchet: counts may fall and the tool will ask you to refresh the file, and raising one needs `--allow-growth "<reason>"`, which prints a banner. `--self-test` builds a throwaway tree with one violation of every rule planted in it and proves each rule goes red. `docs/LAYERING.md` has the rules, the measured counts, and the list of what its parser cannot see.

A gate that examined nothing has not passed

This is the sharpest idea in the whole test suite and it deserves its own section.

A checking tool usually ends the same way: return failure if problems were found, success otherwise. That is correct right up until the collection it was checking comes back **empty**. Then there are no problems, because there was no work - the loop body never ran - and the tool prints its success line having verified nothing at all. The denominator went to zero and the verdict stayed green.

So there is a guard. You count what you actually examined, and say so before you are allowed to succeed:

```
from gatecheck import examined
...
examined(len(cases), "boot container cases", minimum=3)
return 1 if problems else 0
```

Trap. The exit code for abstaining is **2, not 1**, deliberately. A gate

that failed found something wrong. A gate that abstained found nothing to look at - a broken harness, a bad path, a moved corpus - which is a different problem needing a different fix. Anything scripting these tools should treat 2 as "this told you nothing" rather than folding it into "failed".

The guard has its own self-test, whose comment is the whole philosophy in one line: *a guard nobody has seen fail is a guess.*

Note. Honest gap: at the time of writing, that guard is imported by

exactly **one** gate. The idea is right and its adoption is not finished. If you are writing a gate that iterates a corpus, use it.

The template

Every emitted gate has the same shape. Here is a complete one, in full:

```
#!/usr/bin/env python3
"""Compile and execute the nonblocking SNTP service state/ownership gate."""

from __future__ import annotations

import argparse
import os
from pathlib import Path
import sys
import tempfile

import tcp_multiif_emitted_check as emitted
```

```
def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--compiler", default=os.environ.get("PMF_COMPILER"))
    parser.add_argument("--interp", default=os.environ.get("PMF_A64_INTERP"))
    args = parser.parse_args()
    compiler = emitted.required_path(args.compiler, "PMF_COMPILER")
    interp = emitted.required_path(args.interp, "PMF_A64_INTERP")
    emitted.PROBE = emitted.ROOT / "RaspberryPi4" / "Tests" / "ntp_service_emitted_gate.pi4"
    a64 = emitted.load_interpreter(interp)
    with tempfile.TemporaryDirectory(prefix="anvil-ntp-service-emitted-") as temporary:
        image = emitted.build(compiler, Path(temporary))
        result, steps = emitted.execute(a64, image)
    if result:
        print(f"ntp_service_emitted_check: FAIL assertion {result} after {steps:,} A64 instructions")
        return 1
    print(f"ntp_service_emitted_check: PASS - 35 assertions, {steps:,} A64 instructions")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Adding a check

```
1. Name it <subject>_check.py in tools/, or <subject>_emitted_check.py
   if it compiles and executes. Gates about the instruction set live in
   tools/a64/ as a64_<subject>_check.py.
2. Give it a one-line docstring saying what is compiled and executed.
   argparse takes it as the description.
3. Two options, both defaulting from the environment: --compiler from
   PMF_COMPILER, --interp from PMF_A64_INTERP. NEVER a hard-coded path.
4. Resolve them through required_path, so a missing tool is a sentence
   naming the variable, not a traceback.
5. Put the assertions in a .pi4 gate source under RaspberryPi4/Tests/,
   not in the Python. The thing under test is the emitted code.
6. Compile into a temporary directory. Write nothing into the tree.
7. Execute under the sandbox: alignment guard on every access, a read
   whitelist of image, BSS and stack, a write whitelist of BSS and
   stack, a step limit. An access outside those ranges is a refusal
   naming the address, not a silent zero.
8. Return the failing assertion number in x0. Zero is pass.
9. Print one line: "<gate>: PASS - <n> assertions, <steps> A64
   instructions", or the FAIL form naming the assertion.
10. Write the mutants.
11. Say what it does not prove.
```

Trap. Item 3 is not style. Thirty gate harnesses once read a hard-coded

path to one particular working copy, which meant they tested that copy and not the tree they were run from. A hard-coded path in a test is a test of somebody else's disk.

The mutants are what make it a gate

A test that only proves the good path is half a test. The shared runner rebuilds the same gate source with deliberate defects injected and requires each one to be caught - and not merely caught, but caught by **the specific assertion that owns that property**:

```
if mutation_result != expected:
    print(f"FAIL {mutation} mutation returned {mutation_result}; expected assertion {expected}")
```

Not "it failed". *Which* assertion caught it. A mutant caught by the wrong assertion means two checks are entangled and one of them is not testing what its name says.

> **Note.** The build-number gate is the fullest worked example: twelve named

checks - bump by one, date and time stamped, ledger line, every caller, a fixture is not the monitor, frozen, refusals, counted once, the compiler is named, concurrent builds, the lock blocks - and **eight deliberate sabotages**, each of which must be rejected by the check that owns that property. Drop the lock. Drop the stamp. Bump by two. Forget an already-counted compile. Bump a frozen marker. Leave the compiler out of the ledger line.

The last paragraph of every gate

Every gate document in this repository ends by naming its own boundary. Two examples, quoted:

```
The gates execute emitted code over a modelled panel. They cannot show
that the bytes at the scanout base are the bytes the glass is lit with,
that a real 12 MiB region is free on real silicon, that a 4 MB copy fits
in the window between a payload returning and the console repainting, or
that a picture survives the trip over UDP. Those are board facts and
they are owed.
```

```
Neither is a hardware claim. There is no MMU, no cache, no DMA and no
firmware in either; the device register is interpreter RAM and cache
maintenance is a stub. Silicon acceptance is a separate, board-slot
claim.
```

> **Note.** Write that paragraph for your own gate, and write it **first**.

It is much harder to write honestly after you have watched the thing turn green.

And then the board

A green gate set means the emitted code does what the assertions say under a model. It does not mean the hardware agrees.

When a board is attached, the board is where the verdict comes from. The reason is not ideology: an emulator agrees with itself. On one project's first silicon run of a suite that had been passing under a model, the board caught a wrong expectation the model had been happily confirming, and found failures that passed emulated - two of them the reverse of what had been predicted.

The model stays right for the things a board cannot do: exact instruction counts, constant-time work, a run with no board attached, and developing the model itself.

Pi4 optional Bluetooth TX queue evidence

tests/pi4_bt_tx_queue_gate.pi4 passes 296,896 physical byte push/pop checks, including empty/full refusal, wraparound, bursts and unchanged UART-open state. It never opens Bluetooth. Two warm eight-direction speech-only payloads then retain complete generated PCM and recurrent-state identity, with zero underruns, RX errors, late events, core faults or disconnect timeouts. The second also verifies an empty queue, closed UART and zero transmit timeouts after shutdown. Peak occupancy stays below 1,600 of 4,095 bytes.

Against the same blocking-UART application, sustained generation ratios fall from 0.909..0.981 to about 0.841..0.908. First non-silent scheduled stream positions stay below 350 ms; these are sender positions, not microphone measurements. The audio-service maximum gap is 1.93 ms and 2.71 ms in the two buffered runs versus 4.21 ms blocking. This improves headroom but does not prove simultaneous map rendering. All payloads return with healthy cores, the deadman off and Vulkan console restored; no firmware flash is required.

Exact shared SBC analysis movement and matrix reduction

The fixed-point encoder now forms one reversed frame-plus-history window instead of shifting 72 history words for each of 16 blocks. Eight exact coefficient symmetries replace the 16-term cosine matrix sum; integer regrouping preserves every result. History is committed once per frame. There is no sample-rate, bitpool, rounding, precision or voice change.

tests/pi4_sbc_linear_gate.pi4 retains the prior analysis as its reference. The physical differential covers 1,024 entire encoded mono/stereo frames, all 128 subband values and all 80 history words, including silence, extrema, alternating signs, ramps, impulses and pseudorandom sequences. A paired cached run measured 4,783,149 versus 3,678,749 counter ticks, 23.1% less encoder time. In the whole warm eight-direction Bluetooth run, encoding/resampling/ diagnostic capture falls from 1.314 s to 1.118 s; sustained generation ratios fall from 0.841..0.908 to 0.834..0.901. Complete speech PCM/state identity, zero underruns/transport faults and clean Bluetooth shutdown all remain. Simultaneous map rendering still needs its own later acceptance run.

Where the engineering notes are

Seventy-odd notes sit beside this book. This chapter is the map, arranged by what you are trying to do.

Why this chapter exists

This book is the way in. It is not the whole record.

Beside `docs/guide/` there are about seventy engineering notes. Each one is the account of a single piece of work: what was built, how it was proven, and - the part that matters most - exactly where its boundary is. They are written for somebody who already knows the machine, and several of them exist mainly to stop a future reader from believing something is finished when it is not.

You do not read them in order. You go to the one that owns the thing you are touching. This chapter tells you which one that is.

> **Note.** Read the STATUS SENTENCE FIRST. Most of these notes open with a

status line, and it is load-bearing: "desk-proven", "not integrated into a board build", "passed on Pi 4 silicon in builds 98 and 99", "no hardware run or cold-boot causality claim". The difference between "proven on a desk" and "proven on a board" has cost this project real days. The status line is the note telling you which kind of claim follows.

Start here

```
ARCHITECTURE.md     The layer model, the resident-monitor role, runtime
                    memory ownership, the rendering boundary. If you read
                    one note, read this one.
PORTABILITY.md      How includes and extensions are arranged so the
                    portable core stays portable.
LAYERING.md         The gate that keeps a library from depending on one
                    board, why it resolves names instead of walking
                    includes, and what its parser cannot see.
MIGRATION.md        How this repository came to exist.
PROVENANCE.md       The two records Anvil keeps instead of trusting an
                    old export hash. PROVENANCE_INVENTORY.md is the
                    review; PUBLICATION_REVIEW.md is the publication
                    question separately.
LICENSING.md        What is MIT and what is not, and the boot-media
                    names for redistributable firmware.
THIRD_PARTY_NOTICES.md  The third-party map.
NEW_BOARD_PORTS.md  What the non-bootable board stubs are for.
```

Building and identity

```
BUILD_NUMBER.md     Where the build number lives, how it moves, the
                    ledger, and why there is no way to build without
                    counting. Chapter 2 is the short version.
```

Payloads, services and the console

```
PAYLOAD_LIFECYCLE.md  The contract a returning payload signs: hardware
                      ownership, the Ethernet recovery path, and the
                      checks that are still owed. Chapter 5's source.
BOARD_RUN.md          Running a payload from the host and what is
                      recorded about it. Chapter 6's source.
CONSOLE_PROMPT_STYLE.md  Why the prompt looks the way it does.
WALL_CLOCK.md         Wall time: the sources, the trust boundary, and
                      why an unset clock says so.
HTTP_SERVER.md        The portable HTTP connection engine.
EXFAT.md              The target-neutral exFAT filesystem.
```

Cores

```
RAW_CORES.md        The first milestone for joining the other BCM2711
                    cores - the desk-gated, inactive foundation.
MULTICORE_STATUS.md Where secondary cores actually stand, including what
                    passed on silicon and what has not. Chapter 7's
                    source.
CORE_ACCEPTANCE.md  The bounded acceptance command for the cores.
SCHEDULER_PLAN.md   Lifecycle, cooperative contexts, and the separate
                    asynchronous work.
SCHEDULER_BINDING.md The explicit EL3 binding, with its emitted-code
                    desk proof.
```

EL3 and boot firmware

```
EL3_BOOT_ACCEPTANCE.md  The cold-boot acceptance record, and the exact
                        remaining boundary. Read before touching boot
                        configuration.
EL3_ACCEPTANCE_COMMAND.md  el3test: an opt-in silicon diagnostic that is
                        never run during boot.
EL3_DESK_STATUS.md      The desk-gate status.
EL3_STARTUP_REVIEW.md   The startup review, desk-only.
```

Driver modules

```
MODULE_FORMAT.md    The pinned PMFMOD v1 binary contract and the current
                    HAL seam. Chapter 9's source.
MODULE_PIPELINE.md  Why a module pipeline is wanted at all.
```

Networking

```
NET_TRANSFER_ROUTE.md      The contract get and put are built on, and
                           the two status paths.
NET_RECEIVE_OWNERSHIP.md   A frame comes off a receive queue exactly
```

```
                           once, and what went wrong when two readers
                           existed.
NET_RECV_ORDERLY_CLOSE.md  Why a peer's close is not the end of an
                           upload.
WIFI_RECOVERY.md           The automatic reconnect path, stated as an
                           explicit policy rather than a reflex.
WIFI_INITIALIZATION_RETRY.md  The radio-initialization retry slice.
WIFI_RX_GLOM_DESIGN.md     Receive-glom loss and the parser change.
WIFI_RX_READINESS_TELEMETRY.md  Evidence collection for idle polling.
```

USB, PCIe and storage

```
USB_HOST_PHASES.md        What the two boot-log lines actually cover.
USB_COLD_READINESS.md     Why the successful boot and the intermittent
                          failure are separate questions.
USB_HOST_BOOT_HANG_20260912.md  The narrowed boot stop at USB host.
USB_LOCAL_FLOW_GATE.md    A gate that overstated its own coverage, and
                          the correction. Worth reading as an example.
XHCI_RETAINED_MARKERS.md  The trace hook that does nothing but mark.
XHCI_STOP_OWNERSHIP_AUDIT.md  A read-only source audit, explicitly not a
                          claim about the hang.
PCIE_ADOPT_ORDER.md       PCIe ownership and adoption order.
VL805_COLD_FIRMWARE.md    The cold PCIe path and the endpoint.
```

Display, touch and graphics

```
EARLY_DISPLAY_BOOT.md      Where display comes up in the boot order.
EARLY_V3D_BOOT.md          When the V3D console is activated.
DSI_CACHE_MAP.md           What CacheEnable maps for the panel.
DSI_EARLY_CACHE.md         Arming exactly the next ScreenUp.
SCREEN_DSI_READOPTION.md   Changing drawing geometry without changing
                           ownership.
BANNER_STATUS_ROW.md       The identity row, and the seam argument about
                           what a service-table call may reach.
BOOT_TIMING_RECORD.md      screen timing: reading back the boot record.
BOOT_I2C_TRACE.md          touch trace: reading retained failure
                           evidence without probing again.
TOUCH_VISUAL_TEST.md       touch test, after touch reports ready.
TOUCH_KEYBOARD_LABELS.md   How key labels get their face.
DMA_CACHE_WALK.md          The per-line cache walk kept in registers.
DMA_QUIESCENCE.md          A desk-tested correction, with no causality
                           claim attached.
VULKAN_COMPATIBILITY_STATUS.md  The honest inventory: implemented
                           layers, missing prerequisites, and no claim
                           of a production Vulkan device.
Anvil/Graphics/Vulkan/COVERAGE.md  The four proof classes and the
                           current public command boundary.
Anvil/Graphics/Vulkan/ROADMAP.md  What remains after the resident Neon
                           adapter and before broader Vulkan coverage.
VULKAN_IDE_CHROME_ACCEPTANCE_2026-09-16.md  Exact Pi 4 atlas/chrome
                           artifacts, hashes, commands and acceptance boundary.
```

```
VULKAN_NEON_CHROME_ADAPTER_2026-09-16.md  The persistent widget adapter,
                          real-widget desk ledger and remaining Pi proof.
```

The remaining `VULKAN_*` notes are dated proofs of individual slices - descriptor state, sampled state, dynamic render geometry - and are best read from the compatibility inventory outward.

The current Vulkan conversion boundary is specific. Public draw lists can snapshot one dynamic scissor per draw. The Pi backend normalizes that rectangle and changes the V3D clip packet only when needed. Atlas sampling, UV vertices, push tint, exact source-over blend and one display-DMA presentation are joined by `vulkanAtlasChromeProof.pi4`; cache ranges are maintained as one batch with one final barrier. The final build-148 RAM-only Pi 4 run returned exact x0=0x6C85A0 in 10.2 seconds. Its report records one TFU advance, one bin/render pair and one display-DMA presentation; the exact report and pixel oracle pass all 211 checks, and fresh capture sequence 8 is a clean rotated 800x1280 picture. The monitor returned to its prompt with the deadman off and no core leases, so this scene is now a silicon pass.

The first real adapter dependency is now present too. Neon can publish its current font's linear CPU RGBA8 raster, exact dimensions and copied glyph UVs through a read-only contract. A font change invalidates every query until the next successful ensure. The CPU-only ensure does not run the TFU or V3D and does not expose or publish the private tiled atlas. It also reserves one opaque white texel and copies its UV rectangle so boxes and glyphs can share one sampled+tinted pipeline. Its emitted gate passes 26 source checks, 41 run-time assertions and five hostile mutation checks.

That refactor and the renderer hook change the returning diagnostic's bytes. The current tree builds a 977,076-byte PMF with expected report pointer x0=0x6C85C8. It is desk-green, but another lane owns the shared Pi 4, so the book does not transfer the older image's silicon result to those new bytes. The same bounded returning run is still required after the lease is released.

The persistent Vulkan-backed adapter for Neon's boxes, atlas text, fans, outlines, line lists and clipping is now desk-green. It keeps its pipeline, descriptor set, atlas and vertex storage resident, installs one complete primitive family, and submits once per frame without processor pixel drawing. Fan and line builders are bounded and lower only into public Vulkan triangle-list vertices. The real existing panel, menu, list and text widgets produce 109 quads, 47 draws, 654 vertices and five scissor states; the scene consumes one present intent and calls display DMA once. The next gate is that exact frame plus the new primitive shapes and screenshot on the Pi 4, then clean return on HDMI, DSI, rotation and resize.

Other boards

```
PI3_BOOT.md                 Pi 3 cold entry, desk implementation.
PI3_SELF_UPDATE.md          Direct firmware boot-file staging and replacement.
PI3_INTERRUPT_TIMER.md      Core 0's physical timer ownership.
PI3_USB_FOUNDATION.md       The controller-independent USB layer.
PI3_USB_ENUMERATION.md      Enumeration, not yet boot-wired.
PI3_USB_DIAGNOSTIC.md       usbdiag in the updater.
PI3_LAN9514_TRANSPORT.md    The USB Ethernet transport, desk-proven.
PI3_LAN9514_RUNTIME.md      Its runtime half.
ROCKPI4C_FOUNDATION.md      RK3399: MiniDP first light on hardware.
ROCKPI4C_DISPLAY_INITIALIZATION.md  The MiniDP contract.
```

Related project notes

The forum notes - `FORUM_PLAN.md`, `FORUM_COMPATIBILITY.md`, `FORUM_REGISTRATION.md` - describe a separate application built on this foundation, not a part of the monitor.

Where this book lives, and when it has to move

One house rule governs this book itself, and it applies to every public repository in this family:

```
Each public repository carries its book in docs/guide/, as chapter
files and as a rendered PDF beside them.

The book is updated in the SAME COMMIT as any change to what a
developer does - a command, a flag, a file name, a printed message, a
procedure, a boundary.

A book that lags the code is a DEFECT, not a chore. It is filed and
fixed like any other.
```

The reasoning is the same one that runs through the rest of this repository. A reader who follows a documented command and gets something else has learned that the documentation lies, and they stop trusting all of it - including the parts that are right. Stale documentation is worse than none, because none at least sends people to the source.

So the test for a change is not "did I break a chapter". It is: **would a developer following this book now do something that does not work?** If the answer is yes, the book moves in that commit, the PDF is re-rendered, and both go in together.

> **Note.** The command that renders the PDF is in `docs/GUIDE.md`, beside the chapter index. It takes any folder of these chapter files, so every book in this family is produced by the one renderer - which is what stops two books drifting into two formats.

The one habit worth copying

Notice what almost every note in this list has in common: it says what was NOT established. "Desk-proven, not integrated into a board build and not silicon-proven." "No hardware run or cold-boot causality claim." "The previous frame gate overstated its coverage."

That is the whole reason the set is usable. A note that only recorded successes would leave you to discover the boundaries yourself, one board at a time. If you add a note, write the boundary down first and the achievement second.

Anvil on a Raspberry Pi 3

The same Anvil core on a BCM2837 board. The firmware starts a small

Pi 3 EL3 stub, then enters the complete monitor directly as `kernel8.img`.

What this board runs

The Raspberry Pi 3 Model B runs the shared Anvil core with backends for the BCM2837. Its composition root is `RaspberryPi3/Board/board.pi3`; its own firmware stub is `RaspberryPi3/Board/armstub8.asm`. Shared commands and filesystems come from `Anvil/`.

```
RaspberryPi3/Board/board.pi3       composition, placement and prompt
RaspberryPi3/Board/armstub8.asm    firmware EL3 entry stub
RaspberryPi3/Lib/mmu.pi3           BCM2837 page tables
RaspberryPi3/Lib/*.pbi             Pi 3 timer, UART, mailbox and SDHOST
```

Direct firmware entry

The normal cold boot does not run an Anvil slot selector. Firmware loads the custom `armstub8.bin` at address zero, then the complete Anvil monitor as `kernel8.img` at `$200000`. The stub configures the BCM2837 counter and SMP coherency, establishes EL3 state, masks exceptions, parks secondary cores, and enters AArch64 EL3h. It passes the firmware DTB address in x0; the compiler startup reaches Main with that value intact. The monitor verifies EL3 at entry and in `Main`, and refuses a lower-level handoff.

```
monitor image and entry     $200000
firmware DTB                $1000000
monitor BSS                 $1100000
monitor stack top           $1F00000
```

These values are checked against the emitted PMF metadata and firmware config. The Pi 3 stub uses the BCM2837 local-control block at `$40000000` and its 19.2 MHz counter source. It does not configure the Pi 4 GIC.

Startup and storage

Main captures x0 before its first ordinary runtime call. It validates the firmware DTB and RAM extent, performs required firmware mailbox queries before enabling the MMU, then initializes the console and prompt. It does not mount a filesystem or verify slot metadata before showing the prompt. SDHOST starts lazily on the first card operation, using the CORE clock remembered before MMU/cache enable; its cached execution path refuses an absent clock handoff or a mixed MMU/cache state.

The monitor keeps Anvil's `boot <name>` command for file-backed payloads after startup. That payload loader is separate from firmware's cold loading of `kernel8.img`. On the verified Model B v1.2, the configured TrueType font is loaded before the first banner; saved Wi-Fi connects automatically, obtains DHCP, and serves the UDP console. A 64 KiB wireless transfer passed its board CRC check. These are hardware results for that board revision, not claims for every Pi 3 variant.

VideoCore IV has a returning-payload proof for power-domain activation, IDENT read, and a 64×64 offscreen clear. It does not yet provide Pi 3 Vulkan or a GPU-to-scanout presentation path. USB host and USB Ethernet are also not implemented.

Building and checking

```
python tools/build.py pi3 --compiler <PureMetalForge.exe>
python tools/pi3_direct_boot_check.py
```

The board target builds the stub and complete monitor. The direct-boot gate checks the stub firmware slots, EL3 register setup, primary DTB handoff, secondary-core parking and release, PMF placement, and the actual emitted stub-to-compiler-startup-to-Main path. It is a host check; a board cold boot is still required to establish firmware and silicon behavior.

Replacing the boot files

Use `tools/pi3_boot_stage.py` to validate the exact monitor and stub hashes against the pinned firmware set and create a new bundle. Review its `SHA256.json` manifest, then use `tools/pi3_provision.py` on an already-mounted boot volume with the reviewed manifest hash and a new host backup directory. The provisioner backs up files it changes before writing, checks readback hashes, and does not partition or format media. The normal update replaces direct firmware files; it is not an A/B slot transaction. See `docs/PI3_BOOT.md` and `docs/PI3_SELF_UPDATE.md` for the full contract and commands.

TrueType font container

T0 admits an in-memory sfnt font or the first face of a TrueType collection after checking every table boundary and checksum.

The memory contract

T0 is the file admission layer for the TrueType engine. It accepts a font already in memory and proves that its sfnt header, table directory, table bounds and table checksums are safe to inspect. It also accepts a TrueType collection and selects its first face. It does not load a file from a disk, decode metrics or character maps, read outlines, rasterise a glyph, or draw.

The public memory contract is:

AnvilTrueTypeOpen(data, bytes) AnvilTrueTypeClose() AnvilTrueTypeTableCount() AnvilTrueTypeTableTag(index) AnvilTrueTypeTableOffset(index) AnvilTrueTypeTableLength(index) AnvilTrueTypeFind(tag) AnvilTrueTypeError() AnvilTrueTypeErrorTag() AnvilTrueTypeErrorOffset() AnvilTrueTypeErrorText() ; pointer to a bounded NUL-terminated ASCII sentence

Table offsets in a collection are offsets from the beginning of the whole collection. Every table is checked before it is read. A duplicate tag, overlapping table, truncated table, unsupported collection header or checksum failure refuses the font. The refusal names the table and offset when those are available through both numeric accessors and the bounded diagnostic text buffer. The head table checksum is calculated with its checkSumAdjustment field treated as zero, as required by the sfnt format.

Every open attempt invalidates the previous table view, including a failed attempt. `AnvilTrueTypeClose()` clears the table count, data accessors and diagnostic sentence. A decoder that caches table pointers should keep the T0 revision token and discard those pointers after any close or new open.

The gate uses two small in-memory fonts, one standalone sfnt and one collection whose first face is selected. It then corrupts the directory in several independent ways: truncation, an out-of-bounds table, a bad checksum, an overlap, a duplicate tag and an invalid collection offset. These are container checks only; lifecycle cases also prove that a refusal, a later successful open and close do not leave stale table data or diagnostics. Passing them does not claim metrics, cmap, outline, raster or renderer support. Later atlas work will continue to use the existing single rasterise-once cache and its Vulkan texture path.

Format references

The container follows the OpenType sfnt rules in the [OpenType overview](https://learn.microsoft.com/en-us/typography/opentype/spec/otff) and the [`head`](https://learn.microsoft.com/en-us/typography/opentype/spec/head) table specification. The [Apple TrueType Reference Manual](https://developer.apple.com/fonts/TrueType-Reference-Manual/) documents the TrueType collection and outline format.

TrueType font metrics

T1 turns a checked font file into bounded font-wide metrics and per-glyph advances.

The four required tables

T1 consumes a font that T0 has already opened. It reads `head`, `maxp`, `hhea` and `hmtx`. The parser validates each table before publishing any metric. If a later open replaces the font, the old metrics become unavailable.

`head` supplies units per em and the `loca` offset format. `maxp` supplies the glyph count. `hhea` supplies signed ascender, descender and line gap values, plus the number of long horizontal metrics. `hmtx` supplies each long advance and left side bearing.

Most fonts store fewer long metrics than glyphs. After the last long metric, each glyph has only a side bearing. T1 uses the final long advance for those later glyphs, as the table format requires. The gate checks a three-glyph font with two long metrics, including the inherited final advance and a negative side bearing.

Optional OS/2 metrics

The `OS/2` table is optional. When present, T1 checks the minimum length for versions 0 through 5 before reading weight, width, typographic ascender, descender and line gap, or Windows ascent and descent. Missing `OS/2` is reported as missing; it is not a font error.

The signed values in `head`, `hhea`, `hmtx` and `OS/2` remain signed. T1 reports a table tag and a byte offset when it refuses a short table, an invalid version or a value outside the supported range.

The reader contract

Open T0 first, then call `AnvilTrueTypeMetricsOpen()`. The getters are `AnvilTrueTypeUnitsPerEm()`, `AnvilTrueTypeIndexToLocFormat()`, `AnvilTrueTypeGlyphCount()`, `AnvilTrueTypeAscender()`, `AnvilTrueTypeDescender()`, `AnvilTrueTypeLineGap()` and `AnvilTrueTypeNumberOfHMetrics()`. Call `AnvilTrueTypeHorizontalMetric(glyph, advanceOut, bearingOut)` to fetch one glyph's advance and left side bearing into caller-owned integer storage.

Use `AnvilTrueTypeHasOS2()` before reading optional fields. `AnvilTrueTypeMetricsClose()` clears the decoded values. Every getter also checks the T0 revision, so closing or replacing the source font makes old metrics unavailable even if the caller forgot to close T1 first.

T1 does not map characters to glyphs, read outlines, rasterise, lay out text or draw. Those belong to later milestones. The shared glyph atlas remains the rasterise-once cache consumed by the Vulkan text path.

Gate and independent fonts

The emitted gate builds a checksummed sfnt in memory. Its 45 cases cover positive and negative metrics, long and tail horizontal metrics, missing optional `OS/2`, truncated or invalid required tables, invalid `head` and `hhea` values, OS/2 version bounds, glyph range refusal and stale metric state after replacing or closing a font. A source mutant weakens the `hmtx` length check; the gate rejects it.

The same emitted reader was checked against `Abel-Regular.ttf` and `CourierPrime-Regular.ttf`. An independent host parser read their table fields and compared units per em, location format, glyph count, ascender, descender, long metric count and three horizontal metric pairs. Abel reports 2,048 units per em and 259 glyphs; Courier Prime reports 2,048 units per em and 415 glyphs.

The font specimen set contains Abel, Acme, Aladin, Almendra, Amarante, Anton, Arbutus, Arvo, Bangers, Courier Prime, Crimson Text and Lato. These static TrueType fonts come from the Google Fonts repository at commit `f2bd09badbc763d8757951d52deec29da27e85fb`; each is distributed with its SIL Open Font License 1.1 text. The monitor recorded the 12 fonts (1,536,152 font bytes), 12 license files, a manifest and a README on the Pi 4 USB data partition. All files were read back with matching bytes and SHA-256 hashes; the manifest records each source URL, size and digest. See `_work/truetype-fonts-20260918/board-readback.json` for the recorded evidence.

The format references are the [OpenType overview](https://learn.microsoft.com/en-us/typography/opentype/spec/otff), [`head`](https://learn.microsoft.com/en-us/typography/opentype/spec/head), [`maxp`](https://learn.microsoft.com/en-us/typography/opentype/spec/maxp), [`hhea`](https://learn.microsoft.com/en-us/typography/opentype/spec/hhea), [`hmtx`](https://learn.microsoft.com/en-us/typography/opentype/spec/hmtx) and [`OS/2`](https://learn.microsoft.com/en-us/typography/opentype/spec/os2) specifications, and Apple's [TrueType Reference Manual](https://developer.apple.com/fonts/TrueType-Reference-Manual/).

The gate proves metrics only. It does not claim character mapping, outlines, raster coverage, cache behavior or board text rendering.

TrueType character mapping

T2 maps Unicode scalar values to glyph IDs through checked cmap format 4 and format 12 subtables, and decodes strict UTF-8 into caller-owned storage.

Mapping contract

Open T0 and T1 first, then call `AnvilTrueTypeCmapOpen()`. `AnvilTrueTypeGlyphForCodepoint(codepoint)` accepts a Unicode scalar value and returns a glyph ID. Zero is the not-found glyph; invalid scalars are reported through `AnvilTrueTypeCmapError()` and `AnvilTrueTypeCmapErrorOffset()`. UTF-16 surrogate values and values above U+10FFFF are refused.

The reader checks the cmap directory and the selected format 4 and format 12 subtable bounds before it reads mapping data. Format 4 handles BMP mappings; format 12 supports supplementary-plane groups. The decoder checks segment/group ordering, range-offset bounds, glyph limits and UTF-8 continuation bytes. It rejects overlong encodings, surrogates and out-of-range sequences.

`AnvilTrueTypeDecodeUtf8(text, byteCount, glyphOut, capacity)` returns the number of glyph IDs written. Empty input is valid and returns zero. Invalid input or insufficient capacity returns -1. A first pass validates the whole byte sequence and checks capacity, so an undersized output array is not partially filled.

Emitted proof

The emitted gate builds format 4 and format 12 subtables and checks BMP and supplementary mappings, missing and invalid scalars, valid empty text, a multibyte UTF-8 sequence, overlong and surrogate encodings, and output-capacity atomicity. It also corrupts the format 4 reserved pad and range offsets, and format 12 group ordering; each malformed table is refused. A source mutant weakens the capacity check and must fail the gate.

The emitted reader was also compared with the independent `fontTools` best-cmap mapping for five scalar values each in Abel and Acme. These are host reference checks against the actual font tables; they do not claim shaping, bidi, variation selectors or text layout.

Format references

The [OpenType `cmap` specification](https://learn.microsoft.com/en-us/typography/opentype/spec/cmap) defines encoding records, format 4 segments and format 12 groups. UTF-8 validity follows Unicode scalar-value encoding; this reader decodes only and does not normalize or shape text.

TrueType outlines

T3 reads bounded `loca` and `glyf` data into caller-owned font-unit points and inclusive contour endpoints, including transformed composite components.

The outline contract

Open T0 and T1, then call `AnvilTrueTypeOutlinesOpen()`. `AnvilTrueTypeGlyphOutline(glyph, points, pointCapacity, contours, contourCapacity, pointCountOut, contourCountOut)` expands a simple or composite TrueType glyph. Each point uses three 32-bit fields: signed x, signed y and flags, where bit zero is the on-curve flag. Each contour entry is its inclusive final point index in the combined output.

The module validates `loca` length and monotonic offsets against `glyf`. Short `loca` values are big-endian half-offsets. Simple glyph flags and coordinate deltas are bounded before use. Composite arguments distinguish signed XY offsets from unsigned point indices; scale and matrix transforms are applied before point attachment. The decoder rejects recursive cycles, conflicting transform flags, invalid indices, output-capacity overflow and coordinates outside signed 32-bit range. An explicit bounded stack handles nested components because this compiler does not support recursive procedures.

Instructions are not executed, and hinting is not applied. The module handles the TrueType `glyf` outline format; CFF/CFF2 outlines are outside this reader.

Emitted and independent proof

The small emitted fixture exercises big-endian short `loca`, a simple triangle, negative byte-sized XY component offsets, a nonidentity scaled component attached by point index, cycle and conflicting-transform refusal, and output capacity. A source mutant reads short `loca` values as little-endian and must fail.

The emitted fixture compares a multi-contour Abel exclamation mark and Bangers Aacute against `fontTools` resolved coordinates. Bangers Aacute exercises a larger composite outline: 76 points across three contours. The complete gate also rejects a little-endian short-`loca` source mutant. This proof covers outline decoding; it does not prove raster coverage, atlas caching, shaping or drawing.

Format references

The [OpenType `loca` specification](https://learn.microsoft.com/en-us/typography/opentype/spec/loca) defines short and long locations. The [OpenType `glyf` specification](https://learn.microsoft.com/en-us/typography/opentype/spec/glyf) defines simple points, composite component arguments, flags and transforms.

TrueType coverage rasterization

T4 converts expanded TrueType outlines into bounded 8-bit grayscale coverage masks. The reader does not apply hinting or depend on a display backend.

The raster contract

Open the T0 font, T1 metrics, and T3 outline modules, then call `AnvilTrueTypeRasterizeGlyph(glyph, pixelHeight, coverage, capacity, widthOut, heightOut, strideOut, bearingXOut, bearingYOut, advanceOut)`. It expands one glyph and writes a caller-owned coverage buffer. `pixelHeight` must be 1 through 512. Width and height are each limited to 1024 pixels; point, contour and flattened-edge work is bounded to 4096 points, 2048 contours and 16384 edges. Coverage is one byte per pixel with values from 0 through 255. The caller supplies the byte capacity. Stride equals width. The x bearing is the floored scaled left bound, the y bearing is the ceiled scaled top bound, and the advance is rounded to output pixels. Malformed glyphs, unsupported formats, arithmetic overflow, and undersized buffers fail without reporting a successful bitmap.

The current rasterizer uses four-by-four subpixel samples and adaptively flattens quadratic curves with a maximum one-eighth-pixel chord error. Empty glyphs have zero bitmap extent. Contours use nonzero winding, so reversed inner contours preserve holes. Atlas packing, cache lifetime, shaping, hinting, and final drawing are separate layers and are not proven by this gate.

Emitted and independent proof

The emitted gate checks an exact two-contour ring mask with reversed inner winding, an on-curve start with an off-curve end, an all-off-curve contour whose implied midpoint is fractional at low UPEM and 64-pixel output, an empty glyph, and capacity refusal. The raster is compared with independently flattened `fontTools` outlines. The reference casts rays at the same four-by-four sample positions; a second direct ray-cast implementation cross-checks the optimized crossing reference. Every differing real-font pixel must intersect the dense reference outline within a one-eighth-pixel-expanded pixel rectangle, and no pixel differs by more than three four-by-four sample quanta. The separate 32-by-32 reference reports high-resolution deviation; it is a quality measurement, not the rasterizer's sampling contract.

The recorded corpus comparisons use Abel Regular and Bangers Regular, glyphs O and é, at 16 and 32 pixels. The complete measurements and interpreter counts are in `_work/truetype-tests-20260918/t4-results.txt`; the emitted image and symbols are retained under `_work/truetype-tests-20260918/t4-work/`. These desk gates verify masks and bounds for this corpus. They do not establish hinting quality, performance on every font, or visible output through the Vulkan renderer.

Format references

The [OpenType `glyf` specification](https://learn.microsoft.com/en-us/typography/opentype/spec/glyf) defines quadratic outline points and contour flags. The [OpenType `head` specification](https://learn.microsoft.com/en-us/typography/opentype/spec/head) defines units per em and glyph-location format.

TrueType pair positioning and line layout

T6 adds bounded legacy kern and GPOS PairPos lookup to the TrueType engine, then lays out UTF-8 glyph runs into caller-owned fixed-point positions. The layer remains independent of the display and Vulkan atlas.

Open and close order

Open the font with `AnvilTrueTypeOpen`, then open metrics, cmap, outlines, kern, and GPOS in that order. `AnvilTrueTypeSlotSelect` performs this parser lifecycle for resident fonts. Close in reverse dependency order; the slot layer closes GPOS and kern before outlines, cmap, metrics, and T0. Every module binds itself to the T0 revision and refuses stale table pointers after a later open or close.

`AnvilTrueTypeKernOpen` reads version-0 horizontal format-0 pairs. `AnvilTrueTypeGposOpen` selects the `kern` feature from the `latn` script, or `DFLT` when no Latin script is present, and supports PairPos formats 1 and 2 plus extension lookup type 9. PairPos class definitions and coverage are bounded and validated before lookup. Multiple selected GPOS lookups accumulate in LookupList order.

Pair values and fallback

`AnvilTrueTypeGposPair(left, right, values)` writes eight signed design-unit values in this order: x/y placement and x/y advance for the first glyph, followed by the same four values for the second glyph. The output pointer addresses eight native integers. Keeping both advances separate lets layout apply the first value after the left glyph and the second after the right glyph.

`AnvilTrueTypePairPosition` uses the GPOS kern feature when the face has one. A matching GPOS pair is returned as eight values; a miss returns eight zeros and does not apply legacy kern. If the font has no selected GPOS kern feature, the horizontal legacy `kern` value is returned as the first glyph's xAdvance. This face-level source choice avoids double kerning.

Device and variation records are range-checked but their pixel-dependent deltas are not evaluated. Lookups with nonzero LookupFlags and unsupported pair-position formats are reported by `AnvilTrueTypeGposError` as unsupported. Layout keeps drawing with zero pair adjustment and suppresses legacy fallback for that selected feature. Malformed table structure still makes GPOS open fail. Mark attachment, cursive positioning, script-specific shaping outside the selected Latin/default feature, ligatures, bidi, and vertical layout are not implemented here.

UTF-8 layout

`AnvilTrueTypeLayoutUtf8(text, byteCount, pixelHeight, maxWidth, lineHeight, glyphOut, xOut, yOut, advanceOut, alignment, capacity, glyphCountOut, lineCountOut, measuredWidthOut)` decodes strict UTF-8 through cmap and writes glyph IDs, x positions, y positions, and advances to caller-owned arrays. The current bounded scratch capacity is 2,048 glyphs. Pixel height is limited to 512; line height to 4,096 pixels; wrap width to 32,767 pixels. The output arrays each need room for `capacity` native integers.

Positions and advances are signed 16.16 pixel values. x is relative to the start of the line; y is positive upward from the baseline. The first line baseline is zero, and later baselines move down by `lineHeight`. Raster bitmap bearings are applied by the renderer when placing the glyph mask. Left, center, and right alignment use `alignment` values 0, 1, and 2. The measured width is the widest positioned line before alignment.

Soft wrapping occurs at glyph boundaries using the actual signed pair advances; a pair is omitted when its glyphs would land on separate lines. CR and LF create hard line breaks, with CRLF counted once. This is a bounded horizontal glyph-run layout,

not word wrapping or full text shaping. Callers should rasterize and cache the returned glyph IDs separately.

Emitted proof

`tools/truetype_t6_check.py` compiles `RaspberryPi4/Tests/truetype_t6_gate.pi4` with `EnableExplicit`. The synthetic sfnt checks PairPos formats 1 and 2 across two lookups, exact value1/value2 positions and advances, wrapping after pair adjustment, center alignment, CRLF and repeated/trailing hard breaks, legacy-kern selection, unsupported-feature behavior, and malformed class rejection. A compiled source mutant that overwrites rather than accumulates lookup values is rejected. Results are recorded in `_work/truetype-tests-20260918/t6-gate.txt`.

These emitted fixtures prove the covered tables and layout cases. They do not establish browser-grade shaping, device adjustments, or layout behavior for every script and font.

Format references

The [OpenType GPOS specification](https://learn.microsoft.com/en-us/typography/opentype/spec/gpos) defines PairPos formats 1 and 2 and the ValueRecord fields. The [OpenType `kern` specification](https://learn.microsoft.com/en-us/typography/opentype/spec/kern) defines the supported legacy pair format.

TrueType console commands and defaults

The font commands load checked TrueType files into bounded resident slots, select a face for rendering, set a shared pixel height, and optionally restore a saved default path at boot. Built-in bitmap text remains available as the fallback.

Load and select a font

`font load <slot> <path>` reads one file path into an empty slot. Slots are numbered 0 through 3; each file is limited to 1 MiB. The path is one whitespace-free token, such as `data:/fonts/Abel-Regular.ttf`. Loading validates the font and its supported tables before publishing the slot. It leaves the current font selection unchanged.

```
font load 0 data:/fonts/Abel-Regular.ttf
font select 0
font info
```

The load response reports the slot and byte count. Selection binds the slot's checked face for rendering. `font select bitmap` returns to the built-in bitmap font. `font unload <slot>` releases a slot and automatically deselects it first when it is active. It refuses the unload if that slot's generation cannot advance safely. The lower-level slot API itself refuses to unload an active slot.

`font info` reports each slot's size and generation, the selected face's basic metrics when its parser is bound, the current pixel height, and the configured default. It marks an unavailable saved default when the bitmap fallback is active.

Size and saved defaults

`font size <pixels>` accepts an integer from 1 through 512. A valid value updates the renderer-neutral TrueType size for the current boot and records `font.size` in settings. The console asks for `settings save` to write pending settings to the boot medium. Invalid values or trailing text leave the previous size unchanged.

```
font size 24
settings save
font default 0
settings save
```

`font default <slot>` saves the path already associated with a loaded slot, selects that slot now, and sets it as the default for this boot. `font default bitmap` selects bitmap text and removes the saved TrueType path. These edits are pending until `settings save` succeeds.

At startup, after settings have loaded, Anvil attempts the configured default path once. A valid file is loaded into slot 0 and selected. If the storage or file is unavailable or validation fails, the monitor reports the problem and keeps bitmap text active; render calls do not retry the load. With no saved font path, startup does not perform an extra font-file read.

Limits and proof

The settings layer persists a default path and pixel height; it does not persist arbitrary slot contents or restore all four slots. Only the default path is loaded at startup. Paths with spaces are not supported by the current one-token command parser. The file is copied into bounded resident memory before its parser is opened, so later rendering does not depend on an open file handle. The Vulkan screen renderer is the current consumer of these TrueType settings; changing the selected face does

not replace the bitmap glyphs in the CPU or native V3D rendering paths. Console command text remains on its existing console renderer path. The command parser accepts its documented ASCII keywords and one-token paths; UTF-8 decoding is provided by `AnvilTrueTypeLayoutUtf8`, not by command parsing.

The emitted `font_command_gate.pi4` checks command parsing, load/select/bitmap behavior, size bounds and trailing-text refusal, settings values, one-time default restoration, and unavailable-file fallback. `truetype_slots_gate.pi4` exercises parser binding and slot generations using Abel-Regular.ttf. These are desk-emitted checks; they do not substitute for a board-specific font appearance proof.

References

The TrueType/OpenType specification defines the sfnt tables accepted by the font reader: https://learn.microsoft.com/en-us/typography/opentype/spec/. Chapter 4, "The console," describes command output and the explicit settings-save step.

TrueType fonts in the resident Vulkan console

The Pi 4 Vulkan console can rasterize selected TrueType glyphs into a bounded atlas and draw them through its existing text path. Emitted CPU gates and returning-payload GPU comparisons pass for their stated cases. Resident font/Vulkan activation has been observed. The corrected offline Build 210 has not been flashed, so post-fix resident visual and stress checks remain outstanding.

Try a checked font

The settings commands from the previous chapter load a font into one of four resident slots, select it, and choose its pixel size. The sample font files were placed flat in the Pi 4 exFAT data partition under `data:/fonts/`. For example:

```
font load 0 data:/fonts/Abel-Regular.ttf
font select 0
font size 20
font info
screen vulkan
```

The font parser reads strict UTF-8 through the public layout API. The interactive command words and file path are still ASCII tokens, and the path cannot contain spaces. The existing console model owns cursor positions and cell dimensions. With a TrueType face selected, the Vulkan console batches ASCII cells at their existing grid positions. A glyph that fits the cell keeps its normal size and is translated only as far as needed to fit; an oversized glyph is uniformly scaled down to preserve its complete bitmap. This fit affects fixed-grid console cells only: regular TrueType text continues to use the requested pixel height. Changing the face or size does not change row height, column width, or cursor movement. The Vulkan frame path also renders the cursor and preserves the touch-keyboard band. The bitmap face remains the console default until you select a TrueType slot. Cursor and keyboard visuals still need board validation with `screen vulkan` owning the display.

The `screen vulkan` command opts into the resident Vulkan console. On Build 207, the resident run loaded the 35,220-byte Abel font from `data:/fonts/`, selected slot 0, reported 259 glyphs, UPEM 2048, ascent 2006, descent -604, and size 16, then confirmed the Vulkan console active at 1280x800 with a 160x42 scale-1 grid. Screenshot review found presentation defects. Commit `a3f1216` corrects banner preservation and fixed-cell glyph fit; commit `d0dcbbf` fixes atlas shelf advancement. Build 210 has passed offline build identity and map checks, but has never been flashed. Do not treat the earlier activation or returning-payload tests below as resident visual acceptance.

The atlas cache

`NeonVkChromeWarmTrueTypeText(pixelHeight, text)` decodes a NUL-terminated UTF-8 string and prepares its glyphs before a frame begins. It returns the number of decoded glyphs, zero for an empty string, or a negative error. The string is limited to 4,095 bytes; the decoder accepts at most 4,096 glyph IDs for a warm call. Pixel height is 1 through 512.

Each cached glyph is identified by the selected slot, that slot's generation, pixel height, and glyph ID. Replacing a slot advances its generation, so old atlas entries cannot be mistaken for the replacement font. The cache holds at most 512 glyph records and has 1 MiB of CPU coverage storage. A new glyph is rasterized once. All new glyphs from a warm call are uploaded together before the frame references their texture coordinates. If a call repeats only cached glyphs and nothing else has dirtied the atlas, it performs no rasterization and no atlas upload.

`NeonVkChromeTTCacheReset()` clears the complete cache only between frames. It returns success on reset; the next warm call uploads the reset atlas and its new glyphs before drawing. If the cache fills while a grid is being prepared, the console resets

and retries the complete current glyph set once. If that cannot fit, the frame fails and the Vulkan owner starts its safe failure transition.

The resident TrueType console batches fixed cells by ink color into at most two draw segments. Chrome reserves `columns × rows + 512` quads for cells and bounded UI geometry; the tested 256×80 grid uses 20,480 cell quads within the 20,992-quad arena. The emitted Chrome gate passed whole-glyph fitting for oversized fixed cells, atomic capacity refusal, two-color flushing, the full 20,480-cell case, and UV/capacity mutants. The resident owner source also reserves the full console dimensions plus two draw segments. This establishes the bounded desk path; dense-grid rendering still needs a resident board run.

`NeonVkChromeFontHeightSet(font, pixelHeight)` sets an optional per-Neon-face height from 1 through 512. A value of zero clears the override. The general TrueType setting remains in the renderer-neutral slot layer. The touch-keyboard mono face uses its current bitmap pixel height as its TrueType raster size when it has no explicit override. When the TrueType measurement backend is active, widget width and line-height queries use UTF-8 layout for the selected face. Width includes pair positioning and determines right- and centre-aligned origins; multiline text uses its widest line. These queries perform layout only: they do not rasterize glyphs, change the atlas cache, or upload textures. If the face is unavailable or the backend is cleared, Neon uses its bitmap measurements.

Ownership and safe failure

The board owner exposes `VulkanConsoleStart`, `VulkanConsoleStop`, `VulkanConsoleReady`, `VulkanConsoleError`, and `VulkanConsoleQuarantined`. `VulkanConsolePayloadSuspend` retires the resident Vulkan renderer before the monitor enters another payload; `VulkanConsolePayloadResume` may restore it after the monitor returns. The normal console stays on its established framebuffer path unless you opt into Vulkan.

After a Vulkan frame failure, Anvil first waits for the Vulkan device to become idle and then asks the resident V3D owner to invalidate its mappings. Only when both releases succeed does it detach the atlas mapping, disable the DMA callback, and repaint through the framebuffer renderer. It restores a previously visible cursor through a direct CPU save/draw path after safe release, without re-entering the console repaint callback. If Vulkan idle cannot be proved, it retains device handles and memory, quarantines the screen, performs no CPU repaint or cursor/keyboard writes, and refuses payload entry. If Vulkan is idle but V3D invalidation fails, it still refuses CPU repaint and leaves the owner quarantined. An explicit `screen vulkan` retry is required after the owner can prove it is safe again.

The emitted owner gate executes the actual destroy, failover, stop, and payload-suspend procedures with controlled device and V3D results. Its final cursor/keyboard version passed 28 assertions in 5,422 interpreted instructions, including retained handles and no repaint on failed idle, one fallback repaint after safe release, quarantine after failed V3D invalidation, and payload refusal. This checks owner state transitions in an emitted CPU fixture; it does not emulate Vulkan execution or prove hardware recovery from a real MMU fault.

What has been proved

The emitted Chrome cache gate stages the exact production cache and warm procedures, uses a real Abel font in resident memory, and stubs atlas upload rather than invoking Vulkan. It passed 28 checks in 34,927,801 interpreted instructions. It verifies cold raster/upload, a warm hit with no second raster/upload, cache identity across size, slot, and generation, UTF-8 refusal, capacity handling, reset timing, and rejection of a generation-key mutant. The result is recorded in `_work/truetype-tests-20260918/chrome-cache-gate.txt`.

The broader desk gates cover sfnt validation, metrics, cmap and UTF-8, glyf outlines, coverage rasterization, pair layout, resident slot lifecycle, command defaults, and mapped-memory bounds. Their case-specific results are in `_work/truetype-`

`tests-20260918/`. The renderer-neutral measurement gate passes 55 assertions in 14,781,204 interpreted instructions against an independent Abel kerning/advance oracle. It checks UTF-8 and multiline widths, right/centre origins, face sizing and line boxes, bitmap fallback, and that measurement causes no rasterization, cache mutation, or upload: `_work/neon-measure-backend-20260919/gate.txt`. The updated T4 emitted gate also compares the actual-glyph scalar and NEON coverage-conversion paths byte for byte on synthetic and real-font glyphs. The corrected run passed for the synthetic ring, start-on/end-off and all-off-curve glyphs, plus Abel and Bangers O/e-acute at 16 and 32 pixels; measurements are in `_work/truetype-neon-dispatch-20260919/t4-dispatch-results.txt`. NEON handles coverage-count conversion only; outline expansion and raster coverage remain on the scalar path. These tests establish bounded parser, layout, raster, cache, and owner behavior for their stated fixtures. They do not establish browser-grade shaping or resident-console GPU behavior; the separate returning-payload proof below verifies the Vulkan glyph path for its stated specimen.

Twelve static SIL Open Font License 1.1 fonts were copied to the Pi 4 data partition. Their flat card paths are:

```
data:/fonts/Abel-Regular.ttf
data:/fonts/Acme-Regular.ttf
data:/fonts/Aladin-Regular.ttf
data:/fonts/Almendra-Regular.ttf
data:/fonts/Amarante-Regular.ttf
data:/fonts/Anton-Regular.ttf
data:/fonts/Arbutus-Regular.ttf
data:/fonts/Arvo-Regular.ttf
data:/fonts/Bangers-Regular.ttf
data:/fonts/CourierPrime-Regular.ttf
data:/fonts/CrimsonText-Regular.ttf
data:/fonts/Lato-Regular.ttf
```

Their corresponding license paths are:

```
data:/fonts/Abel-OFL.txt
data:/fonts/Acme-OFL.txt
data:/fonts/Aladin-OFL.txt
data:/fonts/Almendra-OFL.txt
data:/fonts/Amarante-OFL.txt
data:/fonts/Anton-OFL.txt
data:/fonts/Arbutus-OFL.txt
data:/fonts/Arvo-OFL.txt
data:/fonts/Bangers-OFL.txt
data:/fonts/CourierPrime-OFL.txt
data:/fonts/CrimsonText-OFL.txt
data:/fonts/Lato-OFL.txt
```

The TTF files total 1,536,152 bytes; the twelve license files, `manifest.json`, and `README.txt` bring the set to 26 files. The host copy is pinned to Google Fonts commit `f2bd09badbc763d8757951d52deec29da27e85fb` at `https://github.com/google/fonts/tree/f2bd09badbc763d8757951d52deec29da27e85fb`. The manifest records the exact raw source URL, license URL, and SHA-256 for every font and license. The board readback record says all 26 files matched their host byte counts and SHA-256 on monitor build 204. These static fonts use TrueType `glyf` / `loca`; the inspected set has no `fvar`, CFF, or CFF2 tables. File transfer and byte identity do not prove glyph appearance through the resident renderer.

The first native TrueType Vulkan run did hit `VK_ERROR_DEVICE_LOST` and an L2T MMU fault at `0x0121E340`. After the texture-width fix in commit `d732179`, the returning-payload proof passed on the Pi 4: it returned in 7.8 seconds, passed 67 checks, rendered four draws with 168 vertices, and showed 24 cold rasterizations with three uploads followed by a warm pass with

zero rasterizations, zero uploads, and 30 cache hits. The corrected report and screenshot are in `runs/vulkan-truetype-widthfixed-20260919/`. The captured image is a rotated physical diagnostic captured by the monitor after the payload returned; it proves this returning payload's Vulkan path, not the resident `screen vulkan` console path.

A second returning-payload comparison rendered Abel glyph 19 at 16 px using both normal text and fixed-grid placement at 0° and 90°. All four copied 24×24 BGRA crops matched the expected 7×13 atlas coverage exactly: 42 nonzero pixels, zero per-pixel error, and no pixels outside the glyph bounds. After inverse rotation, the text and grid crops also matched each other exactly. Each rotation used two draws and twelve vertices. Evidence and decoded crops are under `_work/vulkan-truetype-grid-diff-instrumented-20260919/`. This verifies the returning target's text/grid geometry, UVs, and rotation for that glyph. Its atlas was produced by the scalar/default path; this is neither a full-console image nor proof of the resident NEON atlas path.

The resident source reserves a protected 16 MiB BSS window at `0x0D000000..0x0DFFFFFF`. Build 207 from `3c42349` booted but had visual defects. Build 208 is the last confirmed board boot; its renderer-resume behavior was proved, but that does not validate the newer source. The attempted Build 209 boot-medium write is unverified because the board became unreachable. Build 210, compiled from source `ab357cf` with identity record `89b9fde`, is an offline artifact at `_work/final-vulkan-monitor-ab357cf-retry-eol/anvil.img` (4,161,468 bytes, SHA-256 `9c5518a3112bd20f0a5dfeea415f0885c5d92a1a3e6431f6485166f4115d3381`); its image range is `0x00200000..0x005F7FBB` and BSS is `0x0D000000..0x0DDA2AEF` (14,297,840 bytes). It has never been flashed. A post-fix resident screenshot, dense grid above 4096 cells, renderer resume on the new image, and final native metrics checks remain outstanding.

Source and proof references

The cache implementation is in `Anvil/Graphics/Vulkan/neon_vk_chrome.pi4`; the owner and fallback state machine is in `RaspberryPi4/Board/vulkan_console.pi4`; the monitor's payload boundary is in `RaspberryPi4/Board/cache.pi4`. The emitted cache gate is `tools/truetype_chrome_cache_check.py` with `RaspberryPi4/Tests/truetype_chrome_cache_gate.pi4`. The owner gate is `tools/vulkan_console_owner_check.py` with `RaspberryPi4/Tests/vulkan_console_owner_gate.pi4`; its current cursor/keyboard safe-fallback result is 28 assertions / 5,422 instructions at `_work/truetype-tests-20260918/vulkan-console-owner-cursor-gate.txt`. The full-grid Chrome batch gate is `tools/vulkan_grid_batch_check.py` with `RaspberryPi4/Tests/vulkan_grid_batch_gate.pi4`; its result is `_work/vulkan-grid-batch-20260919/gate.txt`.

The earlier native failure record is under runs/vulkan-truetype-mmu-diagnostics-20260918/; the corrected returning-payload record is under runs/vulkan-truetype-widthfixed-20260919/. Neither is a resident screen-vulkan console proof.

Chapter 4 describes the command and bitmap fallback.

Anvil on the ROCK Pi 4C

The original RK3399 ROCK Pi 4C boots one Anvil component from SD at EL3. Anvil owns its filesystem, HDMI console, and bounded payload experiments.

Boot and recovery

The Rockchip DDR/miniloader loads one Anvil component at `$00040000`. That component contains the EL3 entry prefix, full runtime, and board device tree. The entry checks its exception level, device-tree handoff, stack, interrupt mask, MMU and cache state, UART baud rate, and timer. The source root is `RockPi4C/Board/board.rockpi4c`.

The recovery UART runs at 1,500,000 baud. A returning payload uses a stage address reported by the running monitor; the host compiles for that address and checks that its BSS and stack do not collide with Anvil. The hardware deadman is armed and verified before entry. The payload returns to Anvil's EL3 stack and exception vectors. Use a bounded payload to prove a hardware change before making and installing another full monitor build.

The direct-SD board package contains `anvil.bin`, the extracted component, and `trust-anvil.img`, two checked copies for the fixed SD trust range. It is not a blank-card image. From a running direct-SD Anvil, `trustupdate` checks the image, writes the backup and primary copies, and reads both back. Reboot is a separate command after verification.

Files on SD and eMMC

Build 147 mounts the SD card's exFAT partition. It can also select the eMMC user area's 512 MiB FAT32 data partition at LBA 32768. The filesystem parser is shared Anvil code; the Rock Pi supplies separate SD and eMMC block-device adapters. The eMMC boot partitions, RPMB, and SPI are not written by these commands.

Use `sd` or `emmc` to select a medium and `fs` to inspect the mounted volume. `ls`, `stat`, `cat`, `load`, and `get` read files. `put`, `receive` followed by `save`, `mkdir`, `mv`, `rm`, and `rmdir` change files. Uploads use bounded acknowledged windows and a checked staging area. The writer arms for one file operation and disarms afterward. On the original 4C, build 145 passed creation, transfer, flush, remount, readback, rename, and removal on both volumes. Build 147 mounted both volumes again.

Display and graphics

HDMI runs at 1920×1080/60 through the RK3399 VOPB. The VOPB output bus uses its 30-bit mode even when the HDMI input is RGB888. Anvil's shared TrueType rasterizer prepares a glyph atlas. The board composes terminal rows privately and RGA2 publishes them to the live framebuffer. PL330 handles framebuffer transfers and the bounded fallback. Build 145's resident counters showed 46 RGA row jobs, no RGA failures, and zero CPU text-copy bytes. A cropped screen capture showed sharp text and a clean shaded logo.

Build 147 also passed two live CPU-status update stress runs while receiving 400/400 bytes at 1.5 Mbaud with no UART error or discarded command bytes. The UART is pumped during bounded CPU fallback spans as well as DMA waits. The newer build's bounded pixel capture did not complete across the serial ACK/retry transport, so the last accepted visual crop is from build 145.

A returning payload has rendered a guarded 48×48 private surface with the Mali-T860, copied it through RGA2 to a bounded live scanout area, checked pixels and guards, and restored the saved screen region exactly. The production console does not use Mali. This Midgard GPU has no current hardware Vulkan driver. MiniDP bring-up remains separate work.

See `RockPi4C/README.md`, `RockPi4C/EMMC.md`, and `RockPi4C/RENDERING.md` for the board contract and exact proof limits.

Raspberry Pi 5: source bring-up

Anvil reached an EL3 `pmf>` prompt on Pi 5 hardware from commit `f385c7b`. That boot used a processor-drawn HDMI console; PCIe/RP1, storage, Ethernet and GPU work still need separate acceptance. Host checks can validate reference device trees without a board.

Current boundary

The current Pi 5 monitor is the shared `RaspberryPi4/Board/board.pi4` composition under `#PMF_CHIP = 2712`, with the Pi 5 EL3 stub in `RaspberryPi5/Board/armstub8-2712.asm`. The image is linked at `$80000`; `config.txt` must omit `kernel_address`. `tools/pi5_card_build.py` builds `ANVIL5.IMG` from an exact commit and checks the card before writing it. `RaspberryPi5/Boot/board.pi4` is the earlier entry diagnostic, not the full monitor. The Pi 4 `kernel8.img` and `armstub8.bin` are not Pi 5 substitutes.

The serial capture `Raspberry Pi 5/Serial logs/pi5-boot-20260927-6-f385c7b-FIRST-PROMPT.txt` in the vault records the EL3 stub, HDMI console and `pmf>` prompt. It also records SD CMD2 failure and refusal to start PCIe because the RP1 inbound DMA window was not adopted; USB and Ethernet therefore could not start on that boot. A later returning PCIe payload completed two xHCI No-Op DMA requests on silicon, but its monitor integration and remaining gates were parked. This evidence does not establish working boot storage, network or Pi 5 Vulkan.

The vault's `Raspberry Pi 5/SOURCES.md` identifies the pinned official DTBs, Linux driver references, and datasheets. Its `RESUME HERE - Pi 5 bring-up paused 2026-09-27.md` and `HARDWARE-BRINGUP.md` record the board checkpoint and remaining work. There is no Pi 5 board available for the current review.

Check the source device trees

From the Anvil repository root, use the explicit vault paths on this host:

```
$env:PI5_DTB_DIR = "C:\Embedded Compiler\CompilerEmbedded\Raspberry Pi 5\Boot staging"
py -3 RaspberryPi5/Boot/dtb_contract.py "$env:PI5_DTB_DIR\bcm2712-rpi-5-b.dtb" "$env:PI5_DTB_DIR\bcm2712-d-
rpi-5-b.dtb" "$env:PI5_DTB_DIR\bcm2712d0-rpi-5-b.dtb"
py -3 -m unittest discover -s RaspberryPi5/Boot -p test_dtb_contract.py -v
```

The recorded host run reported `Raspberry Pi 5` and `brcm,bcm2712` for all three variants, with an RP1 bridge at `/axi/pcie@1000120000/rp1`. It also resolved `chosen/stdout-path` (`serial10:115200n8`) to the enabled PL011 at `/soc@107c000000/serial@7d001000`. The source RP1 `serial0` and `serial1` nodes are disabled, so the first debug UART image must not assume either one. Six tests pass; corrupt and wrong-board DTBs, invalid early-UART paths, unmapped UART and disabled WLAN are refused. On another host, set `PI5_DTB_DIR` to the explicit folder with those three named files before running the tests.

These are source DTBs. The bootloader may change the selected DTB, especially memory, reserved areas, and boot data. The current monitor checks the live tree and reads its memory end, DMA mask and board stepping before using those values. This preflight remains a host check; the later serial capture is the boot evidence.

Vulkan on a Raspberry Pi 4

Anvil implements a bounded Vulkan core-1.0 subset on the Pi 4. Accepted draws and colour clears run on the V3D GPU; supported transfers use the V3D texture unit or guarded DMA. Unsupported shapes return a diagnostic rather than a partial result.

Rendering and transfers

`vkCmdClearAttachments` accepts one in-bounds color rectangle and one layer. A full-frame clear before the first draw is folded into the render-pass load clear and works on odd-sized targets. Partial rectangles are recorded in draw order and run in the same V3D bin/render transaction, currently only when framebuffer width and height are even. The 64×64 Pi 4 proof checked all 4,096 pixels for two draws and two clears; an out-of-bounds rectangle was refused before GPU jobs. Its trace and decoded result are documented in `docs/evidence/vulkan-clear-attachments-pi4-20260930/README.md`.

The resident Vulkan console uses the public command and resource path. An accepted draw list becomes one V3D bin/render transaction, with GPU tile clear and store. Supported whole-image buffer-to-optimal-image uploads and optimal-to-optimal image copies use the texture unit. Buffer copies, fills, updates, rectangular linear image copies, linear buffer-to-image uploads, and linear image readbacks use guarded DMA on the production Pi 4 backend. `vkCmdUpdateBuffer` captures caller bytes at recording time; submission transfers the captured bytes by DMA. The display owner presents a completed attachment separately.

The production V3D backend advertises buffer and linear-image transfer execution only when its board DMA seam is linked. Without it, the public recorder refuses buffer copy, fill, update and DMA-dependent image commands; submission rechecks already recorded work before any backend job or resource reservation. Complete optimal-image uploads and optimal-to-optimal copies keep their independent TFU path. A separate test backend does perform byte writes as an explicit oracle for desk checks; that backend is not the Pi 4 runtime. CPU work is still needed to record commands, validate resources, stage caller data and rasterize source glyphs for the atlas. These are not GPU drawing operations.

`vkCmdCopyImage` accepts an array of one to 32 nonempty rectangles between bound linear BGRA8 images with transfer usage, compatible transfer layouts, mip zero, one layer, and depth one. The source and destination may have different dimensions and offsets. The same image may serve both roles when both layouts are `GENERAL` and no source or destination texels overlap. The complete array is validated before recording; submission rechecks every row span and refuses any source/destination or destination/destination overlap within the call, including aliased allocations, before DMA starts. A tight whole-image region uses one linear DMA transfer; a supported strided subrectangle uses one 2D DMA submission. Larger row gaps retain guarded DMA row copies. A two-region same-image copy passed on Pi 4 silicon with eight guarded DMA rows and exact values for all source, destination and untouched pixels. Separate `vkCmdCopyImage` calls in one otherwise isolated command buffer also execute as ordered DMA transfers; a two-call 4×4 proof passed with eight DMA rows and exact whole-image pixels (`docs/VULKAN_MULTI_CALL_IMAGE_DMA_PI4_2026-09-27.md`). The bounded optimal-to-linear partial copy described below is a separate path; other partial optimal-tiled copies, mips and layers remain unsupported.

One full level-zero optimal BGRA8 image can now be copied to a distinct, equal-size optimal BGRA8 image in a separate transfer command buffer. The source and destination must have compatible transfer layouts and disjoint bound allocations. The Pi 4 backend preflights both padded UIF_NO_XOR ranges and submits one TFU job, then publishes the destination's final layout only after completion. An 8x8 copy of an atlas already uploaded by two ordered TFU jobs produced a third TFU job and the destination was sampled by V3D with exact pixel checks (`docs/VULKAN_OPTIMAL_COPY_TFU_PI4_2026-09-27.md`). A complete equal-size linear-to-optimal image copy also uses the TFU raster input, including the linear image's real row pitch. A fresh optimal destination sampled by V3D passed 402 exact checks after a fourth TFU job (`docs/VULKAN_LINEAR_OPTIMAL_TFU_PI4_2026-09-27.md`). A separate partial linear-to-optimal path is implemented for one

region per command buffer between distinct, equal-size level-zero BGRA8 images. Linear source offsets may be any in-bounds texel coordinates, with the actual source pitch preserved; destination offsets and extent align to 4×4 utiles, up to 256 utiles. The shared layer preflights the stream, and Pi 4 uses guarded DMA behind a dedicated capability. The 2026-09-30 Pi 4 proof copied 16 exact texels, preserved 240 untouched texels and matched all 256 source texels; 64 padding and 96 guard checks passed. TFU stayed at 1, DMA advanced 0→1 and backend jobs 1→2; a later readback advanced DMA 1→17. A misaligned destination returned `-20005` before readback. MMU, OOM and native fault counts were zero, with one intentional validation fault. The complete-image TFU path is unchanged, and other boards do not advertise this partial path; see `docs/evidence/vulkan-partial-linear-optimal-pi4-20260930/README.md`. Supported buffer copies, fills and updates can share a transfer-only command buffer with buffer-to-image and image-to-image operations in recorded order; the whole stream is preflighted before any hardware job. A DMA staging update followed by TFU upload and TFU image copy passed on Pi 4 with the final destination sampled by V3D (`docs/VULKAN_DMA_TFU_STREAM_PI4_2026-09-27.md`). A Pi 4 run uploaded a changed atlas texel then copied the optimal image into a fresh optimal destination through two TFU jobs, with the destination sampled by V3D (`docs/VULKAN_MIXED_IMAGE_TFU_PI4_2026-09-27.md`). A bounded partial optimal-to-linear `vkCmdCopyImage` shape is now implemented: one region between distinct, equal-size level-zero BGRA8 images; source offsets align to 4×4 utiles; width and height are multiples of four unless that dimension reaches its corresponding source edge; destination coordinates remain in bounds; and the bound allocations are disjoint. The shared command layer preflights the full stream before guarded DMA detiles the accepted region on Pi 4. The 2026-09-30 Pi 4 proof copied 25 exact texels from `(32,8)` extent `5×5` in a 37×13 image, preserving 456 other texels and 32 guard words; the 148-byte row pitch had no padding words. DMA advanced 0→4, TFU 0→1 and backend jobs 1→2. A same-sized non-edge tail returned `-20005` before further transfer work. See `docs/evidence/vulkan-partial-optimal-linear-pi4-20260930/README.md`. Other optimal-to-optimal partial regions remain unsupported except for the bounded right/bottom edge-tail shape below; mips, layers, format conversion, and mixing with draws or clears remain unsupported. At most one partial optimal-to-optimal `vkCmdCopyImage` region per command buffer is also implemented for distinct, equal-size level-zero BGRA8 UIF images. Its source/destination offsets and extents align to 4×4 utiles, with at most 256 utiles per region. The shared layer preflights the command stream; Pi 4 uses guarded 64-byte-utile DMA selected by a dedicated capability. The full-image TFU path is unchanged, and the 2026-09-30 Pi 4 proof matched all 16 copied texels, preserved 240 untouched texels, and passed 64 padding and 96 guard checks. TFU stayed at 2, DMA advanced 0→1 and backend jobs 2→3; a later full readback advanced DMA 1→17. An unaligned request returned `-20005` without extra transfer work. See `docs/evidence/vulkan-partial-optimal-copy-pi4-20260930/README.md`. Other boards refuse it. Mips, layers, format conversion, and mixing with draws or clears remain unsupported. Bounded optimal image-to-buffer partial readback is documented separately. Supported linear image readbacks may join the transfer stream.

A returning Pi 4 diagnostic copied a 17 by 13 rectangle between differently sized linear images after a V3D clear, counted thirteen DMA transfers, and checked every copied and untouched destination pixel. The monitor verified the payload, received `x0=0`, and saved a fresh capture. The measured record is in `docs/VULKAN_IMAGE_RECTANGLE_PI4_2026-09-26.md`.

`vkCmdCopyBufferToImage` accepts one to 32 rectangular upload regions into a linear BGRA8 image with transfer-destination usage and a transfer-destination or general layout. Source buffer offsets must satisfy the backend's published 64-byte alignment on Pi 4. `bufferRowLength` is in texels: zero packs the rows tightly, while a nonzero value at least as wide as the rectangle can leave padding between them. For this single 2D layer, `bufferImageHeight` is zero or at least the rectangle height. The whole array is checked before recording and again before submission. Source/destination and destination/destination overlap, including aliased allocations, is refused before DMA starts. Each tight full-width region uses one linear DMA transfer; a supported strided rectangle uses one 2D DMA submission. Larger row gaps retain guarded DMA row copies. Separate linear upload calls in one otherwise isolated command buffer execute in recorded order through guarded DMA; two calls passed on Pi 4 with eight rows and exact 1,024-pixel destination bytes (`docs/VULKAN_MULTI_CALL_UPLOAD_DMA_PI4_2026-09-27.md`). Optimal-tiled destinations keep one complete-image TFU transaction per call and the same 64-byte source alignment. An explicit padded `bufferRowLength` is passed to the TFU raster-stride register; an 8x8 atlas with 16-texel rows was sampled by V3D on Pi 4 and passed exact pixel checks (`docs/VULKAN_OPTIMAL_STRIDE_TFU_PI4_2026-09-27.md`). Multiple complete optimal-image calls in one command buffer are

preflighted together and executed in order by the TFU; two uploads followed by V3D atlas sampling passed 213 exact board checks (`docs/VULKAN_ORDERED_TFU_PI4_2026-09-27.md`). Within the complete-image TFU route, multiple regions in one call and subrectangles remain unsupported. A bounded partial optimal-image upload accepts up to two separate one-region calls in one command buffer, targeting the same optimal BGRA8 image. Buffer offsets are 64-byte aligned; rows may be tight or padded and the source range ends after the last row payload; destination offsets and extents align to 4×4 utiles. Source byte spans and destination rectangles must be disjoint, and the combined tile count is at most 256. The shared layer preflights the full stream before Pi 4 guarded DMA dispatch; the calls execute in recorded order behind `BUFFER_TO_TILED_RECT_COPY` . Desk checks reject a 257-tile aggregate before backend work; an exact 256-tile aggregate has not been separately tested. The 2026-09-30 Pi 4 proof replaced 32 destination texels and preserved 224; it checked 288 source words including 48 source-padding words, 64 readback-padding words and 128 guard words. TFU stayed at 1, DMA advanced 0→2 and jobs 1→3; later readback advanced DMA 2→18. A misaligned second operation returned `-20005` during recording before submit or DMA. See `docs/evidence/vulkan-two-partial-optimal-uploads-pi4-20260930/README.md` . Full-image TFU uploads remain unchanged, and other boards do not advertise this capability. Edge-tail extension: destination XY remains 4-aligned; a width or height not divisible by four is accepted only when that rectangle reaches the matching image right or bottom edge. The backend clamps each 4×4 UIF tile to its valid columns and rows, issuing guarded DMA for only `validColumns * 4` bytes across `validRows` ; this preserves neighboring texels and padded storage. It requires both `BUFFER_TO_TILED_RECT_COPY` and the separate `BUFFER_TO_TILED_TAIL_COPY` capability. Up to two calls remain allowed, with disjoint rectangles and source spans and a cumulative limit of 256 `ceil(width/4) * ceil(height/4)` tiles. Source bounds remain `(height - 1) * pitch + width * 4` , with DMA row-skip bounds checked for the smallest tail row. The complete stream and every tile address/span are preflighted before DMA. Full-image TFU uploads remain unchanged; other boards fail closed. The 2026-09-30 Pi 4 proof changed 25 texels, left 196 valid texels untouched, and preserved 291 padded UIF backing words outside the copy. It checked 246 source words including 54 source-padding words, 91 readback-padding words and 92 guards. TFU stayed at 1, DMA advanced 0→4 and jobs 1→2; later readback advanced DMA 4→24. A non-edge 5×5 request returned `-20005` during recording before more DMA. MMU, OOM and native faults were zero, with one intentional validation fault. Submit-time malformed-stream preflight remains desk-gate evidence. See `docs/evidence/vulkan-optimal-upload-edge-tail-pi4-20260930/README.md` . One-utile micro upload: a separate guarded-DMA shape accepts a 1–4 by 1–4 BGRA8 rectangle wholly contained in one 4×4 UIF utile, including an unaligned interior pixel origin. It writes only the requested row bytes and rows, preserving the other texels in that utile. It requires both `BUFFER_TO_TILED_RECT_COPY` and `BUFFER_TO_TILED_MICRO_COPY` ; a micro upload is isolated from other buffer-to-image operations in its command buffer. Buffer offsets remain 64-byte aligned, source bounds use `(height - 1) * pitch + width * 4` , and source/image alias, row-stride, utile and guarded-window bounds are preflighted before DMA. Other boards do not advertise this capability. The 2026-09-30 Pi 4 proof changed six requested texels and preserved 250, including ten same-utile neighbors; checks covered 272 source words (262 payload and ten padding words), 64 readback-padding words and 96 guards. TFU stayed at 1, DMA advanced 0→1 and jobs 1→2. A crossing-utile request returned `-20005` during recording. MMU, OOM and native fault counts were zero. See `docs/evidence/vulkan-buffer-optimal-microrect-pi4-20260930/README.md` . Submit-time malformed-stream preflight remains desk-gate evidence. A separate two-call Pi 4 proof used a 13×13 target: 17 texels changed, 152 valid texels remained unchanged, and 343 padded UIF backing words outside the copies were preserved. It checked 186 source words including 58 source-padding words, 39 readback-padding words and 112 guards. TFU stayed at 1, DMA advanced 0→2 and jobs 1→3; later readback advanced DMA 2→18. A non-edge second call returned `-20005` during recording before DMA. Submit-time malformed-second preflight is proven by signed desk regression only. See `docs/evidence/vulkan-optimal-upload-edge-tail-pi4-20260930/README.md` (`two-call-report.bin`).

A returning Pi 4 diagnostic uploaded a 17 by 13 rectangle into a differently pitched 32 by 32 image, counted thirteen DMA transfers, and checked all copied and untouched pixels. The monitor verified the payload, received `x0=0` , and saved a fresh capture. The measured record is in `docs/VULKAN_LINEAR_UPLOAD_PI4_2026-09-27.md` . A later two-region upload used one tight and one padded source rectangle. The Pi 4 diagnostic counted eight guarded DMA rows, checked every destination pixel

and untouched byte, returned `x0=0`, and saved fresh capture 52. See `docs/VULKAN_IMAGE_UPLOAD_ARRAY_DMA_PI4_2026-09-27.md`.

The supported image readback

`vkCmdCopyImageToBuffer` accepts one to 32 nonempty rectangular regions from a linear BGRA8 image into a destination buffer with tightly packed or explicitly padded rows. The source must have transfer-source usage and be in `TRANSFER_SRC_OPTIMAL` or `GENERAL`; the destination must be a bound transfer-destination buffer with four-byte-aligned offsets and enough space for every copied row. The whole array is checked before recording and rechecked before submission. Source/destination and destination/destination overlaps, including aliased allocations, are refused before DMA begins. A tight full-width region uses one linear DMA transfer; a supported strided rectangle or padded destination uses one 2D DMA submission. Larger row gaps retain guarded DMA row copies. Separate readback calls and supported buffer/image transfers in one transfer-only command buffer execute in recorded order through guarded DMA or TFU, after whole-stream preflight. Optimal-tiled sources now support a bounded Pi 4 subset: full level-zero BGRA8 readback, including odd full-image extents, and partial regions with source offsets aligned to 4×4 utiles. Width and height must be multiples of four unless that dimension reaches the corresponding image edge. R2 checked 16 exact texels, 16 padding words and 32 guard words with one DMA and refused an unaligned request with `-20005`. R3 read a padded `(32,8)` `5×5` region ending at the right and bottom edges of a 37×13 image; all 25 texels matched, 15 padding words and 32 guard words remained intact, and DMA advanced from zero to four operations. A non-edge `(28,4)` `5×5` request was refused with `-20005` before extra work. Array layers and mixed render/transfer jobs remain unsupported. See `docs/VULKAN_OPTIMAL_IMAGE_BUFFER_DMA_PI4_2026-09-27.md` and `docs/evidence/vulkan-optimal-edge-tail-pi4-20260930/README.md`.

The whole-image readback passed a desk byte check, including all pixels of a 64 by 64 image and the untouched prefix of a destination buffer. A returning Pi 4 diagnostic then cleared a 64 by 64 image on V3D and copied it by DMA. It returned zero after checking every pixel, prefix and suffix guards, one GPU bin/render pair, one DMA operation and restoration of the previous DMA write bound. The monitor verified the payload and saved a fresh screenshot. The exact record is in `docs/VULKAN_IMAGE_READBACK_PI4_2026-09-26.md`. A later returning diagnostic read a nonzero-offset 17 by 13 rectangle using thirteen DMA transfers and checked all copied and untouched bytes; see `docs/VULKAN_IMAGE_RECT_READBACK_PI4_2026-09-27.md`.

A later Pi 4 run used a 23-texel buffer row length for 17-texel upload and readback rectangles. It counted thirteen DMA transfers in each direction and checked the row padding, outer guards, and every pixel. The measured record is in `docs/VULKAN_STRIDED_IMAGE_PI4_2026-09-27.md`.

A two-region image-to-buffer readback passed on Pi 4 silicon with eight guarded DMA rows. The second region used padded destination rows, and the returning diagnostic checked every destination byte, DMA quiescence and restoration of the original DMA bounds. The monitor verified the payload, returned `x0=0` and saved fresh capture 48; see `docs/VULKAN_IMAGE_READBACK_ARRAY_DMA_PI4_2026-09-27.md`. Two separate readback calls later passed with eight guarded DMA rows, an exact 16,512-byte destination check and fresh capture 54; see `docs/VULKAN_MULTI_CALL_READBACK_DMA_PI4_2026-09-27.md`. A further Pi 4 proof uploaded a distinct 4x4 pattern to a linear image, placed an image transfer barrier, then read it into a different buffer span in the same command buffer. Two DMA operations completed, all 64 result and image bytes matched, and capture 71 was saved (`docs/VULKAN_MIXED_READBACK_DMA_PI4_2026-09-27.md`).

A returning 12-frame pyramid payload now shows the Vulkan draw path on the Pi 4 panel. V3D shades and rasterizes its four faces each frame, while display DMA presents the completed attachment. The CPU currently projects five 3D points and sorts the faces because this bounded Vulkan vertex path takes 2D positions and has no depth attachment. The payload returned x0=0 after 12 GPU draws and 12 DMA presentations; fresh capture 74 shows the last frame (`docs/VULKAN_PYRAMID_DEMO_PI4_2026-09-27.md`).

Build and proof boundary

The normal Pi 4 image compiles with the IDE compiler's `--compile` path and links the board DMA seam. The emitted V3D backend gate checks its source contract, links the production backend, and runs an engine-down no-MMIO fixture. Those desk checks alone do not prove a transfer on silicon; the returning readback diagnostic above supplies that proof for its exact shape. The detailed implemented and refused Vulkan calls are in `docs/VULKAN_COMPATIBILITY_STATUS.md` and `Anvil/Graphics/Vulkan/COVERAGE.md`. Anvil is not a conformant Vulkan 1.0 implementation.

The current Pi 4 image-transfer diagnostic passed all exact pixel, padding, DMA-quiescence and write-bound checks with one 2D DMA submission per supported rectangle, 19 submissions across the complete suite instead of 122 in its earlier row-by-row form (`docs/VULKAN_2D_DMA_PI4_2026-09-27.md`).

One-utile optimal-to-optimal microcopy

A separate Pi 4 capability copies one 1–4 by 1–4 BGRA8 rectangle between distinct equal-size level-zero optimal images. Source and destination offsets are independent; each rectangle must fit within one 4×4 UIF utile, and each image's other texels are preserved. The route requires both `TILED_RECT_COPY` and dedicated `TILED_MICRO_COPY` ; it is isolated as the sole copy/transfer/draw operation in its command buffer. Shared recording and submit validation recheck capability, resource/layout generations, extents and allocation aliasing before dispatch. Pi 4 lowers only the requested row bytes with guarded DMA at a 16-byte row stride; it does not use CPU pixels or TFU. Other backends fail closed. The 2026-09-30 proof changed four destination texels and preserved 252 (including twelve same-utile neighbors); all 256 source texels remained unchanged. Each readback checked 64 padding words and 128 guards. TFU stayed at 2, DMA advanced 0→1 and jobs 2→3; a source utile-crossing request returned `-20005` during recording. MMU, OOM and native fault counts were zero. See `docs/evidence/vulkan-optimal-optimal-microrect-pi4-20260930/README.md` . Submit-time malformed-stream preflight remains desk-gate evidence.

One-utile optimal-image buffer readback

A separate Pi 4 DMA capability supports one interior `vkCmdCopyImageToBuffer` region of 1–4 by 1–4 BGRA8 texels, wholly inside one source 4×4 UIF utile. Destination offset remains four-byte aligned; tight or padded rows are checked against the exact last-row span `(height - 1) * pitch + width * 4` . The shared API requires both ordinary linear-transfer support and dedicated `TILED_MICRO_READBACK` ; it rejects image/buffer aliasing and verifies capability, resource lifetime, bounds and destination stride before any DMA. The Pi 4 backend reads from the source's independent local UIF offset with guarded DMA; other backends fail closed. The 2026-10-01 proof matched four requested texel words, passed twelve row-padding checks, left all 256 source texels unchanged (including twelve same-utile neighbors), and passed 64 full-readback-padding, 16 stage-spare and 128 guard checks. TFU stayed at 1, DMA advanced 0→1 and jobs 1→2. A crossing-utile request returned `-20005` during recording; MMU, OOM and native faults were zero. See `docs/evidence/vulkan-optimal-buffer-microreadback-pi4-20261001/README.md` . Submit-time malformed-stream preflight remains desk-gate evidence. Other backends do not advertise this capability.

One-utile optimal-to-linear microcopy

A separate Pi 4 DMA capability copies one interior 1–4 by 1–4 BGRA8 rectangle from an optimal image into a distinct, equal-size linear image. The rectangle must fit in one 4×4 UIF utile. It requires `LINEAR_TRANSFER` and `TILED_TO_LINEAR_MICRO_COPY` ; the backend checks the linear destination pitch and exact written span, then DMA-copies only the requested rows. Other backends refuse this shape. The 2026-10-01 Pi 4 proof changed four destination texels and preserved 252; all 256 source texels remained unchanged, including twelve neighbors in the same utile. The 64-byte destination pitch had zero row padding. Separate oracle and guard checks covered 64 padded words and 96 guards. DMA advanced 0→1, jobs 1→2, and TFU remained at 1. A crossing-utile request returned `-20005` while recording. Submit-time

whole-stream preflight is desk-gate evidence only. See `docs/evidence/vulkan-optimal-linear-microcopy-pi4-20261001/README.md`.

One-utile linear-to-optimal microcopy

A separate Pi 4 DMA capability copies one interior 1–4 by 1–4 BGRA8 rectangle from a linear image to a distinct, equal-size optimal image. The rectangle must fit in one destination 4×4 UIF utile. It requires `LINEAR_TRANSFER`, `LINEAR_TO_TILED_RECT_COPY` and `LINEAR_TO_TILED_MICRO_COPY`; the actual linear pitch, exact source span and destination utile window are checked before DMA. Other backends refuse this shape. The 2026-10-01 Pi 4 proof changed four destination texels and preserved 252, including twelve same-utile neighbors, while all 256 source texels remained unchanged. Source pitch was 64 bytes with zero row padding; offset/span were 332/72 bytes and the UIF destination offset was 1,316 bytes. Separate oracle and guard checks covered 64 padded words and 96 guards. TFU stayed at 1, DMA advanced 0→1, and jobs 1→2. A crossing-utile request returned `-20005` while recording, before DMA. Submit-time whole-stream preflight remains desk-gate evidence only. See `docs/evidence/vulkan-linear-optimal-microcopy-pi4-20261001/README.md`.

Two-by-two UIF grid buffer upload

A separate Pi 4 DMA capability accepts one interior buffer-to-optimal region that intersects up to two adjacent 4×4 UIF tile columns and two rows. It requires `BUFFER_TRANSFER`, `LINEAR_TRANSFER`, `BUFFER_TO_TILED_RECT_COPY` and `BUFFER_TO_TILED_GRID_COPY`; a 64-byte source offset, tight or padded row pitch and exact source span are validated. All tile source spans and destination windows are checked before any DMA. Other backends refuse this shape; the full-image TFU upload is unchanged. The 2026-10-01 Pi 4 proof copied a `5×5` rectangle at `(3,3)` through intersections `1×1`, `4×1`, `1×4`, `4×4`; it changed 25 texels and preserved 231, including 39 neighbors within the touched grid. Source checks covered 25 exact words plus 327 sentinels (including 15 padding words); separate oracle checks covered 64 padding words and 96 guards. TFU remained at 1, DMA advanced 0→4, jobs 1→2, and later readback advanced DMA 4→20. The recorded `6×6` refusal was tested with only the original ≤2×2 grid cap, before the extended cap existed. With `BUFFER_TO_TILED_GRID_4X4_COPY` enabled, an otherwise-valid `6×6` region can be admitted; that exact size has not been silicon-tested. Submit-time malformed-stream preflight is desk-gate evidence only. See `docs/evidence/vulkan-buffer-optimal-grid-pi4-20261001/README.md`.

Extended optimal-image buffer upload grid

A separate Pi 4 DMA capability handles a single `vkCmdCopyBufferToImage` region that crosses three or four adjacent UIF columns or rows, up to 16 intersections. It has a dedicated `BUFFER_TO_TILED_GRID_4X4_COPY` cap/tag; the earlier `BUFFER_TO_TILED_GRID_COPY` remains limited to at most two columns and rows. The buffer offset remains 64-byte aligned, with tight or padded pitch and exact source span. The whole tile grid is validated before any guarded DMA. Other backends do not advertise the extended shape. The 2026-10-01 proof copied a `(3,3)` `9×9` region using buffer offset 64, row length 12, 48-byte pitch and 420-byte span. Intersections were `1×1`, `4×1`, `4×1`, `1×4`, `4×4`, `4×4`, `1×4`, `4×4`, `4×4`; 81 texels changed and 175 remained unchanged, including 63 inside the touched 12×12 grid. Source checks covered 81 exact words plus 271 sentinels (27 padding words), 64 oracle-padding words and 96 guards. TFU stayed at 1, DMA advanced 0→9, jobs 1→2, and later readback advanced DMA 9→25. All reported statuses and fault counts were zero. The fifth-row/column refusal and submit-time malformed-stream preflight are desk-only checks. See `docs/evidence/vulkan-buffer-optimal-grid4-pi4-20261001/README.md`.

Bounded partial optimal-to-optimal edge-tail copy

The shared `vkCmdCopyImage` API supports one partial region per command buffer between distinct, equal-size level-zero BGRA8 `UIF_NO_XOR` images. Source and destination XY offsets remain 4-texel aligned. Width or height may end with a short UIF tile only when that dimension reaches the right or bottom edge in both images; the ceil-divided tile count is capped at 256. This shape requires `TILED_RECT_COPY` plus the separate `TILED_EDGE_TAIL_COPY` capability. Whole-image copies retain their TFU path, aligned full-utile rectangles retain the rectangle path, and one-utile microcopies retain their separate path. The

Pi 4 backend preflights all clamped tile windows and performs guarded DMA for valid texel rows only, preserving source data, untouched destination texels and padded UIF storage. Other backends fail closed.

The 2026-10-01 Pi 4 proof copied a `(8,8)` `5×5` region between distinct 13×13 images: 25 destination texels changed, 144 stayed unchanged, and all 169 source texels were unchanged. The report checked 338 exact staging words plus 78 staging-padding words, 91 readback-padding words in each full readback, and 160 guard checks. TFU stayed at 2, partial DMA advanced 0→4 and jobs 2→3; the oracle readbacks advanced DMA 4→20→36. A non-edge `(4,4)` `5×5` request returned `-20005` during `vkEndCommandBuffer` before further DMA. Status slots were zero; MMU, OOM and native faults were zero, with one intentional validation fault. The measured trace is `docs/evidence/vulkan-optimal-optimal-edge-tail-pi4-20261001/report.bin`; the PMF SHA-256 is `8594C760D6E3206CFDC5DEAE41DBAAF6C0A770EDDA5FB5A369EBC3AA40ED027F`. Right-only `5×4` and bottom-only `4×5` cases have desk validation but no separate silicon proof.

Transactional Neon atlas generations

Bitmap-font generation changes now prepare a new atlas image, view and descriptor and publish them only after the upload completes. A capacity refusal preserves the old handles, generation and texels. If completion is uncertain, both generations stay allocated and the owner stops accepting draws until device-idle retirement. Ordinary TrueType dirty-region uploads keep their existing partial/full path.

On Pi 4, memory-table exhaustion preserves every byte of the 2 MiB resident atlas. Three later font swaps keep allocation counts stable and produce 49,152 exact text/sprite pixel bytes. A diagnostic-only injected failure checks retention followed by real device-idle teardown without hanging the GPU. All 245,760 current scene bytes and 124 widget checks remain exact, and the established partial-upload GPU oracle passes. Historical line scenes already differ from the old native golden under the 14 October pixel-placement contract; the golden was not replaced. See docs/evidence/neon-atlas-generation-pi4-20261025/README.md for the proof boundary.

The counted full monitor build 329 compiles but is not installed. Its BSS extends beyond the old variable-reservation window tracked in topic 1249; that deployment prerequisite remains open. RAM proofs do not establish a new monitor installation, the entire Neon port, or Vulkan 1.0 conformance.

Combined Neon draw records

The shared Vulkan recording pool, private V3D slots and configurable Chrome draw ceiling hold 6,144 records, the sum of native Neon's 4,096 flat and 2,048 textured records. A smaller ceiling requested by a caller remains enforced. The Pi 4 proof renders that mixed draw count, checks refusal without cursor or guard changes, frees all records, and checks a recreated two-draw owner. All 32,768 observed GPU pixel bytes match the independent oracle. Separate family vertex accounting and maximum vertex workloads remain open.

The final diagnostic returns in 21.5 seconds with a 15-second watchdog fed only at bounded forward-progress checkpoints. The initial whole-payload 15-second trial failed with a watchdog restart; it is not a passing result. No firmware was installed. Evidence and repeatable tools are documented in docs/evidence/neon-combined-draws-pi4-20261025/README.md.

Every command

This is the monitor's own `help`, as each board prints it, so it can never say less than the board does.

Every command the monitor accepts is listed here with the text its `help` prints. The chapters before this one explain the commands people use most and show what they print; this chapter is the complete list. It is generated from the help text itself by `tools/guide_check.py --write`, and the same tool fails a commit that changes the help without regenerating it. Lines in angle brackets are worked out by the board when it prints them, and a line saying "only on a board with" marks text a board prints only when it has that capability.

Every board

```
Every command is a whole word. The old single letters all still work
and are shown in brackets after each one, because the host tools type
them and because U-Boot's own loads and go land here too.
Every command prints something, on every path, whether it worked or
not. If one ever returns to the prompt in silence, that is a fault.

GETTING CODE IN
  receive <address> <length> <checksum>  [b]
                  the fast way in. All three numbers are hexadecimal.
                  The monitor answers rdy, the host sends that many
                  raw bytes in chunks of 1024, and one + comes back
                  per chunk. The last line is ok or crc. The checksum
                  is the IEEE reflected CRC32, polynomial EDB88320.
  srecord                               [l, loads]
                  the slow way in, and the proven one. S3 records
                  carry the data and an S7 record ends the stream. A
                  record that never arrives cannot be detected, since
                  each checksum covers only its own record, so this
                  is the one to fall back to rather than the one to
                  reach for. One bad record blocks run until an image
                  loads cleanly.
  load <name> [address]                 [f, fatload]
                  read a file off the boot medium into memory. What
                  that medium IS depends on the board - type f on
                  its own and it says, partition by partition. A path
                  may start N: for partition N - 2:/folder/file - and
                  with no N: it is partition 1. A path may instead
                  start data: - data:/models/MODEL.PMW - which means
                  under the data.path setting: one partition, leave it
                  unset (partition 1's root); two partitions,
                  settings set data.path 2:/ and settings save.
                  Quote a path with spaces; case does not matter.
                  Without an
                  address it loads at <address>. The range is checked
                  against the payload windows, and the length is the
                  file's, not yours - which is why an address that
                  looks fine can still be refused.
  mount                                 [f]
                  bring the boot medium up and report what it is,
                  loading nothing. This is the command to type when
```

```
                  load has failed, because it names the step that
                  went wrong.
  fatls [path]                          [ls]
                  list a directory: each entry's name and its size in
                  bytes, a <DIR> marker for a subdirectory, and a
                  summary line. With no path it lists the root of
                  partition 1; ls 2:/ lists partition 2, and
                  ls 2:/folder a folder on it. FAT32 and exFAT
                  partitions list any folder by long name.
                  Any keystroke stops a long listing, which then says so.
  save <name> <address> [length]
                  write that many bytes out of memory onto the boot
                  medium, replacing that file and resizing it to
                  match. The length is hexadecimal; leave it out and
                  the file keeps the size it already has. A file that
                  is not on the medium yet is created, but only when a
                  length is given, because a file that does not exist
                  has no size of its own to keep. The path takes N:
                  like load; its folder must already exist. Writing
                  is available when this board reports a writable
                  boot medium. This is how Anvil replaces files on
                  its boot filesystem, including a staged image.
  mkdir <path>
                  make a directory. Its parent must already exist.
  rm <path>                             [del]
                  remove one file, or one EMPTY directory. Nothing is
                  ever removed recursively.
  rmdir <path>
                  remove one EMPTY directory, and refuse a file.
  rename <old path> <new path>          [mv]
                  rename or move an entry within one partition. The
                  new name must not exist yet, except as the same
                  entry in a different case.

RUNNING IT
  run [address]                         [g, go]
                  jump to a payload and start it. Without an address
                  it uses the entry point of the last thing loaded,
                  or <address> if nothing has been loaded. If the
                  payload ever comes back, its x0 register is
                  printed. A stock image never comes back.
  booti [addr [initrd [fdt]]]
                  boot an arm64 Linux Image. Checks the Image magic,
                  keeps the firmware's device tree (or the fdt you give,
                  after a dash for the skipped initrd: booti addr - fdt),
                  sets x0 to it, drops from EL2 to EL1 and branches. It
                  refuses loudly rather than jump into anything that is
                  not a kernel. Does not return.
  bootm [addr [initrd [fdt]]]
                  boot a container. A legacy uImage holding a single
                  uncompressed arm64 Linux kernel is booted; a FIT
                  (.itb) is refused (no device-tree parser here); a bare
                  Image is sent to booti. Anything else is refused by name.
  boot <name>
                  THE BOOTLOADER. Read a payload container off the
```

```
                boot medium, place its image at the address the
                payload was BUILT for, check its SHA-256, and only
                then enter it. No address is typed anywhere: the
                container carries the load address, the entry
                point, the zero-init extent and the hash, because
                the compiler knew all four and writing them down
                beats retyping them. The container is the .pmf file
                the compiler writes beside the flat image.
                It REFUSES, loudly and having done nothing, on a
                bad magic, a version it cannot read, a length that
                disagrees with the file, a load or zero-init range
                that runs over the monitor or outside the payload
                windows, an entry outside its own image, and above
                all on a hash that does not match. The hash check
                is mandatory and it is the last thing before the
                jump.
boot mem <address> [shot]
                the same, for a container already in memory - one
                that arrived over receive or over the air, with no
                medium involved at any point. The word shot keeps
                the frame the payload leaves on the screen, before
                this monitor repaints over it; read it back with
                the shot command.
boot status
                what this board would do at its next start, and
                the four settings behind it. Bare boot is this.
boot ok
                clear the failure counter: the payload came up.
                Say it when the payload is actually working -
                saying it every boot regardless turns the
                reboot-loop guard off.
boot clear
                stop booting a file at start-up. The prompt only.
autoboot [address]                    [a]
                the QUICK path, and a different thing from boot.
                It takes a bare ADDRESS in DRAM, checks nothing
                about what is there, and lives in a memory cell
                that survives a reset but NOT a power cycle.
                autoboot <address> arms it, autoboot 0 clears it,
                and autoboot on its own says what it is set to. An
                armed monitor waits about two seconds for a key
                when it starts and then runs what it points at.
                It is taken BEFORE boot.file, because somebody
                typed it in this power cycle and boot.file is
                standing configuration.
deadman [seconds | off]               [d]
                a watchdog that covers the NEXT payload and
                nothing else. If the payload hangs, the board
                resets itself instead of sitting there. It is
                never running while you are at this prompt. The
                seconds are decimal. The counter behind it is 20
                bits at 65536 ticks a second, so a fraction under
                16 seconds is the ceiling; anything larger is cut
                down to 15 rather than 16, because a deadman that
                fires sooner than it promised is a trap.
```

```
  last run
                  what happened to the last payload entered: it
                  returned and what x0 held, it stopped at a
                  processor exception, or the board restarted under
                  it with the deadman armed. Kept where a reset
                  cannot erase it; the first line is for tools.

LOOKING AROUND
  map
                  the whole memory map, as this image measures it:
                  what this monitor occupies and the byte count that
                  came from, every region it refuses writes to, its
                  stack top, the address to stage a file at, and both
                  payload windows. NOTHING IN IT IS A CONSTANT. This
                  program may be any size, so its extent is the size
                  of the image the firmware loaded and the low window
                  begins above it - both move when this program does.
                  The upload tools ask for this rather than assuming.
  memory <address> <count>              [m, dump, md]
                  print a block of memory as hexadecimal, up to 4096
                  bytes at a time, with 64 as the default. Both
                  numbers are hex and the count is in bytes. The
                  address is not WINDOW-checked - reading is the whole
                  point of a debugger, and being able to point it at
                  a device register block is most of its value - but
                  the BOARD is asked about it first. See force below.
                  md.b, md.w and md.l issue true 8, 16 or 32-bit reads
                  and group each line in little-endian values. The count
                  is still bytes whichever you pick. .w/.l require a
                  2-/4-byte-aligned start and a byte count containing
                  whole units; invalid ranges read nothing. A bare md
                  reads bytes, as it always did.
                  It ends by printing the byte total it dumped and the
                  grouping it used, like the rest of this family.
  readback <address> <length>
                  the same bytes, for a MACHINE instead of a person.
                  Both hex, no length limit and no default. It prints
                  a header line, the range as base64 at 48 bytes a
                  line, then the byte count and a crc32 over it:
                    readback 57400000 1812000 bytes base64
                    <64 base64 characters>  ... and 37,750 more ...
                    readback end 1812000 bytes crc32 A1B2C3D4
                  A host reads until that last line rather than waiting
                  for silence, and checks both numbers - the crc32 uses
                  the same polynomial as crc32 and as the host tools, so
                  the two answers are independent. THE PAYLOAD GOES TO
                  WHOEVER TYPED IT and to nobody else: over the network
                  console it goes to that peer at the link's own speed,
                  never through the serial port, which is what makes it
                  hundreds of times faster than reading a hex dump back.
                  The other consoles still see this command and its last
                  line - the parts a person can read. Ctrl-C stops it and
                  the closing line then says stopped, with the count and
                  checksum of what really went. base is not added.
  write <address> <value>               [w, poke]
```

```
                write one 32-bit word and read it straight back.
                Both hex. The address must be a multiple of four,
                and an address inside the monitor is refused.
compare <first> <second> <count>      [cmp]
                compare two blocks and name the first byte that
                differs, and its address on both sides. Hex. A bare
                compare counts BYTES (U-Boot's cmp counts words);
                compare.b/.w/.l count bytes, halfwords or words.
copy <source> <destination> <count>   [cp]
                copy a block. Hex, overlapping ranges are safe, and
                the destination's caches are flushed so it can be
                run. A bare copy counts BYTES; copy.b/.w/.l count
                bytes, halfwords or words.
crc32 <address> <count>               [checksum]
                the CRC32 of a block. No size limit - a whole boot
                image is the point of it. This is how you check an
                image before or after save writes it: tools/anvil.py
                computes the same value on the host from the same
                polynomial, so the two agree independently or they
                do not. Any keystroke stops it and a stopped one
                prints no checksum at all.
sha1sum <address> <count>
sha256sum <address> <count>
                a cryptographic checksum of a block, lowercase hex, the
                same as the sha1sum / sha256sum tools on any machine -
                so a published image's checksum can be checked against
                what is actually in memory. Like crc32: no size limit,
                and a stopped one prints no digest.
crypto [vectors | speed [kb] | list | <module>]
                the known-answer self-test for every crypto module,
                run HERE, on the same vectors the desktop gates use -
                RFC 3174, 2202, 4231, 5869, 6070, 3394, FIPS-197, the
                NIST CAVP files and the NIST/McGrew GCM table. One line
                per vector with its CNTPCT_EL0 ticks; every GCM vector
                is decrypted again with one bit of its tag flipped and
                the forgery must be REFUSED. crypto speed adds one
                throughput line per module - all eleven, over a 64 KB
                buffer - which is the one number a desktop model cannot
                produce. Two of the eleven cannot be given 64 KB and each
                says under its own rate what it ran and what capped it.
                Not one vector is typed into the image; they are
                generated from the documents. Any keystroke stops it,
                and a stopped sweep prints no verdict.
fill <address> <value> [count]        [mw]
                set a range to one value. A bare fill writes 32-bit
                WORDS and count defaults to 1, which is what U-Boot's
                mw does; fill.b/.w/.l write and count bytes, halfwords
                or words. The byte count is always printed.
mm <address>                          modify memory, interactive
nm <address>                          the same, address held
                a sub-prompt shows one address and its value at a
                time. Type a hex value to write it, Enter to keep it,
                . or q or Ctrl-C to quit - the way out is on every
                line. mm steps to the next location, nm stays put.
                mm.b/.w/.l and nm.b/.w/.l set the width; bare is a
```

```
                word. Writes into the monitor are refused.
mtest <start> <end> [pattern [iters]] DESTRUCTIVE RAM test
                writes and reads back every word in start..end, so
                the range is OVERWRITTEN and NOT restored. No default
                range and it must lie inside a payload window - it
                refuses the monitor and everything else you do not
                own. Address, moving-inversion and walking one/zero
                tests; iters defaults to 1 (it does not loop forever)
                and any key stops it. A true DRAM test only with the
                D-cache off, which is the boot default.
base [address]
                a default address added to the address arguments of
                memory, write, fill, copy and compare, so repeated
                calls stop needing one. Starts at zero. It is NOT
                added by crc32 - which matches U-Boot - nor by
                readback, which is in the same family, and every
                command that does add it says so on the line.
                base 0 turns it off.
force <address>                       override one refused address
                EVERY command above that takes an address from you
                asks the board first whether that address is safe to
                touch, and refuses if it is not. This is what lets
                you go ahead anyway.
                Why it exists: on one of the boards Anvil runs on, a
                read of a block whose clock is off does not fail -
                the processor stops on it and the board has to be
                unplugged. A single memory command has done that
                three times. The board keeps a list of the blocks it
                knows about, and a refusal names the block and says
                whether the answer is measured or merely unknown.
                How it works: type force and the address, then
                repeat the refused command on the very NEXT line.
                It arms exactly that one address, for exactly one
                command line, and then expires whether it was used
                or not. force on its own says what is armed; force
                off disarms early.
                Why two lines and not a flag on the command: a flag
                is something a script can carry without anybody
                having decided, and deciding is the entire point.
                Writing the address twice, under two command words,
                cannot happen by accident.
                Type info to see what this board knows and where
                each answer came from.
Coming from U-Boot: md is memory, mw is fill, cmp is compare and cp
is copy, and all four of those U-Boot spellings are accepted here, so
a line copied out of a U-Boot session still runs. mm, nm and mtest
keep their U-Boot names.
<the .b/.w/.l width rule, worked out from the width table at run time>
loop is deliberately absent - it cannot return at all.
info                                  [i, board]
uptime                                [up]
                what board this is, how hot it is, and every clock
                rate the firmware will tell us about.
messages        what background services reported - a lease, the
                radio rekeying or recovering, the network console
```

```
                  arming on a new address. They are never printed on
                  the command line; this is where they are read, and
                  a screen shows the newest one on the banner until
                  this has been run.
  clock [rate]                          [c]
                  show or set the processor clock. The rate is in
                  hertz and it is DECIMAL - the one decimal argument
                  in this monitor. It prints what the firmware
                  applied and then what is actually measured,
                  separately, because the firmware clamps a rate it
                  dislikes and still reports success.
  time [status]
                  live wall time, zone, provenance and synchronization
                  state. An unset clock says so; a restored disk value
                  is identified as a floor, not trusted current time.
  time set <unix-seconds-UTC>
                  set trusted manual UTC time from decimal Unix seconds.
  time zone <UTC | America/Chicago | fixed:+HH:MM | fixed:-HH:MM>
                  choose display civil time. America/Chicago applies
                  CST/CDT automatically; a fixed offset never does.
(only on a board with NET:)
  time sync       schedule a nonblocking SNTP refresh now.
  time server <IPv4-or-hostname | default>
                  choose SNTP explicitly, or restore DHCP option 42 then
                  time.cloudflare.com fallback. A server never chooses
                  the zone. Plain SNTP is not authenticated.
                  Unknown zones/servers are refused. Time changes stay
                  in memory until settings save. Default display is UTC.

FROM U-BOOT, AND HARDWARE-FREE
  echo [-n] <args>
                  print the arguments, separated by single spaces, then
                  a newline. -n as the first word drops the newline. No
                  variable expansion - this monitor has no shell, so
                  echo $x prints those three characters.
  sleep <seconds>
                  wait that many seconds and come back. Decimal seconds
                  - a leading 0x or $ is hex, like clock. Any key ends
                  the wait early, because a wait that could not be
                  broken would be a hang. The ceiling is a day.
  version [full]                        [ver]
                  what this monitor is, WHICH build it is, and which
                  target it was built for. The board name, the
                  processor, the exception level and the target all
                  come from the board, so this describes THIS computer
                  and not some other one.
                  It prints the build number with its date and time,
                  and this image's own size and extent - those two
                  together already tell two builds apart.
                  version full adds a crc32 over the running image:
                  the digest that says WHAT is in it rather than which
                  build it is. It is a separate word because it reads
                  every byte and is not instant, and bare version
                  prints the exact crc32 command instead.
                  Unlike U-Boot it does not print a compiler or
```

```
                  linker version, because neither is stamped into the
                  image and a guessed one is worse than none.
  setexpr <name> <val1> [<op> <val2>]
                  work out an expression and store it in the settings
                  store as hex. op is one of + - * / % & | ^ << >>, both
                  values hex. setexpr addr 400000 + 1000 puts 401000 in
                  the setting called addr, ready for memory or run to use.
                  The *addr, fmt and gsub forms are not here yet.

THE SCREEN
  screen [on [dsi | hdmi | number] | off | displays
          | rotate [0|90|180|270]]              [s]
                  the console on a screen. Everything the serial
                  line gets, the screen gets; the wire stays the
                  complete record and the screen is the
                  convenience.
                  THERE ARE TWO KINDS OF SCREEN ON THIS BOARD and
                  they are not the same thing. An HDMI display is
                  the firmware's: it must be connected BEFORE the
                  board powers on, because the firmware only looks
                  at boot. The Waveshare DSI panel is THIS
                  monitor's - the firmware owns no DSI display and
                  never brings DSI1 up, so this monitor lights the
                  panel itself out of its own memory.
                  AT BOOT, WITH NOTHING TYPED: a DSI panel that
                  answers on the display connector wins, because
                  somebody physically plugged it in; otherwise an
                  HDMI display if the firmware found one;
                  otherwise no screen, which is not a failure. The
                  boot log says which.
                  screen on its own says whether it is on, and
                  turns it on if it is off. screen on dsi and
                  screen on hdmi move it between the two.
                  screen displays lists BOTH kinds truthfully -
                  the firmware's census and the panel separately,
                  because the panel is not one of the firmware's
                  displays and reporting it as one would be an
                  invention. This board has two HDMI sockets;
                  screen on 1 brings the console up on the second
                  display in that list. The number is a position
                  in the list, NOT the number on the socket: with
                  one cable, in either socket, it is number 0.
                  Plain screen on lets the firmware choose among
                  its own, which is what it has always done.
  screen rotate [0|90|180|270]
                  which way up the console sits ON THE DSI PANEL.
                  The panel SCANS 800 across by 1280 down and no
                  setting changes that, but it is mounted long
                  edge across, so THE DEFAULT IS 90 - landscape,
                  1280 by 800, 160 columns by 42 lines. IF THE
                  PICTURE IS UPSIDE DOWN, type
                  screen rotate 270 once: which of the two is the
                  top depends on how the flat cable leaves the
                  board, the setting is SAVED, and the panel comes
                  up that way afterwards with nothing typed.
```

```
                  Nobody rebuilds the monitor to turn a screen.
                  THE COST, said every time you ask: this hardware
                  has no 90 degree rotation - the video scaler can
                  mirror but not turn - so 90 and 270 draw into a
                  buffer of their own and transpose every repaint
                  onto the panel, and a scroll becomes a full
                  repaint. 0 and 180 move no pixel and keep the
                  hardware scroll, and 0 is 100 columns by 72
                  lines of THE SAME SIZE type, standing on its
                  side - taller and narrower, not smaller. The
                  letters are the same physical size at every
                  rotation; see screen scale for why.
                  Rotation is the panel's: HDMI is landscape as
                  the firmware hands it over and is not turned.
  screen scale [n|auto]
                  how big the type is. IT IS NORMALLY COMPUTED:
                  this monitor works out how many pixels there
                  are to the inch on whatever screen it is
                  drawing on - the diagonal in pixels over the
                  diagonal in inches - and magnifies its 8 by 16
                  font by the whole number nearest 0.110 inch to
                  a line, which is what reads at a bench. Both of
                  those are facts about the GLASS and neither
                  changes when the picture is turned, which is
                  why turning the console no longer changes the
                  size of a letter. The DSI panel reports its own
                  size over I2C - 10.1 inches, 149 pixels to the
                  inch, magnification 1. NOTHING ON AN HDMI
                  DISPLAY CAN SAY HOW BIG IT IS, so 24 inches is
                  assumed and every report that uses it says so.
                  screen scale <n> overrides all of that if it
                  does not suit your eyes or your distance from
                  the screen, and it is SAVED; screen scale auto
                  forgets it and goes back to the calculation.
                  The boot log and screen print the density, the
                  magnification, the columns and rows, and the
                  height of a line in inches.
(only on a board with TOUCH:)
  screen keyboard [show|hide|state]
                  put the touch keyboard on the panel, take it
                  away, or describe it - from ANY console channel,
                  including this one. It goes through the same
                  path a finger on the prompt line takes, so what
                  it opens is the keyboard, not a picture of one:
                  the console gives up its bottom rows without
                  losing a character of what is on them, and the
                  keys type into the same line this wire does.
                  The key labels are set in the anti-aliased face
                  the banner's title uses, on both renderers.
  screenshot                          [p]
                  send a picture of the screen back over this wire.
                  It is run-length encoded, because the raw picture
                  would take twenty minutes at 115200 baud, and
                  tools/pmfshot.py on the host turns it into a PNG.
                  It takes several minutes. Only Ctrl-C stops it, so
```

```
                  that a stray keypress cannot throw the capture
                  away, and a stopped one is marked as stopped so
                  the host never writes half a picture.
  shot
                  send the KEPT picture - the frame a payload left
                  on the screen, copied out of the way before this
                  monitor repainted over it. Same format as
                  screenshot, so the same host tool decodes it, with
                  one machine line of header in front of it.
  shot arm
                  keep the frame the NEXT payload leaves behind,
                  whether or not it asks. Armed for one run.
  shot now        keep the screen as it is at this instant
  shot status     whether a capture is armed, and what is kept

OVER THE NETWORK
  This board has a gigabit Ethernet socket and this is what it is for:
  a 260 KB image takes twenty-four seconds down the serial line and well
  under a second over the wire. It speaks TFTP, which is what every
  board has been fetched over since 1981 and what U-Boot uses, so any
  machine that can serve a file to a router can serve one to this.
  get <name> [address]
                  fetch a file from the TFTP server into memory. The
                  name is the file as the server knows it; the address
                  is hex and defaults to <address>, the same place load
                  uses. The destination must be inside a payload window,
                  and the transfer stops at that window's end rather
                  than writing past it - a TFTP client does not know how
                  long a file is until it ends, so the room is checked
                  instead of the length.
  put <name> <address> <length>
                  send that many bytes of memory to the server as a
                  file. Both numbers hex, the length in bytes. Most
                  servers refuse to be written to until they are told to
                  allow it, and the server's own refusal is printed here
                  word for word when that happens.
  net             the four addresses and the state of the wire. This is
                  the command to type when get or put has failed,
                  because it names the step that is missing.
                  ONLY net server IS NEEDED FOR A TRANSFER. The other
                  three are the SAVED-ADDRESS source, one of the three
                  ways an interface can come to hold an address; get
                  and put read this board's own address, netmask and
                  gateway off whichever interface can reach the
                  server, whether that address came from a lease, from
                  these settings, or from the board probing one for
                  itself on a bare cable.
  net address <a.b.c.d>      the address this board answers to
  net netmask <a.b.c.d>      also spelled net mask
  net gateway <a.b.c.d>      also spelled net router. Use 0.0.0.0 when
                             there is no router, which is the truth on a
                             cable straight between two machines.
  net server <a.b.c.d>       the machine serving the files
  net dns <a.b.c.d>          the DNS resolver, for dns and ping by name
  net up          bring the Ethernet up now and report every step. get
```

```
                and put do it themselves; this is the diagnostic.
net down        stop it. It is also stopped automatically before every
                run, because a payload must not share memory with a
                controller that is still writing into it.
net forget      take all four addresses back out
net stats       WHAT THE NETWORK CONTROLLER ITSELF COUNTED on the
                interface now in use - pause frames sent and received,
                framing errors, dropped frames, and how many times the
                link has been brought up or seen to drop. A counter
                that reads zero is not printed, so anything you see is
                a number worth reading. This is the command to type
                when the MACHINE AT THE OTHER END of a cable is
                misbehaving: net says what this monitor did, net stats
                says what the hardware saw. Also spelled net counters.
http serve [port]  start public HTTP pages (default port 8080)
http status        show the server; http stop closes it
                   Plain HTTP, / and /health only; no accounts.
                   Cooperative prompt service with bounded client slots.
net recv <port> <addr> [max-length]
                RECEIVE ONE IMAGE OVER A TCP CONNECTION, into memory
                at that address, and print how many bytes arrived and
                the SHA-256 of what is now in that memory. This is the
                fast way in: the same image over the serial line at
                115200 is about two minutes. The port is decimal and
                the address is hexadecimal; leave the length out and
                the rest of that payload window is the ceiling.
                The listener is opened for ONE connection and closed
                again whatever happens, nothing is booted, and nothing
                is written to the medium - boot mem <addr> enters a
                container and save <name> <addr> <len> writes a file.
                A peer that sends more than the window holds is cut
                off AT the last allowed byte, never past it.
ping <host>     send ICMP echo requests and time the replies, one a
                second, until a key is pressed. The host is a dotted-
                quad, or a name if a DNS server is set. It prints each
                reply's measured round trip and a loss/min/avg/max
                summary at the end.
dns <hostname>  look a name up and print the address(es). Needs a DNS
                server - net dns, or dhcp. Refuses honestly with none.
dhcp            acquire an address by DHCP and report the IP, netmask,
                gateway, DNS server and lease. On success it sets the
                interface's runtime lease, so ping and dns work at once.
                DHCP option 42 is retained per lease for the
                nonblocking SNTP clock service when a server supplies it.
                It does not overwrite saved static addresses or set a
                TFTP server, so get and put still need net server -
                and that one address is all they need.
The four addresses live in the settings store, under net.address,
net.netmask, net.gateway and net.server, so settings save keeps them
and settings load brings them back. Coming from U-Boot, those four are
its ipaddr, netmask, gatewayip and serverip.
tftpboot and tftpput are NOT accepted as aliases, deliberately.
U-Boot takes the address first and the name second and this monitor
takes the name first, to match its own load and save. An alias whose
arguments mean something else is worse than no alias: a line pasted
```

```
  out of a U-Boot session would run, and fetch a file called 0x400000.

SETTINGS THAT SURVIVE A POWER CYCLE
  This is what U-Boot's printenv, setenv and saveenv were for, and those
  three names now work here too, over this same store:
    printenv [name...]         settings show, or the named settings
    setenv <name> <value>      settings set. With no value it does NOT
                               delete (U-Boot does) - use settings
                               remove, so a keystroke cannot quietly
                               destroy a setting.
    saveenv                    settings save
    env print / set / save / load / delete <name>
                               the same, spelled U-Boot's env <sub> way
    env exists <name>          say whether one is set
    env grep <text>            list the ones whose name or value has it

  THE PERSISTED BOOT TARGET lives in this same store, as four names:
    boot.file       the container to boot at start-up, e.g. APP.PMF.
                    Unset, this board goes straight to its prompt and
                    runs nothing by itself - and waits for nothing,
                    because a countdown that can only end one way
                    teaches people to ignore countdowns.
    boot.delay      seconds to count down first. Default 2. Any key
                    during it drops to this prompt, and the failure
                    counter is not touched - nothing was attempted.
    boot.fails      attempts since anything last confirmed a good
                    boot. THE REBOOT-LOOP GUARD. It is written to the
                    medium BEFORE each attempt, not after, because a
                    payload that hangs never gets to an after.
    boot.maxfails   how many attempts before this monitor stops
                    trying and says why. Default 3.
  A payload marks itself good by getting boot.fails back to 0. The
  operator types boot ok; a payload that RETURNS can have the operator
  do it; a payload that keeps the board writes the setting through this
  same store before it takes over - set boot.fails to 0 and save. The
  guard exists because a bootloader that enters a payload automatically
  has invented a way to lose a board: a payload that hangs is re-entered
  every start, and there is then never a prompt long enough to fix it.
    setenv boot.file APP.PMF
    saveenv
  is the whole of arming it.
  A password is still masked by printenv, the same as settings show,
  because this output reaches the HDMI console and the next screenshot;
  settings reveal prints one in full. Up to
  thirty-two names and values, kept in memory and written to a plain
  text file called SETTINGS.TXT in the root of the boot medium. That
  file opens in any text editor on any computer, which is the recovery
  story: a board that will not boot because of something in its
  settings is fixed by pulling the stick and deleting a line.
  settings                              [settings show]
                  list everything. Anything whose name contains
                  password, passphrase or secret is HIDDEN, because
                  this output also goes to the HDMI console and into
                  the next screenshot. The true length is shown.
  settings set <name> <value>
```

```
                the value is the whole rest of the line, so it may
                contain spaces. The input line is 144 characters,
                which is the longest legal one of these - a
                31-character name and a 95-character value - so
                nothing has to be set by editing the file any more.
settings remove <name>
settings reveal <name>
                print one in full, hidden or not. The deliberate
                keystroke that puts a password on the screen.
settings save   write SETTINGS.TXT through this board's file writer;
                availability depends on the mounted medium. The
                whole file is rewritten, so hand-typed comments in it
                are lost.
settings load   read it back. Nothing loads it for you: bringing the
                medium up is slow and noisy and the prompt comes
                first.
settings discard
                a load that hit a bad line stops there and blocks the
                next save, because saving would delete the lines it
                never read. This says throw them away.
wifi            list every network slot, passphrases hidden. wifi
                show is the same command spelled out. FOUR networks
                can be stored and THE SLOT NUMBER IS THE ORDER THEY
                WOULD BE TRIED IN - slot 1 first. Put the one you
                most want joined in slot 1.
wifi ssid <slot> <name>
                the name is the rest of the line, so it may contain
                spaces. One to thirty-two characters.
wifi phrase <slot> <phrase>
                the passphrase for that slot, the rest of the line,
                8 to 63 characters, and STORED IN PLAIN TEXT - the
                command says so at length every time. Give this
                board a guest network. It is a DIFFERENT WORD from
                wifi password because that one takes the whole rest
                of the line, so a slot number in front of it could
                not be told from the start of a passphrase.
wifi find <name>
                which slot holds that name. The comparison is EXACT:
                case counts and a 2.4ghz suffix is part of the name,
                because two real networks here differ by no more.
wifi forget <slot>
                clear one slot, name and passphrase together. The
                others do not move up - the numbers are a preference
                order and renumbering them would change it.
wifi keys       the settings names the slots live under, to hand to
                settings reveal.
The single network this monitor used to keep is still there and is
still READ - LAST, after every numbered slot - so a stick written by
an older build keeps working and no saved passphrase went missing:
wifi network <name>       the name is the rest of the line
wifi password <phrase>    also spelled wifi passphrase
wifi forget     take that pair back out. It does NOT touch the slots.
wifi incidents  every Wi-Fi failure on record (WIFILOG.TXT on the stick):
                what the radio saw before it, saved before recovery,
                and each recovery step and how it ended.
```

```
  wifi rekey ask [group|pair]  ask the access point to rekey right now.
  wifi deaf <s> [ask]  leave the radio unserviced for s seconds, the way a
                  payload does; with ask the access point is asked to rekey
                  first, so it kicks the board while nobody listens.
  wifi iovar <name>  read one 32-bit firmware variable as the radio holds
                  it now (bcn_timeout, roam_off, assoc_listen ...).
  wifi scan       bring the radio up and list the stored networks that
                  are on the air now (needs BRCMFW.BIN, BRCMNV.TXT and
                  BRCMCLM.BLB on the stick).

THE BOARD
  gpio [status] [all | <pin>]
                  report the general-purpose I/O pins: each pin's
                  function, its level read back from the actual pad, and
                  its pull. With no argument it shows the user-header
                  pins; gpio all shows every line the board has; gpio
                  with a pin number shows one. status is the default and
                  may be written out (gpio status, gpio status all, gpio
                  status 21). Reading any pin is always allowed.
  gpio set <pin>
  gpio clear <pin>
                  drive a pin high or low. This makes the pin an output,
                  and says so. The line printed back is the ACTUAL pad
                  read after the write, not what was asked for.
  gpio toggle <pin>
                  invert an output pin. A pin that is not already an
                  output has nothing to invert and is refused - use set
                  or clear to start driving one.
  gpio output <pin>
  gpio input <pin>
                  set the pin's direction, without driving a level.
                  PINS ARE DECIMAL, like clock and unlike the memory
                  commands - a GPIO number is written in decimal on the
                  header and in the datasheet. A leading 0x or $ still
                  forces hex.
                  A WRITE to a pin the board itself is using is refused
                  with a sentence naming the function, not a silent no-op:
                  on this board that is the serial console (GPIO 14/15)
                  and the Wi-Fi/Bluetooth radio's bus (GPIO 34-39). The
                  micro-SD card is on a dedicated socket, not a GPIO, so
                  it needs no such guard.
                  A board with no GPIO says so and does nothing, because
                  Anvil is one monitor across many boards and not every
                  board has pins to report.
  fan [status]
  fan pin <gpio> | none
  fan auto | off | on | <percent>
  fan curve <lowF> <highF>
  fan hyst <degreesF>
  fan hz <n>
                  the fan, the part's own temperature, and the curve
                  between them. EVERY TEMPERATURE HERE IS FAHRENHEIT,
                  typed and printed. fan on its own prints the reading in
                  whole degrees F, with the celsius and the raw
                  millidegrees in brackets so the number can be checked
```

```
                  against a datasheet or another tool; which of the
                  board's two thermometers answered; the mode, the duty,
                  the pin, and the pins the board will drive - a star
                  marks the
                  ones with a real modulator behind them, which are
                  steady at any frequency and are what a four-wire fan
                  wants; the rest are toggled by software from this
                  prompt's own idle loop, which is fine for a two-wire
                  fan on a transistor and is COARSE - it stops
                  modulating while a long command runs and parks the pin
                  at the level the board considers safe.
                  NO PIN IS DRIVEN UNTIL ONE IS NAMED. The board does
                  not know where a fan is wired, so fan pin <gpio> is
                  the first thing to type and it is remembered.
                  auto lets the curve decide: off below the low
                  temperature, ramping to full at the high one, with a
                  few degrees of hysteresis so it does not hunt, a brief
                  full-speed kick to get a stationary fan turning, and
                  FULL SPEED whenever no temperature can be read at all.
                  It starts at off below 113 F, full at 158 F, with 5 F
                  of hysteresis - which is where this part idles quiet
                  and where it still has headroom before the firmware
                  starts slowing it down at 185 F.
                  off and on override it; a bare percentage holds a duty.
                  Degrees, percentages, pins and hertz are all DECIMAL.
                  A CURVE SAVED BY A BUILD FROM BEFORE 2026-09-07 IS IN
                  CELSIUS. It is converted on the way in and the boot log
                  says so; type settings save to write the converted
                  numbers back.
                  Settings are remembered in the table but NOT written to
                  the file until settings save, like every other setting.
                  A board with no modulator, or no thermometer, says so
                  and does nothing.
  i2c probe
  i2c read  <addr> <reg> [count]
  i2c write <addr> <reg> <byte...>
  i2c speed [hz]
                  talk to devices on the header I2C bus (SDA/SCL on
                  GPIO 2 and 3). probe lists every 7-bit address that
                  answers; read points a device at a register and reads
                  count bytes (default 1); write sends a register then
                  one or more data bytes. Addresses, registers, bytes
                  and count are HEX; the speed is DECIMAL, like clock.
                  A register read puts a STOP between the pointer write
                  and the read, not a repeated start, so a device that
                  needs a repeated start will not read right. On a NACK
                  or a wedged bus it says which one happened and that it
                  did nothing. A board with no I2C says so and does
                  nothing. The clock rate shown is derived from the
                  divider source MEASURED on the pin, not the nominal.
(only on a board with TOUCH:)
  touch [points | panel power|prepare|light <n>|reset |
         bus display|header | axes <s> <x> <y>]
                  the DSI panel's touch controller and its power MCU.
                  NOT the bus above: this one is BSC0 on GPIO 44 and 45
```

```
                at ALT1, the display and camera connectors, and it is
                the default here because that is where the panel is
                plugged in. No jumper wires. It puts the header bus
                back when it finishes so i2c never names the wrong
                pair of wires. touch on its own says what answered and
                what it is, and prints the pads before and after the
                mux - both lines low on THIS bus is a cable answer,
                because its pull-ups are on the panel board. panel
                power is the MCU's own probe write; panel prepare is
                the panel's rails and reset; panel light takes a
                DECIMAL 0 to 255 and is NOT inverted on this
                generation; panel reset pulses the touch chip's reset,
                which is a pin on that MCU and on no Pi GPIO - so the
                touch chip cannot answer until panel power has run.
                points reports live coordinates until a key is pressed -
                raw, and where each one lands on the console.
                THERE IS NOTHING TO TURN ON. A finger moves the SAME
                cursor a USB mouse moves and a tap presses the same
                button, with nothing typed, whenever the console is on
                the panel. Where a contact lands is worked out by the
                DISPLAY layer, through the same rotation and scale the
                picture uses, so screen rotate turns the finger with the
                console and needs nothing typed here. axes on its own
                shows the mapping in force, and where the corners land
                once the controller has said how big it is. With three
                0-or-1 numbers it sets the one correction an eye may
                still have to make - which way the touch chip counts
                against the panel it is glued to - and it is SAVED, so
                it is typed once.
mod [list | info <file> | devices | seams | load <file> [seam] |
     unload <file> | detach <seam>]
                driver modules. Not every board has the same hardware,
                so a driver can live in a .MOD file on the card instead
                of inside this monitor. MODULES.TXT in the root, one
                file name per line, loads them at boot - each is
                verified against its own digest, placed in a reserved
                arena, matched to a device this board declares, probed
                without side effects and initialised exactly once. mod
                on its own lists what is loaded, where and in what
                state; devices lists what is here to drive and seams
                lists what each service is answered by. An optional
                seam name NARROWS a module to part of what its header
                declares and can never widen it. unload is REFUSED while
                anything is still using the service - detach asks those
                users to let go first, and they fall back to whatever
                they used before the module was loaded. An unloaded
                module's memory stays reserved until a reset.
mmc [info | read <blk> <addr> [count] | write <blk> <addr> [count]]
                raw block access to the SD card, with no filesystem -
                this is the card underneath mount/load, seen as 512-byte
                blocks. mmc on its own, or mmc info, prints the card's
                class, block size, block count and capacity. mmc read
                copies count blocks (default 1) from the card at block
                <blk> into RAM at <addr>; mmc write copies them the other
                way, from RAM onto the card. ALL THREE NUMBERS ARE HEX -
```

```
                <blk> a block index, <addr> a RAM byte address, <count> a
                number of blocks. The RAM address must be 4-byte aligned
                and a range that touches the monitor is refused, both for
                read and for write. mmc write is destructive and
                unconditional: it overwrites raw blocks, a filesystem
                included, with no undo and no are-you-sure. A board with
                no card controller says so and does nothing; a board that
                can read but not write refuses mmc write and says why.
usb [info | tree | link | storage | read <blk> <addr> [count]]
gnss [watch [s] | raw [s] | source usb|bluetooth|ble|network]   a satellite receiver
https get <url>   fetch a page over TLS 1.3, verified against the root store
tls [roots|reload]   the TLS connection and the root store (2:/tls/roots.pem)
                look at the USB bus. usb, or usb info, lists the host
                controller, the ports and every device that answered
                enumeration - its speed, its VID:PID and its class. usb
                tree draws the topology: controller, root ports, the
                internal hub and its ports. usb link prints the raw
                registers of the link the controller sits on - the
                clock, the trained speed and width, the inbound window
                and the power policy - which is the evidence behind the
                one sentence usb tree summarises them in. usb storage
                reports a
                memory stick's vendor, block size and capacity. usb
                read pulls raw 512-byte blocks off the stick straight
                into memory, without a filesystem: the block number,
                the destination and the count are ALL hexadecimal, and
                the destination is range-checked against the payload
                windows exactly like load. It shows what the stack
                already drives for mount and the cursor; it starts and
                resets nothing. A board with no USB host says so and
                does nothing.
font info
                list the four resident TrueType slots, selected face,
                file sizes and basic metrics. font load <slot> <path>
                reads a static TrueType file of at most 1024 KiB into
                an empty slot only after structural validation; failed
                loads preserve the current selection. font select <slot>
                binds a loaded face; `font select bitmap` returns to the
                built-in face. font unload <slot> frees a slot and
                returns to the built-in bitmap face if it was selected.
baud [rate]
                change how fast this serial line runs. The clock
                feeding the serial port on this board is 48 MHz,
                so 1500000 and 3000000 divide into it exactly -
                thirteen and twenty-six times the default. Reopen
                the port at the new rate and press Enter. If
                nothing answers within eight seconds the board
                puts 115200 back by itself, so a rate your adapter
                cannot manage costs a pause and not a power cycle.
reset                           [r, reboot]
                reset the board immediately. The firmware reloads
                this monitor from the stick. There is no are-you-sure
                question, on purpose: the cost of being wrong is
                a five second reboot into a working loader.
help                            [h, ?, commands]
```

```
                  print this. A question mark on its own does the
                  same thing.

WHERE A PAYLOAD MAY GO
These are the two windows of memory a payload may be loaded into:
  <this board's payload windows, printed at run time>
  The gap between them holds this monitor, the three parked processor
  cores, and a stretch where there is no DRAM at all. A payload may not
  straddle that gap, and one that tries is refused rather than half
  written.
WHAT THIS MONITOR OWNS
  <this board's memory map, printed at run time>
  Every command that writes memory refuses to write into any of it,
  and the refusal names the region.
  Load a payload at the staging address on that line, whatever it was
  built for: an image is the same bytes at any --load-addr, so one
  made last month wants staging here rather than rebuilding. Leave
  512 KiB above the base clear - that is where its own uninitialised
  data goes, and nothing here checks it for you.
  The other window starts at 0x<address>.
  Add --entry-returns if you want the payload to come back to this
  prompt and have its x0 register printed. A stock image never comes
  back at all.
```

Raspberry Pi 4 adds

```
PROGRAMS
  programs             the payload containers (.pmf) on the boot medium
  <name>               a listed name typed on its own runs that program

PI 4 EL3 ACCEPTANCE
  el3test              one bounded secure-timer IRQ, then one controlled BRK
  coretest status      resident secondary-core ownership and last result
  coretest run         16 coherent job rounds on cores 1, 2 and 3
  coretest stop        acknowledge and permanently park those workers
These are explicit diagnostics, never boot actions. After coretest run,
restart before changing cache policy or entering a payload.
```